

DIGITAL ARCHITECTURE LINEAGE

數位建築脈絡

章節閱讀 · 七章敘事

從 1963 年的 Sketchpad 一路讀到 2022 之後的擴散模型 — 電腦如何從一塊更快的製圖板,一步步成為設計的協作者。



起手骨架 v0.1 · 共 7 章

目錄 · CONTENTS

01	電腦走進建築	1960 - 1989
02	形式的解放	1990s
03	參數化的時代	2000s
04	演算法成為設計者	2005 - 2015
05	從設計到製造	2010s
06	BIM 與整合工作流	1990s - 2010s
07	人工智慧的協作者	2018 -

電腦走進建築

1960 - 1989

從 1963 年的 Sketchpad 到 1982 年的 AutoCAD,電腦花了二十年走進建築 — 但它一開始只是一塊更快的製圖板。真正重要的事,藏在少數人提出的一個更大膽的問題裡:電腦會不會不只是工具,而是設計的協作者?

數位建築的故事,常被誤會成始於「用電腦畫圖」。但若只看製圖,這段歷史會無聊得像把鴨嘴筆換成滑鼠。真正的源頭,是 1960 到 1980 年代之間,一群人分別從電腦圖學、控制論、人工智慧與設計理論的角度,問了同一件事:當設計可以被「描述成規則與關係」,而不只是「畫成一張圖」,建築會變成什麼?

這一章談的就是這些伏筆。它們在當時多半沒有蓋成房子,卻悄悄定義了之後六十年的所有發展。

零點:Sketchpad,以及「設計是一套關係」

整條數位建築的軸線,如果要指出一個零點,那會是 1963 年的 Sketchpad。但這套系統的故事,得從 1961 年春天 MIT 林肯實驗室的一場討論說起。當時二十出頭的 Ivan Sutherland 正在電機系攻讀博士,指導教授是日後被尊為「資訊理論之父」的 Claude Shannon;為了找論文題目,他去找了 TX-2 主機的總設計師 Wesley Clark 討論 — 而正是這台機器,決定了 Sketchpad 能成為什麼。TX-2 在 1958 年啟用,是當年罕見的電晶體電腦,擁有六萬四千個三十六位元字的核心記憶體;更關鍵的是,它被刻意設計成一台能「即時互動」的機器,而不是當時主流那種把程式丟進去、隔天才拿到結果的批次主機。沒有 TX-2 這種即時性,就不會有 Sketchpad。

1963 年完成的 Sketchpad,讓使用者拿著一支光筆,直接在圓形的 CRT 螢幕上「畫」。你把光筆按在螢幕上拉動,線條就跟著筆尖延伸 — 這個我們今天視為理所當然的「橡皮筋」即時回饋,在當年是石破天驚的:在那之前,人和電腦之間隔著打孔卡與報表紙,沒有人「看著螢幕、即時地」與電腦來回。Sketchpad 第一次讓繪圖變成一場人與機器的對話。它也因此被同時譽為史上第一套圖形使用者介面、第一套電腦輔助設計工具,以及物件導向程式設計的雛形 — 一個程式,三條科技史主線的共同源頭。

但如果 Sketchpad 的貢獻只是「畫得比較快」,它不會這麼重要。它真正劃時代的地方,在於兩個更抽象的概念。第一個是「約束」(constraints):在 Sketchpad 裡,你不只是畫出一條線,你還能告訴系統 — 這兩條線必須永遠平行、這幾段必須等長、這兩個點必須重合。設定好這些關係之後,當你拖動其中一個元素,系統會自動重新計算,解出一個仍然滿足所有約束的幾何。換句話說,設計者描述的不再是「最終的樣子」,而是「元素之間該維持的關係」,剩下的交給機器去求解。這個從「畫出結果」到「描述關係」的轉向,正是整個數位建築日後反覆出現的核心母題。

第二個概念是「主件與實例」(master / instance):你可以定義一個圖形當作主件,再到處複製出它的實例;而一旦修改主件,所有實例會同步更新。Sketchpad 的第二代甚至支援「遞迴的實例」 — 實例裡還能包含實例,讓複雜的圖樣由簡單元件層層組合而成。為了在 1963 年那麼有限的記憶體裡管理這些彼此參照的關係,Sutherland 還發明了一種「環狀資料結構」來儲存元件之間的拓樸連結。這些聽起來很技術的設計,其實預示了兩件大事:一是物件導向程式設計裡「類別與物件」的關係,二是今天每一套參數化與 BIM 軟體裡「元件庫、族群、實例化」的運作方式。

Sketchpad 的這些能力不是紙上談兵。Sutherland 在論文裡親自示範了好幾個例子,其中最具啟發性的一個是橋樑桁架:畫好桿件、設定接點之間的約束之後,對結構施加模擬荷載,Sketchpad 會即時算出並顯示各桿件的受力與變形。這等於在 1963 年就預演了「設計與結構分析整合在同一個模型裡」的構想 — 而要再過將近半個世紀,這件事才在效能化設計與 BIM 裡真正成為日常工作。它滿足約束的方式,是一種逐步逼近的「鬆弛法»:反覆微調幾何,直到所有條件同時被滿足;這在數學上並不總是有解,Sutherland 也誠實地討論了系統可能卡住的情況。再配合可無段縮放達兩千倍的畫布,使用者能在一塊小小的螢幕上處理遠大於螢幕本身的圖面。把這些示範擺在一起,它們共同傳遞了一個在當時極具顛覆性的訊息:電腦不只能拿來「計算」,它還能拿來「設計」 — 而這,正是後來六十年所有努力共同的前提。

把這兩個概念合起來看,Sketchpad 埋下的是一個對建築影響極深的世界觀:設計不是一張靜態的圖,而是一套關係的系統。一棟建築可以被描述成「柱距等分、窗台對齊樓板、屋頂隨跨距變化」這樣一組規則,而不必一條一條把線畫死。今天當你在參數化設計裡拉動一個滑桿、整個量體跟著重算,或在 BIM 裡改一扇門、所有平面圖與門窗明細同步更新時,你操作的正是 Sketchpad 在 1963 年就定義好的邏輯。數位建築六十年來不斷追問的那個問題 — 設計能不能被描述成規則 — Sketchpad 其實早就給了肯定的答案,只是當時還沒有人意識到它對建築的意義。

值得一再強調的是,Sketchpad 從來不是為建築而生的。Sutherland 想解決的是一般的工程繪圖問題,它的影響也最先在電腦科學裡爆發。年輕的 Alan Kay 在猶他大學讀到 Sketchpad,深受其「以物件與實例組織一切」的想法啟發,日後在全錄 PARC 發展出 Smalltalk 語言與 Dynabook 構想,把物件導向與圖形介面推向全世界。換句話說,我們今天用的視窗、圖示、滑鼠桌面,以及大半現代程式語言的物件思維,都和這套 1963 年的繪圖程式同源。建築只是它眾多下游之一 — 但因為它定義了「人如何與幾何互動」這個最底層的問題,建築怎麼也繞不過它。

Sutherland 本人並沒有停在 Sketchpad。1968 年,他在哈佛與學生做出名為「達摩克利斯之劍」的頭戴顯示器 — 一套會隨使用者頭部轉動而即時更新影像的系統,公認是史上第一套虛擬實境裝置,也預告了今天 VR/AR 在建築呈現與沉浸式設計裡的角色。同年,他與 David Evans 共同創立 Evans & Sutherland,把即時 3D 圖學變成一門產業,並訓練出後來 Pixar、Adobe 等公司的諸多創辦者。1988 年,他以 Sketchpad 獲頒電腦界最高榮譽的圖靈獎。回到主題:Sketchpad 是這條軸線無可爭議的零點 — 此後的 CAD、參數化、演算法、BIM,核心邏輯全都能一路回溯到這台老主機上、那支光筆畫下的第一條會自己保持平行的線。

製圖板上的電腦:CAD 的雙重起源與它的局限

電腦輔助設計(CAD)其實有兩個幾乎同時、卻互不相識的源頭,一個在學院,一個在工廠。學院這端是 Sketchpad;工廠那端,是 IBM DAC-1。DAC-1 的全名是 Design Augmented by Computer,由通用汽車從 1959 年起構想、IBM 在 1960 年加入合作,1963 年投入生產、1964 年在底特律的聯合電腦會議上公開亮相。它跑在 IBM 7090 主機上,工程師用光筆把汽車鈑件的輪廓數位化,再旋轉、縮放、投影,最後輸出到生產線——這是史上第一次,「設計」與「製造」在電腦裡被真正串成一條線。通用汽車一路把這套系統改用到 1970 年代。

如果說 Sketchpad 證明了「人能和幾何即時互動」,DAC-1 則證明了這件事能撐起一整個產業、能省錢、能造車。一個給了概念,一個給了商業現實,兩者後來合流,成為今天整個 CAD 產業的共同祖先。值得注意的是,這兩個源頭都不在建築——一個在電機實驗室,一個在汽車廠。建築其實是 CAD 的「後進者」,它要再等將近二十年,才會被這股數位化浪潮整個捲進去。

DAC-1 還埋了另一條重要的線:它的設計資料能直接輸出去驅動數值控制(NC)工具機。早在 1950 年代末,MIT 的 Douglas Ross 團隊就為 NC 工具機開發了 APT 語言,讓加工路徑能被「程式化」;DAC-1 等於把「設計—加工」這條數位鏈第一次接了起來。這在當年還只是汽車與航太業的事,卻預演了半世紀後數位建築最重要的口號之一——file-to-factory:設計檔案本身就是製造指令。我們會在第五章看到,機器手臂與 3D 列印如何把這個夢想真正帶進建築工地。

而在 CAD 能畫出像樣的建築之前,有一個看似冷僻、卻決定性的難題得先被解決:曲線。早期電腦擅長直線與圓,卻很難精確又可控地描述一條「自由曲線」——偏偏汽車鈑件、船體、機翼到處都是自由曲面。解法來自法國的汽車業。雷諾的工程師 Pierre Bézier 推廣了一種以少數「控制點」定義曲線的方法:你不直接畫出曲線,而是擺幾個控制點,曲線就被它們「拉」成想要的形狀。他把這套數學實作進雷諾的曲面系統 UNISURF,而我們今天就叫它 Bézier 曲線。

但這裡有一段常被遺忘的公案。早在 1959 年,雪鐵龍的數學家 Paul de Casteljau 就獨立發展出同一套曲線數學,而且可能比 Bézier 還早一點;今天計算這類曲線最穩定的遞迴方法,就叫「de Casteljau 演算法」。只是雪鐵龍把它當商業機密鎖著,直到 1974 年才准發表——於是名聲幾乎全歸了願意公開的 Bézier。這個小故事很能說明數位設計史的一條潛規則:「誰先想到」往往不如「誰先公開、誰先被用」重要。Bézier 曲線之所以成為全球標準,正因為它經由 UNISURF 流通出去、被整個產業採用。

曲線之外,還需要「曲面」。這一塊的奠基者是 MIT 的 Steven Coons。他在 1967 年提出後人稱為「Coons patch」的數學方法(那本薄薄的技術報告被暱稱為「小紅書」),用四條邊界曲線就能定義出一塊光滑的曲面。Coons patch 是後來 B-spline、NURBS 等所有曲面技術的源頭——換句話說,1990 年代之後那些連續流動的建築曲面,數學上的祖父就是它。而 Coons 與 Sutherland 同屬 MIT 的互動圖學圈子:

這條「幾何數學基礎」的線索,和 Sketchpad 的「人機互動」線索,其實是同一群人在同一個年代一起鋪下的。

事實上,1960 年代的 MIT 幾乎同時點燃了三把火:Sutherland 的互動繪圖、Coons 的曲面數學,以及 Douglas Ross 那條通往數值製造的 APT。今天你在任何一套 CAD 軟體裡習以為常的功能 — 即時繪圖、自由曲面、輸出加工 — 幾乎都能在那十年的 MIT 找到源頭。可以說,數位設計的整個地基,是在那裡、那段時間一次澆灌完成的。

真正把 CAD 帶進每一間事務所的,是 1982 年的 AutoCAD,而它的關鍵不在技術多先進,在策略。當別的 CAD 還綁在昂貴的大型主機或工作站上時,John Walker 與夥伴創立的 Autodesk 把 AutoCAD 做成能跑在便宜個人電腦上的軟體。隨著 PC 在 1980 年代末、1990 年代普及,AutoCAD 以低價與通用性橫掃市場,十年內就成為幾乎每一間建築與工程事務所的標準配備,終於完成了製圖工具的全面數位化。

AutoCAD 與它的同輩,確實深刻改變了事務所的日常。圖層、區塊、外部參照,讓圖面能被分工、複製、無限修改;晒圖機與描圖紙逐漸退場,通宵趕圖的「製圖室」一間間數位化。但這場效率革命也帶來爭論:當畫線變得這麼快、這麼乾淨,設計者會不會反而少了在筆與紙之間慢慢推敲的那段功夫?「電腦會不會讓建築師畫得更多、想得更少」這個疑問,從 CAD 普及的第一天起就沒消失過 — 甚至到今天的參數化與 AI,它都還在被反覆提起。

但故事的轉折也在這裡。在建築領域,CAD 長期背著一個尖銳的批評:它只是把製圖桌搬上了螢幕。設計者畫得更快、改得更方便、出圖更整齊 — 可是他做的事,本質上還是「畫線」,而不是 Sketchpad 早就示範過的「描述關係」。電腦在建築裡,有將近三十年只是一塊更好用的「電子製圖板」,並沒有真正進入設計的思考。這個落差不是小事;正是這份不滿,後來成了形態探索、參數化、演算法生成等所有運動共同的引擎:大家都想跨過製圖板,把電腦帶回 Sketchpad 當年暗示、卻被產業擱置的那條路 — 讓電腦參與「想」,而不只是「畫」。

更激進的提問:電腦能不能是協作者?

當大多數人還把電腦當成更快的製圖板時,MIT 的 Nicholas Negroponte 在 1967 年與數學家 Leon Groisser 一起創立了 MIT Architecture Machine Group(簡稱 Arch Mac),問了一個當時近乎狂想、今天卻越來越尖銳的問題:電腦能不能不只是被動的工具,而是一個會和設計者對話、會從互動中學習、甚至能提出自己想法的「協作者」?這個提問,把電腦在建築裡的角色,從「手」一口氣拉高到了「腦」。

這個念頭並非憑空而來。1960 年代的 MIT,正是人工智慧最樂觀的年代 — 隔壁 Minsky 與 Papert 的 AI 實驗室,相信機器很快就能像人一樣思考。Negroponte 把這股樂觀引進建築,主張既然機器能下棋、能證明定理,為什麼不能和建築師討論設計?Arch Mac 因此從一開始,就站在建築與人工智慧的交界上 — 這個定位,讓它日後比任何純粹的 CAD 研究,都更接近今天「AI 與設計」的對話。

1970 年,Negroponete 把這套構想寫進《The Architecture Machine》一書。書名本身就是宣言 — 他要的不是「幫人畫圖的機器」,而是一台真正的「建築機器」:能理解人的意圖、能反問、能在來回之間調整自己。在那個連讓電腦好好「對話」都還做不到的年代,這個願景幾乎是科幻;但 Negroponete 把它當成一個正經的工程問題在做,而不只是宣言。

在書裡,Negroponete 講得很具體。他認為電腦最該接手的,是設計中那些「定義明確、卻繁瑣」的部分,好讓人專注在真正需要創造力的判斷上;更進一步,機器應該能觀察設計者的習慣、為使用者建立模型,久了甚至能預測你想要什麼。他還談到「智慧環境」 — 建築本身裝上感測與運算,能回應居住者。這些 1970 年的構想,聽起來活脫是今天「智慧建築」與「設計輔助 AI」的提綱,只是早了半個世紀,而當時的硬體完全跟不上想像。

最接近這個願景的實驗是 URBAN5。它長自 Negroponete 與 Groisser 在 1968 年開設的「電腦輔助都市設計」課程:使用者在螢幕上用方塊堆疊都市量體,系統會用圖形與英文兩種語言回應 — 先給提示,再從你的操作裡推測你的設計準則,最後試著與你討論,扮演一面「反映使用者自身設計邏輯的客觀鏡子」。以今天的標準看它笨拙得可愛,但它是史上第一次,有人認真地把「人和電腦來回討論同一個設計」當成研究對象。

URBAN5 其實會「頂嘴」。當你的操作違反了先前設定的規則 — 比方在不該開口的牆上開了窗 — 它會偵測到衝突、提出抗議,要你解釋或修改。這種「機器主動參與、甚至質疑設計者」的姿態,在 1968 年前所未有,也立刻引來批評:一台機器,憑什麼替設計做價值判斷?這個張力 — 電腦究竟是順從的工具,還是會反對你的協作者 — 正是 Negroponete 想逼出來的問題,也正是今天我們面對生成式 AI 時,得再一次回答的老問題。

不過 Arch Mac 最有名、也最發人深省的作品,其實是一件「失敗」的裝置:SEEK。1970 年,Negroponete 與團隊把它送進策展人 Jack Burnham 在紐約猶太博物館策劃的《Software》展。SEEK 是一個壓克力箱,裡面堆滿小金屬方塊,住著一群沙鼠;一支電腦控制的機械手臂,會試著按照程式「猜測的沙鼠意圖」,把被沙鼠推亂的方塊重新排好。理想中,這是一座會感知居民、即時自我調整的微型城市 — 是 URBAN5 的對話夢想,被搬進了真實的物理世界。

現實則狠狠打了臉。沙鼠根本不照劇本走,牠們的亂撞遠遠超出程式的預期,機械手臂越理越亂,最後整個系統當機收場 — 有評論家形容,展場裡只剩下「在故障手臂前發抖、沾滿自己排泄物的沙鼠」。SEEK 表面上證明了「會回應環境的建築」有多難做,但它真正的價值,是極其誠實地把那個核心難題攤在眾人面前:要機器理解活生生的人(或動物)的意圖,到底有多困難。這個既謙卑又尖銳的問題 — 機器能不能真的「懂」使用者 — 從 SEEK 的沙鼠開始,一路延伸到今天我們對生成式 AI 的所有期待與疑慮。

SEEK 之後,Arch Mac 並沒有退縮,反而把「人機互動」越做越廣。1978 年的 Aspen Movie Map 讓使用者坐在螢幕前「開車」逛遍阿斯彭小鎮 — 被視為今天 Google 街景的遠祖;1979 年的 Put-That-There 則讓人用語音加上手勢同時操作畫面,是多模態介面的先聲。這些作品有個共同點:把電腦從「畫圖的桌子」,

一步步推向「能對話、能感知的環境」。也正是這條路,讓 Arch Mac 在 1985 年順理成章地長成 MIT Media Lab — 內核始終是 Negroponte 最初那個提問:機器能不能真正理解、並協作於人?

Architecture Machine Group 在 1985 年演化成了大名鼎鼎的 MIT Media Lab, Negroponte 本人也成為數位文化的旗手。但對數位建築而言,他在 1967 年問的那個問題 — 電腦能不能是建築的協作者 — 才是真正的遺產。它像一條暗線,穿過後來的參數化、演算法與最佳化,一路潛行到 2022 年之後的生成式 AI;到那時, Negroponte 的提問會以一種他當年想像不到的方式,得到一個既令人興奮又十分棘手的答案。我們會在最後一章,回到這隻沙鼠提出的問題。

控制論與互動建築:來自英國的另一條線

幾乎在 Negroponte 提問的同時,大西洋另一邊的英國,有一條更偏向「控制論」的線索在發展。要理解它,得先回到 1948 年 — 那一年,數學家 Norbert Wiener 出版《Cybernetics》,把「回饋」變成一門正式的科學:一個系統如何透過感知結果、修正行為,來維持自己的目標。這個概念橫掃了從工程、生物到社會的各個領域,也很快滲進建築。如果建築也能「感知使用者、回饋、調整」,那它就不再是凝固的容器,而是一個活的系統。把這個念頭推到最遠的,是英國建築師 Cedric Price。

1961 年前後,Price 與前衛劇場導演 Joan Littlewood 一起構想了 Fun Palace。它不是一棟「建築」,而更像一座「文化機器」:一個巨大的鋼構棚架,裡面的牆、樓板、樓梯甚至天花,全都吊掛在天車上,可以依當天的活動被吊起、移動、重組。今天想上課、明天想看戲、後天想做實驗,空間就跟著變。它刻意沒有固定的立面與形式 — 因為它的「形式」,就是「不斷改變」本身。

讓 Fun Palace 不只是個可動棚架、而真正成為「控制論建築」的,是學者 Gordon Pask。他主持了 Fun Palace 的控制論委員會,把「回饋」做進空間的運作:用一套(在當時還相當概念性的)系統去感知人群的使用,再據此建議空間如何重組。Pask 自己的「對話理論」更主張,學習與互動的本質是一場持續的對話 — 而這正是 Price 想讓建築與使用者之間發生的事。Fun Palace 最終沒有蓋成,卻把「建築是一個會感知、回饋、適應的系統」這個觀念牢牢釘進往後的論述,也直接啟發了 1977 年龐畢度中心那種「外露機械、彈性平面」的想像。

Pask 並不是紙上談兵的理論家。早在 1953 年,他就做過一台叫 Musicolour 的機器:它把演奏者的聲音即時轉成燈光,而且會「習慣」 — 當輸入太規律,它會「覺得無聊」、開始忽略,逼演奏者即興求變。一台會無聊、還會反過來影響使用者的機器,正是控制論最迷人的地方,也是二十多年後 Generator「閒置太久會自己重組」的直系祖先。Pask 把這種「人與機器互相適應」的關係,當成了設計的核心命題。

Price 與 Pask 也不是孤例。同一個 1960 年代的英國,Archigram 那群年輕建築師正用漫畫式的拼貼,想像「插接城市」(Plug-In City, 1964)與「電腦城市」 — 建築像零件一樣可插拔、整座城市由電腦協調。雖然多半是紙上烏托邦,但它們和 Fun Palace 共享同一種信念:在電子與控制論的時代,建築應該是可變的、

會回應的、像生物一樣的系統,而不是一塊永恆的紀念碑。這股想像,後來會在日本新陳代謝派、在高科技建築裡留下清楚的迴響。

這條線最驚人、也最具體的成果,是 1976 年的 Generator Project。Price 為美國佛羅里達一處林間基地,設計了一座由大約 150 個十二英尺見方、可移動可組合的方塊組成的退修中心;負責運算的,是 John Frazer 與 Julia Frazer 夫婦。Generator 不只是「可以被重新配置」— Frazer 為它寫的程式,讓這座建築具備了某種主動性:當它被閒置太久、長期維持同一種配置時,它會「感到無聊」,主動提出重新排列的建議。

請仔細體會這件事的年代感:1976 年,個人電腦都還沒上市。當主流產業才刚开始把 CAD 當成更快的製圖板,Price 與 Frazer 已經在認真設計一座「會自己思考、會無聊、會主動求變」的房子。也因此,John Frazer 後來把 Generator 稱為「史上第一座智慧建築」— 它把 Negroponte 那個「機器作為協作者」的提問,從螢幕裡的對話,推進到了真實空間的自我調整。SEEK 的沙鼠失敗了,但 Generator 在概念上更進一步:不是機器替你整理積木,而是建築本身參與了居住的決策。

為什麼這條「控制論—互動」的線這麼重要?因為它提出的問題,和 CAD 那條「畫得更快」的主流,根本是相反的方向。CAD 想讓設計者更有效率地產出一個「最終、固定」的設計;而控制論建築問的卻是:設計能不能根本不要有「最終」狀態,而是一個永遠在回應、在變化的過程?這個哲學上的分歧,會在日後反覆出現— 從互動建築、反應式立面,一直到今天會隨資料即時生成、調整的「活的」建築。

這也解釋了為什麼這條線在當時注定邊緣。1970 年代的營造體系、法規、造價與工法,全都建立在「房子是固定的」這個前提上;一座會自己動、會自己改的建築,既難蓋、難管、也難賣。控制論建築其實是太早了— 它需要的硬體(便宜的感測器與致動器)、軟體(即時運算),甚至社會條件(願意住進「會變的房子」的人),都還沒就位。於是它先成為一種思想,靜靜地等待技術追上自己。

在當時,這幾乎是一段「失敗者」的歷史:Fun Palace 沒蓋成,Generator 沒蓋成,控制論建築在 1980 年代一度沉寂。但它埋下的種子異常頑強。John Frazer 把它帶往演化式建築(就是開頭提過、他 1995 年的《An Evolutionary Architecture》);到了 2000 年代,當感測器、運算與致動器終於便宜到位,Price 與 Pask 當年的夢— 一座會感知、會回應、會與你對話的建築— 才開始以「互動建築」「反應式環境」之名,真正站了起來。

把設計變成運算:文法、模式與邏輯

如果說前面幾條線在問「電腦能幫設計做什麼」,還有一群理論家在問一個更根本的問題:設計本身,能不能被寫成一套可運算的規則?在 1960、70 年代,這個提問同時在好幾個地方冒出來,而且大多帶著濃濃的數學味。

1971 年,George Stiny 與 James Gips 提出了「形狀文法」(shape grammar)。它的想法既單純又激進:就像語言有文法、能用有限的規則生成無窮的句子,形狀也可以有一組「形狀規則」 — 看到符合條件的圖形,就把它替換成另一個圖形 — 反覆套用,就能生成,或反過來分析,一整類設計。它等於把喬姆斯基的生成語法,搬到了視覺與空間上。

形狀文法最有名的示範,是 Stiny 與 William J. Mitchell 在 1978 年做的「帕拉底歐文法」:他們把文藝復興大師帕拉底歐的別墅平面,拆解成一組生成規則,再用這組規則,生成出一批「帕拉底歐從沒畫過、卻完全符合帕拉底歐風格」的全新平面。這是個震撼的概念證明 — 它暗示:風格不是玄學,而可能是一組能被描述、甚至能被執行的規則。設計第一次被當成一種可以形式化、可以生成的語言,而不只是天才的靈光一閃。

形狀文法的威力,遠不只帕拉底歐。後來的研究者用它分析、生成中國傳統窗櫺的冰裂紋、印度蒙兀兒花園的平面、萊特的草原住宅 — 一種風格只要能被歸納成幾條替換規則,電腦就能不斷生成出「同款的新作」。更深刻的是,形狀文法堅持「用形狀本身來運算」,而不是先把形狀翻成符號或數字再算;它逼我們承認,視覺與空間有自己的一套邏輯,值得被當成一種語言來研究。這個立場,到今天的生成式設計裡仍然是個尚未解決的根本問題。

與形狀文法平行、卻走得更廣的,是 Christopher Alexander。1964 年的《Notes on the Synthesis of Form》裡,他就試圖用數學(集合論、圖論)把複雜的設計問題拆解成可處理的子問題;1965 年那篇著名的〈A City is Not a Tree〉則反過來警告:真實的城市是錯綜交織的網絡,不能被簡化成乾淨的樹狀層級。1977 年的《模式語言》是他的集大成 — 他主張好的設計可以拆解成兩百多個「模式」,每個模式描述「在某情境下、針對某問題、給出一個經過驗證的解法」,而這些模式能像語言一樣彼此組合,讓一般人也能參與設計。

耐人尋味的是,這套源自建築的思想,後來對「軟體」的影響甚至超過了對建築本身。1987 年,程式設計師 Kent Beck 與 Ward Cunningham 受 Alexander 啟發,把「模式」的概念帶進程式設計;1994 年,著名的「四人幫」(Gang of Four)出版《Design Patterns》,把設計模式變成全球軟體工程師的共同語言,連分類方式都幾乎照搬 Alexander。甚至今天維基百科賴以運作的 wiki,最初都是 Ward Cunningham 為了整理這些模式而發明的。一位建築師,意外成了現代軟體工程的祖師爺之一 — 這大概是「設計即規則」這個想法,最出人意料的一次開花。

有趣的是,Alexander 本人對軟體界的熱情並不全然領情。他真正在意的,是建築能不能讓人感到「生機」與「完整」 — 一種他晚年用「生命的結構」去描述、卻很難量化的品質。他甚至覺得,軟體工程師抓住了「模式」的形式,卻錯過了他最在乎的那個靈魂。這個落差點出一個貫穿全書的張力:當設計被化約成規則,那些無法被規則捕捉的東西 — 氛圍、生機、難以言說的好 — 會不會就此流失?這正是日後每一波運算式設計,都得反覆面對的質疑。

在英國,Lionel March 則從另一個角度推進同一件事。他與 Philip Steadman 在 1971 年的《The Geometry of Environment》,把組合數學、圖論與對稱理論引進建築與都市的空間分析,主張平面、量體、土地使用都能用嚴謹的數學來描述與枚舉。他在劍橋推動的研究,讓「用數學與電腦研究建築形態」成為一個正當的學術領域 — 這正是後來計算式設計教育的搖籃之一。

到了 1990 年,William J. Mitchell 的《The Logic of Architecture》把這些零散的努力收攏成體系。他系統性地論證:建築形式可以被視為一種「語言」與「運算」 — 有詞彙、有文法、有可被推理檢驗的邏輯。這本書幾乎把前面二十年的所有嘗試(形狀文法、模式、形態數學)整合成一套能教、能學的理论,為日後全世界的計算式設計課程奠定了底盤;Mitchell 本人後來也在 MIT 成為這個領域的領袖。

還有一個問題值得想:這些 1970 年代就成熟的理論,為什麼要等三十年才真正普及?答案是「門檻」。形狀文法、模式語言當年都得靠手工推演或艱深的程式,只有少數學者玩得動;它們是強大的「思想」,卻不是好用的「工具」。要等到 2000 年代,Grasshopper 把寫規則變成拉節點、人人都能上手,這些理論才從論文走進設計課與事務所。思想先行、工具殿後 — 這個時間差,本身就是數位建築史一再重演的母題。

這一節的這幾個人,沒有一個是在「畫圖」。他們做的,是把「設計」這件抽象的事,翻譯成規則、模式、文法與邏輯 — 也就是把設計,變成電腦聽得懂、甚至能自己執行的東西。這正是 Sketchpad 那個「設計是一套關係」的暗示,在理論層面的全面展開。當 2000 年代 Grasshopper 讓每個學生都能「拉節點、寫規則」時,他們其實是在無意間,實現這群理論家早在四十年前就寫下的願景。

資料的伏筆,與第一次實戰

這一章的最後,要埋下幾條看似零散、卻分別預告了數位建築後半段的伏筆:把建築存成資料、用運算撐起真實工程,以及一個太早出現、幾乎被遺忘的「參數」先知。

第一條伏筆是「資料」。1974 年,任教於卡內基美隆的 Charles Eastman 提出了 Building Description System(BDS)。他的主張在當時近乎異端:建築不該被存成一疊互不相干的圖紙,而該被存成一個描述「整棟建築」的資料庫 — 每一道牆、每一扇門,都是一筆有屬性、彼此關聯的資料。

這樣做的好處是革命性的:你改動其中一處,所有相關的平面、立面、剖面與明細表都會自動跟著更新 — 因為它們本來就是同一筆資料的不同呈現。這正是 Sketchpad 「主件 / 實例」邏輯,在整棟建築尺度上的放大。但在 1974 年,電腦又貴又慢,這個想法太過超前,幾乎沒有人採用。它沉睡了將近三十年,直到電腦足夠強大、產業足夠成熟,才以另一個名字醒來 — BIM。Eastman 也因此被公認為「BIM 之父」,我們會在第六章好好講這個故事。

Eastman 當年描述的能力,放到今天看簡直就是 BIM 的規格書:自動產生各種圖面、計算材料數量、甚至檢查不同構件之間有沒有「打架」(碰撞檢查),他後來也真的做出了原型系統。問題是,1970 年代的 CAD

廠商全都忙著賣「電子製圖板」——因為畫圖是建築師當下就肯付錢的需求,而「把建築存成資料庫」的好處,要等整個產業的流程改變才看得到。於是最好的點子,輸給了最好賣的產品,一輸就是三十年。

第二條伏筆是「運算第一次撐起真實工程」,主角是雪梨歌劇院。1957年,當時沒沒無聞的丹麥建築師 Jørn Utzon,以一組詩意的殼體草圖贏得競圖。問題是:那些自由起伏的殼,沒有人知道怎麼算、怎麼蓋。接下這個難題的,是工程顧問 Ove Arup 的團隊,他們和 Utzon 一起苦撐了好幾年。

轉機出現在 1961 年:他們找到了著名的「球面解」——讓每一片殼,都取自同一顆半徑約七十五公尺的球面,如此一來所有殼片的曲率一致,就能用重複的預鑄構件來建造。而為了驗算這些前所未有、又彼此牽連的結構,Arup 動用了 Ferranti Pegasus 等早期電腦,累積了大約兩千個機時——這是史上最早把電腦用於建築結構分析的案例之一。Utzon 後來在 1966 年因與當局衝突而憤然離開,沒能親眼完成它;但 1973 年落成時,沒有人會說它是「數位建築」,它卻是電腦第一次,真正撐起了一座地標。

雪梨歌劇院不是孤例,只是最戲劇化的那一個。同一個年代,工程界正悄悄被電腦改變:有限元素法(FEM)讓複雜結構第一次能被逐塊算清楚,薄殼、網殼、張力結構等大膽嘗試於是成為可能。值得一提的是,德國的 Frei Otto 走的是另一條「類比運算」的路——用肥皂膜、懸鏈模型,讓物理自己「算」出最省力的形狀。數位與類比在這個年代其實是並行的;但長期來看,能被精確記錄、複製、自動化的數位運算,終究成了主流。

第三條伏筆最讓人意外,因為它太早了。早在 1940 年代,義大利建築師 Luigi Moretti 就提出了「architettura parametrica(參數化建築)」這個詞;1957 年他成立 IRMOU 研究所,專門研究電腦能為建築做什麼,並與機率論大師、數學家 Bruno de Finetti 合作。1960 年,他在第十二屆米蘭三年展上展出一系列體育場設計——那些造型,是用大約十九項參數(觀眾視角、混凝土造價等)透過一台 IBM 610 電腦計算出來的。換句話說,早在 Grasshopper 出現的半世紀前,Moretti 就已經在做「設定參數、讓電腦算出形狀」的參數化設計。只是時代還沒準備好,他的成果長期被遺忘,直到參數化在 2000 年代爆紅,人們才回頭驚覺:原來這位「參數化建築之父」早就來過。

Moretti 的故事,其實是這整章的縮影。好幾個關鍵概念——參數、智慧建築、建築資料庫——都在 1960、70 年代就被想出來了,卻因為太早、太難、或太不合時宜而被埋沒,要等幾十年後技術成熟,才被「重新發明」並冠上新名字。讀數位建築史,常會有種既視感:今天看似前衛的新潮,很多其實是當年那些伏筆,隔了半世紀傳回來的回聲。

還有一條「效能」的線。1969 年前後,英國史崔克萊大學在 Tom Mather 主導下成立了 ABACUS Strathclyde 研究群,把電腦用了一個和製圖、也和造形都不同的方向:設計評估。他們開發工具,讓設計者能在設計早期就用電腦預測一個方案的採光、能耗、造價與動線表現——也就是「在蓋之前,先知道它好不好」。這條把電腦當成「分析與決策夥伴」的路線,日後會在永續設計、效能化設計與今天的模擬驅動設計裡,一再重現。

到 1980 年代末回頭看:電腦確實走進了建築,但大多數時候,它還只是那塊更快的「電子製圖板」。然而所有的種子 — 把設計看成關係(Sketchpad)、把電腦當協作者(Negroponte 與 Arch Mac)、讓建築會感知會適應(Price 與 Pask)、把設計寫成規則(Stiny 與 Alexander)、用參數生成形狀(Moretti)、把建築存成資料(Eastman)、用運算撐起真實結構(雪梨歌劇院) — 都已經埋下。它們大多沒有在當下開花,有些甚至被當成失敗或怪談。但接下來的十年,當動畫軟體闖進建築學院,第一個掙脫束縛的,將會是「形式」。

來源

1. Sketchpad — Wikipedia
<https://en.wikipedia.org/wiki/Sketchpad>
2. DAC-1 — Wikipedia
<https://en.wikipedia.org/wiki/DAC-1>
3. Nicholas Negroponte, The Architecture Machine (1970, PDF)
https://monoskop.org/images/1/1f/Negroponte_Nicholas_The_Architecture_Machine_1970.pdf
4. URBAN5 — The Architecture Machine (MIT Press)
<https://mitp-arch.mitpress.mit.edu/pub/m27d3vdh/release/1>
5. From the Fun Palace to the Generator: the conception of the first intelligent building (ARQ / SciELO)
https://www.scielo.cl/scielo.php?script=sci_arttext&pid=S0717-69962015000200007
6. Shape grammar — Wikipedia
https://en.wikipedia.org/wiki/Shape_grammar
7. A Pattern Language — Wikipedia
https://en.wikipedia.org/wiki/A_Pattern_Language
8. Building information modeling (history) — Wikipedia
https://en.wikipedia.org/wiki/Building_information_modeling
9. The spherical solution — Sydney Opera House
<https://www.sydneyoperahouse.com/our-story/the-spherical-solution>
10. Prof Tom Maver (ABACUS) — University of Strathclyde
<https://www.strath.ac.uk/staff/mavertomprof/>

形式的解放

1990s

1990 年代,建築的主流語彙在短短十年內,從稜角分明的碎裂,翻轉成連續流動的曲面。推動它的,是一個哲學概念(摺疊)、兩種數位工具(航太軟體與動畫軟體),以及一群想讓形式重新「長出來」的建築師。

如果說萌芽期是電腦悄悄潛入建築的三十年,1990 年代,就是它第一次真正改變了「建築長什麼樣子」。

這一章談形式如何被解放 — 從解構主義的碎裂,走向摺疊與 blob 的連續曲面;也談這場解放背後,理論的渴望與數位工具的能力,如何在同一個十年裡一拍即合。

從解構到摺疊:一場轉向

如果說萌芽期是電腦悄悄潛入建築的三十年,1990 年代則是它第一次真正改變了「建築長什麼樣子」。而且改變來得又快又猛:短短十年內,建築的主流語彙,就從稜角分明的碎裂,翻轉成連續流動的曲面。這場轉向的名字,叫「摺疊」(folding)。

要理解摺疊,得先看它反抗的對象。1988 年,紐約現代美術館 (MoMA) 辦了一場影響深遠的展覽《Deconstructivist Architecture(解構主義建築)》,由 Philip Johnson 與 Mark Wigley 策劃,集結了 Peter Eisenman、Frank Gehry、Zaha Hadid、Libeskind、Koolhaas、Tschumi 與 Coop Himmelblau 等人。解構主義的形式語言,是衝突、碎裂、傾斜、互相穿刺 — 它刻意讓建築看起來不穩定、不和諧,以對抗現代主義那種純淨的秩序。

但碎裂與衝突,很快讓一些建築師感到不滿足:難道前衛只能是「把東西打碎」嗎?有沒有一種方式,既能容納差異與複雜,又不必訴諸對立與斷裂?答案,來自一位法國哲學家。

1988 年,Gilles Deleuze 出版《Le Pli:萊布尼茲與巴洛克》(英譯《The Fold》於 1993 年問世)。在書裡,「摺疊」是一個迷人的概念:物質與靈魂像一塊不斷對摺、再對摺的布,差異不是被切開、對立,而是被「摺進」同一個連續體裡。對厭倦了解構碎裂的建築師來說,這簡直是一把鑰匙 — 它提供了一種「連續地容納差異」的可能。

最早把摺疊認真做進建築的,是 Peter Eisenman。在 1990-91 年的法蘭克福 Rebstockpark 規劃案裡,他借用數學家 René Thom 的「突變理論」,把基地的紋理像一張紙一樣折曲、變形,讓建築與地景連成一個起伏的連續面。這不再是解構那種尖銳的碰撞,而是平滑的、漸變的彎折。Eisenman 證明了:摺疊不只是哲學隱喻,它能變成具體的形式操作。

摺疊不是一兩個人的事,它有一個熱鬧的理論現場。1991年起,Eisenman 與一群建築師、哲學家辦了一系列名為「Any」的國際會議(ANY、Anyone、Anywhere…),Deleuze 式的思想、拓樸學、複雜性科學在這裡被反覆討論與辯論。哲學家 John Rajchman、理論家 Sanford Kwinter 等人,把「連續」「差異」「事件」這些概念,一次次譯成建築能用的語言。這股理論熱,正是摺疊與 blob 能在學院裡迅速燎原的溫床。

真正讓「摺疊」成為一場運動的,是 1993 年的一本雜誌。那一年,年輕的 Greg Lynn 為《Architectural Design》編了一期影響深遠的專輯《Folding in Architecture》。他在其中的文章主張:建築該追求「curvilinearity(曲線性)」— 用平滑、柔順、連續的形式去整合差異,取代解構主義那種對立的拼貼。這期專輯收錄了 Deleuze、Jeffrey Kipnis、Eisenman、Gehry 等人的文字與作品,等於一次幫一整代建築師,把「摺疊」這個關鍵字釘上了檯面。

值得一提的是,這場轉向也悄悄改寫了某些人的生涯。Zaha Hadid 在 1988 年的解構展上,還是以尖銳、爆裂的繪畫聞名;但進入 1990 年代,她筆下的形體越來越流動、連續,最終在下一個十年,成為參數化最耀眼的代言人。從解構到摺疊,不只是一次風格的更替,更是一整代前衛建築師,集體把「形式的理想」從『斷裂』改寫成『流動』的過程。

圍繞在 Lynn 身邊的,是一群擅長把哲學翻成建築語言的理論家,Jeffrey Kipnis 是其中健將。他主張新建築應該追求一種「單一、連續、卻內含異質」的整體— 用今天的話說,就是一個光滑外皮底下,藏著高度複雜的東西。這套論述,為摺疊與隨後的 blob,提供了強大的理論正當性,也讓「連續」與「平滑」一夕之間,從保守的代名詞變成了前衛的標誌。

這裡有一個關鍵的巧合:正當建築師在理論上渴望「連續、平滑、可漸變的形式」時,能畫出這種形式的數位工具— NURBS 曲面、動畫軟體— 也剛好成熟了。理論的渴望與工具的能力,在 1990 年代初一拍即合。沒有 Deleuze,電腦的曲面只是炫技;沒有電腦,Deleuze 的摺疊只能停在紙上。是這兩者的相遇,真正點燃了形式的解放。

這場轉向也悄悄改變了設計的「起點」。建築師不再總是從機能或類型出發,而越來越常從一個抽象的「圖解」(diagram)— 一條力線、一次摺疊、一個拓樸關係— 開始,讓形式從這個生成性的圖解裡長出來。設計的第一步,從「畫一張平面」,變成了「定義一套生成的邏輯」。這個觀念上的位移,會直接通往下一個十年的參數化。

當然,摺疊也很快迎來批評。當「連續曲面」變成一種人人爭相模仿的視覺風格,有人質疑:它是不是只剩下造型、丟掉了原本的哲學深度?一條漂亮的曲線,真的就比一個直角更能「容納差異」嗎?這個「形式淪為風格」的隱憂,其實會跟著數位建築走很久— 每一次新工具帶來新形式,幾乎都會被再問一次同樣的問題。但無論如何,1990 年代初,連續曲面已經奪下了前衛的話語權。

於是 1990 年代初,舞台佈置完成了:一邊是渴望連續與差異的理論,一邊是終於能造出連續曲面的工具。接下來的故事,會由兩個人、兩條路徑同時展開— 一個用航太軟體,把巨大的曲面真的蓋起來(Frank Gehry);

另一個把動畫軟體,變成生成形式的引擎(Greg Lynn)。我們先看前者。

Gehry 與 CATIA:複雜形體第一次蓋得起來

如果摺疊是 1990 年代的理論,那 Frank Gehry 就是讓全世界相信「複雜形體真的能蓋」的那個人。有趣的是,Gehry 本人對 Deleuze、對艱深理論幾乎不感興趣 — 他是個徹頭徹尾的「手作」建築師,用瓦楞紙、布料、揉皺的紙團做模型,追求一種雕塑般的、流動的造型。問題是:這種隨手捏出來的形狀,要怎麼精確地畫成施工圖、又怎麼真的蓋出來?

1989 年,Gehry 在德國完成了 Vitra 設計博物館 — 一棟白色的、扭轉碰撞的小建築。它的曲面當時還是靠工匠手工放樣硬蓋出來的,過程既昂貴又充滿不確定。Gehry 很清楚:如果想做更大、更複雜的案子,光靠手工放樣遲早會破產。他需要一種方法,把指間捏出的造型,精確地轉譯成可以施工、可以估價的資料。

答案,來自一個意想不到的地方:航太業。1992 年的巴塞隆納奧運,Gehry 做了一座巨大的金屬「魚」雕塑(巴塞隆納魚)。為了把這條扭曲的魚精確造出來,他的團隊第一次採用了 CATIA — 一套原本由法國 Dassault(達梭)公司開發、用來設計戰鬥機與飛機曲面的軟體。他們把實體模型用三維方式數位化,在 CATIA 裡建出精確的曲面模型,再輸出施工資料。這是 CATIA 第一次被用在建築上。

這套流程最關鍵、也最反直覺的一步,是「數位化」。Gehry 團隊會用一支三維感測探針(早期甚至借用航太業的掃描設備),把實體模型表面成千上萬個點的座標,一點一點讀進電腦,再在 CATIA 裡重建出對應的曲面。換句話說,電腦不是用來「想」出形狀,而是用來「精確地記住」Gehry 用手做出來的形狀,並把它變成可運算、可加工的資料。在這裡,手與機器是接力,而不是取代。

CATIA 帶來的,是一套全新的工作流。Gehry 事務所的設計仍然從手作模型開始 — 這一點他始終沒變 — 但模型一旦定案,就被精密地掃描進電腦,變成一個帶有精確座標的數位主模型。從這個主模型,可以自動算出每一塊曲面板的形狀、每一根結構桿件的位置、每一個接點的角度。設計與營造之間那道最危險的鴻溝 — 「圖紙畫得出來、現場卻蓋不出來」 — 第一次被一個數位模型填平了。

這一切,在 1997 年的畢爾包古根漢美術館達到頂點。那些覆滿鈦金屬、像船帆又像花瓣般翻捲的量體,在 CATIA 裡被精確定義,三萬多片鈦板,每一片都由電腦算出形狀、編號與定位。落成那一刻,全世界都被震撼了:原來電腦真的能讓這麼瘋狂的形狀,準時、又不嚴重超支地蓋起來。

畢爾包也不是孤例。Gehry 早在 1991 年就設計、卻拖到 2003 年才完工的洛杉磯華特迪士尼音樂廳,同樣靠 CATIA 馴服了那些翻捲的不鏽鋼曲面。一連串這樣的案子,讓「數位營造複雜曲面」從一次性的奇觀,變成一套可重複、可信賴的方法 — 這正是它能真正改變產業,而不只是留下一座網紅地標的關鍵。

畢爾包的衝擊是雙重的。對大眾,它帶來了「畢爾包效應」 — 一座建築,竟能讓一個沒落的工業城翻身成國際觀光勝地。對建築界,它傳遞了一個更深的訊息:複雜、自由的曲面形體,不再只是渲染圖裡的幻想或

紙上的烏托邦,它是可以被精確建造的現實。數位工具第一次,不是用來「畫得更快」,而是用來「蓋出原本根本蓋不了的東西」。

畢爾包還示範了一件影響深遠的事:從數位模型,可以直接驅動製造。鈦板與鋼構桿件的加工資料,全都從同一個 CATIA 模型輸出 — 這正是萌芽期 DAC-1 那條「設計到製造」的線,第一次在建築的尺度上、以這麼壯觀的方式被實現。它預告了第五章「數位製造」的整個時代:當設計檔案就是生產指令,建築的複雜度,就不再被「畫圖與放樣的人力」綁死。

Gehry 的故事裡,藏著一個耐人尋味的矛盾。他用的是最尖端的數位工具,出發點卻是最原始的手作 — 揉紙、剪布、堆瓦楞紙板。對他來說,電腦從來不是用來「生成」形式的(這一點,和我們下一節要談的 Greg Lynn 完全相反),而是用來「實現」他手作的直覺。這也提醒我們:數位建築從一開始,就存在兩種截然不同的態度 — 電腦,究竟是「造型的工具」,還是「造型的作者」?這個分歧,會貫穿之後的每一章。

畢爾包還讓一個古老的角色悄悄回來:「master builder(營造大師)」。Gehry 主張,正因為有了精確的數位模型,建築師才能重新掌握對營造的主導權 — 不必再把複雜形體的責任,丟給承包商去猜。但批評者並不買單:他們質疑 Gehry 的造型太過任性、雕塑性蓋過了建築性,數位工具只是讓「建築師的個人簽名」變得更巨大、也更昂貴。「這到底是建築,還是放大的雕塑?」這個爭論,至今沒有定論。

畢爾包之後,Gehry 事務所把這套流程系統化,甚至成立了 Gehry Technologies,把他們基於 CATIA 發展的工具(後來叫 Digital Project)推向整個產業。複雜曲面的數位營造,從 Gehry 一家的獨門武器,慢慢變成大型事務所都能取用的方法。一個建築師的造型執念,意外地推動了整個營建業的數位化。

Gehry 證明了數位工具能把「任何形狀」蓋出來,但他刻意迴避了一個問題:這些形狀「從哪裡來、又意味著什麼」?對他,答案是個人的雕塑直覺。但同一個十年,另一群更年輕的建築師,正想讓電腦來回答這個問題 — 讓形式不是用手捏出來的,而是被「算」出來、被「長」出來的。而他們的工具,不是航太軟體,是好萊塢的動畫軟體。

Maya 闖進建築學院:動畫變成設計工具

Gehry 用電腦「實現」手作的形狀;但在同一個十年,一群更年輕的建築師想做相反的事 — 讓電腦「生成」形狀。他們的祕密武器,不是航太軟體,而是好萊塢用來做動畫特效的軟體。領頭的,是 Greg Lynn。

Greg Lynn 原本是哲學系出身,又在 Eisenman 事務所待過,理論底子深厚。1990 年代,他在哥倫比亞大學的「無紙工作室」(Columbia Paperless Studio)教書 — 那是院長 Bernard Tschumi 在 1994 年前後推動、要求學生完全用電腦設計的實驗性工作室。在那裡,一群人開始把原本給電影、給動畫用的軟體,搬來做建築。

動畫軟體和 CAD 根本是兩種世界。CAD 想的是「精確、靜態、可施工」;動畫軟體想的卻是「運動、時間、變形」。它內建了 CAD 沒有的東西:時間軸、關鍵影格、會吸引或排斥物件的「力場」、會流動的粒子、像水銀一樣能融合的「變形球」(metaball)。對 Lynn 來說,這些都不是特效,而是一整套全新的、用來「生成形式」的詞彙。

Lynn 把這套想法寫成 1999 年的《Animate Form(動態形體)》。書的核心主張很激進:形式不該被「畫」出來,而該被「動態地生成」出來。設計者先佈置一個充滿力的環境 — 風、重力、吸引子、流場 — 再放進一些柔軟的幾何,讓它們在這些力的作用下被推擠、拉扯、變形,最後「定格」在某個瞬間,那一格就是建築的形。形,成了力與時間留下的痕跡。

舉個例子:設計一棟濱海建築,Lynn 可能會在電腦裡把海風、人流、視線都模擬成「力」,放進一團 NURBS 曲面,讓它在這些力場裡被吹拂、被擠壓,長成一個沒有人能徒手畫出、卻又對環境「有反應」的流線形體。這和 Gehry 的手作雕塑完全不同 — 形狀不是設計師主觀捏的,而是一套力學邏輯「算」出來的結果。

這套方法最完整的展演,是 Lynn 1998 年起發展的「胚胎住宅」(Embryological House)。它不是一棟房子,而是一個「住宅的家族」:一套參數化的曲面邏輯,只要調整參數,就能長出成千上萬棟「彼此相似、卻沒有兩棟相同」的房子。每一棟都像同一個物種的不同個體 — 這直接呼應了之後會談到的「大量客製化(mass customization)」:工業化的效率,加上每一件都不一樣的個性。

讓這一切成為可能的底層技術,是 NURBS 曲面。它能用少數控制點,精確描述任意平滑的自由曲面 — 是 1960 年代 Bézier 曲線、Coons 曲面那條數學線,長到 1990 年代的成熟果實。動畫軟體之所以能拿來做建築,正因為它和建築共用了 NURBS 這套「連續曲面」的語言。萌芽期那條看似冷僻的曲線數學,到這裡終於開花。

Lynn 那一代用的軟體,名字今天聽來有點陌生:Wavefront、Alias、Softimage、後來合併成的 Maya,以及 form·Z、3D Studio MAX。它們本是為了電影、廣告、遊戲而生;建築師等於「借用」了娛樂工業的算力與工具,來做完全不同的事。這種「把別人家的工具搬來用」的挪用,本身就是數位建築反覆出現的策略 — 從動畫軟體,到後來的遊戲引擎、AI 繪圖,一路如此。

把 Gehry 和 Lynn 並排,正好框出數位建築的兩極。Gehry 說:形狀是我捏的,電腦幫我蓋。Lynn 說:形狀是力場長出來的,我只負責佈置條件。一個是「作者用工具」,一個是「作者設定系統、讓系統生成」。後者 — 設計者退一步,去定義一套生成的規則或環境,再讓形式自己長出來 — 正是下一個十年「參數化」與「演算法生成」的核心姿態。Lynn 其實已經站在那道門檻上了。

哥倫比亞的無紙工作室,效應遠超過幾棟 blob。它訓練、感染了一整代建築師 — Hani Rashid、Greg Lynn 等人從這裡走出去,把「電腦不只是繪圖、而是設計思考的媒介」這個信念,帶進各自的事務所與學校。一個學院的教學實驗,等於替整個行業預演了二十年後的常態:再也沒有人「不用電腦設計」。

當然,blob 也招來猛烈的批評。反對者說,這些流線形體華麗卻空洞,是「為了曲面而曲面」的形式主義;它們在螢幕上很美,落地卻常常難蓋、難用、難維護。更尖銳的質疑是:當形狀是力場「算」出來的,設計者到底還剩下多少責任與判斷?「調一下參數就生出一個造型」,這算設計,還是只是擲骰子?這些問題,Lynn 那一代並沒有完全答出來,但他們確實把它們第一次擺上了檯面。

無論評價如何,Lynn 與無紙工作室做成了一件不可逆的事:他們把「生成」這個動詞,正式帶進了建築設計。在他們之後,「設計一個形狀」和「設計一個會生成形狀的系統」,變成了兩件不同的事 — 而後者,從此成為數位建築最核心、也最迷人的能力。

但要真正理解這些流動的形體,我們還得回頭看一眼它們共同的身體:那層連續、平滑、能無限變形的「皮」。它的數學叫 NURBS,它的美學叫 blob — 而圍繞著 blob,1990 年代末打了一場到今天都還有餘波的仗。

NURBS 與 Blob:連續曲面的數學與美學

前面一直提到 NURBS 與 blob。這一節,我們把這兩個關鍵字拆開來看 — 一個是讓連續曲面成真的數學,一個是它催生的美學與爭議。

NURBS 是「非均勻有理 B 樣條」(Non-Uniform Rational B-Splines)的縮寫,名字嚇人,概念其實優雅:用少數幾個「控制點」,加上每個點的「權重」,就能精確定義一條(或一面)平滑的自由曲線曲面。拉動控制點,曲面就跟著順順地變形;調整權重,就能讓曲面更貼近或更遠離某個點。它能精確表示從直線、圓,到任意自由曲面的一切 — 是工程界找了幾十年的「萬用曲面語言」。

NURBS 不是憑空出現的。它是萌芽期那條曲線數學的集大成: Pierre Bézier 的控制點、Paul de Casteljau 的演算法、Steven Coons 的曲面 patch,一路演化、再加上「權重」與「非均勻」的彈性,才長成 NURBS。1990 年代,當這套數學被包進好用的軟體介面,建築師第一次能像捏黏土一樣,直覺地推拉一塊精確的數位曲面 — 連續形式的解放,技術上就是從這裡開始的。

至於 blob,這個字本身有個有趣的來歷。Greg Lynn 借用了程式設計的術語 BLOB(Binary Large Object, 二進位大型物件),用來稱呼那些以「變形球」(metaball)融合出來的、軟綿綿、像水銀又像細胞的形體。

變形球的原理很迷人:想像幾個看不見的能量球,各自散發一個隨距離遞減的「場」;當兩個球靠近,它們的場會疊加,超過某個門檻的地方就連成一塊表面 — 於是兩顆球會像兩滴水珠一樣「融」成一個更大、有腰身的形體。建築師只要擺放、移動這些隱形的球,就能生成出無限多種有機、流動、彼此連續的量體。這正是「blob 建築」那種細胞感、液態感的技術來源。

1990 年代末,blob 成了最潮的建築語彙。它的美學主張很明確:拒絕直角與盒子,擁抱連續、流動、沒有明確正面背面的有機形體。一時之間,從學生作業到競圖提案,到處都是亮晶晶、圓滾滾的 blob。媒體甚至

造了一個半開玩笑的詞:「blobitecture」。

把 blob 推到最極端、最徹底的,是 Marcos Novak。他乾脆主張,建築根本不必落地成為實體 — 在電腦生成的虛擬空間裡,可以有一種「液態建築」(liquid architecture):會隨使用者、隨資料、隨時間不斷流動變形的空間。他談「transarchitecture(跨建築)」、談「eversion(外翻)」,把數位建築推向一個沒有重力、沒有材料、純粹由資訊構成的疆界。Novak 提醒我們:數位不只改變了建築的「形狀」,還動搖了建築「必須是實體」這個最根本的假設。

blob 的流行,也引爆了一場著名的論戰:「blobs vs boxes(blob 對盒子)」。一邊是擁抱曲面、複雜、流動的 blob 派;另一邊是堅守極簡、理性、直角的「盒子」派。表面上是造型之爭,底下其實是兩種價值觀的對撞:建築該追求表現性與複雜,還是節制與清晰?這場 2000 年前後的辯論,某種意義上至今未歇 — 每次參數化或 AI 又生出一批華麗造型,「blobs vs boxes」的幽靈就會再現一次。

對 blob 最尖銳的批評,是「中看不中用」。許多 blob 在螢幕上美得冒泡,一旦要落地,就面臨難題:雙曲面玻璃貴得嚇人、防水細部是惡夢、室內空間難用、清潔維護困難。批評者說,blob 把建築變成了「螢幕保護程式」 — 一種只在渲染圖裡成立的美。這個落差,逼出了下一個十年的關鍵問題:光是「能做出複雜形狀」還不夠,複雜必須是「有理由的」、「可建造的」、「有效能的」。

也有人認真地把 blob 推向真實與互動。荷蘭的 Kas Oosterhuis 不只做流線形體,還主張建築應該像生命體一樣,即時感知環境、回應使用者 — 把 1990 年代的 blob,和上一章 Cedric Price、Gordon Pask 的控制論夢想重新接了起來。對他來說,blob 的「連續曲面」不只是造型,更是一個能裝載感測與互動的、活的表皮。

回頭看 NURBS,它其實有兩張臉。一張是「解放」:它讓任何曲面都能被精確描述與建造。另一張卻是「限制」:NURBS 本質上是一張被四邊框住的「布」,要拼接、開洞、處理複雜拓撲時相當麻煩。這個技術上的不便,後來會推動建築界轉向另一套幾何工具(網格 mesh、細分曲面 subdivision),也間接促成了 Rhino、Grasshopper 等下一代工具的崛起。工具的限制,往往正是下一步演化的起點。

1990 年代用 NURBS 與 blob,徹底解放了建築的「形」。但解放之後,一個尷尬的問題浮現了:這些形,到底「意味著什麼」?一條曲線真的比一個直角更先進嗎?一個 blob 真的比一個盒子更有道理嗎?當形式可以隨意生成,「為什麼是這個形、而不是那個形」就變成了最難回答的問題。

1990 年代給了建築「連續曲面」這個強大的新身體,卻還沒給它一套夠有說服力的「理由」。要替這些形式找到正當性 — 讓複雜不再只是任性,而是某種邏輯、某種規則的必然結果 — 是下一個十年(參數化)的任務。但在那之前,還有一群理論家,試圖從哲學的高度,為這場形式的解放給出最深的解釋。我們先聽聽他們怎麼說。

摺疊的理論:Deleuze、objectile 與新的物件

前面提過,摺疊的哲學鑰匙來自 Gilles Deleuze。但他的影響,遠不只「把布對摺」這個漂亮的意象。1990 年代的建築理論家,幾乎是把 Deleuze 當成軍火庫,從中搬出一整套概念,來重新定義「形式、物件、設計者」到底是什麼。這一節我們把這套理論講清楚 — 因為它正是「連續曲面」之所以被認為前衛、而非只是好看的原因。

Deleuze 的《摺疊》談的其實是萊布尼茲與巴洛克。在他筆下,巴洛克的世界是一個「無限對摺」的世界:物質像一層層摺起的衣褶,沒有最小單位、也沒有清楚的邊界;靈魂則像一棟兩層的巴洛克房子,上下以摺相連。關鍵在於:摺疊讓「一」與「多」、「連續」與「差異」不再對立 — 差異被摺進同一塊連續的布裡,而不是被切成互不相干的碎片。

建築師還愛用 Deleuze 與 Guattari 在《千高原》裡的另一組概念:「平滑空間」與「條紋空間」。條紋空間,是被格線、座標、分區切割管理的空間(想想現代主義那種方格平面);平滑空間,則是連續、流動、沒有固定方向的空間(想想沙漠、海洋)。1990 年代的建築師,等於是想用數位曲面,造出一種屬於「平滑空間」的建築 — 一種拒絕被格線馴服的、流動的空間。

為什麼建築師對這套哲學如此著迷?因為它正好給了連續曲面一個「深刻的理由」。在 Deleuze 之前,曲面可能只是裝飾、是任性;有了 Deleuze,曲面變成了一種世界觀的展演 — 它宣稱:世界本來就是連續、流變、充滿內在差異的,而建築該忠實地反映這種狀態,而不是用直角與方格,去強加一個假的秩序。形式的解放,就這樣被升格成了一種哲學立場。

把這套哲學落實得最精準的,是法國的 Bernard Cache。Deleuze 在《摺疊》裡,甚至直接引用了 Cache 的一個概念:objectile(可譯為「變體物」或「物件式」)。

objectile 是一個玩文字遊戲、卻極深刻的詞。傳統的「物件」(object)是一個固定、單一、做完就定型的東西;而 objectile 不是一個物件,而是一條「生成物件的曲線函數」 — 它描述的,是一整族可能的變體。你調整參數,就從這條函數裡,長出一個個彼此不同、卻同屬一家的物件。換句話說,設計的對象,從「一個最終的形」,變成了「一套能生成無數形的規則」。

聽出來了嗎?這幾乎就是參數化設計的定義 — 只是 Cache 在 1990 年代中,就用哲學的語言把它講透了。而且他不只空談:他成立了一家叫 Objectile 的公司,用 CNC 數控機具,生產「沒有兩片相同」的非標準木製面板。理論、工具與生產,在他手上第一次接成一條線。他 1995 年的《Earth Moves(大地移動)》,至今仍是數位建築最重要的理論著作之一。

objectile 背後,是一個會撼動整個工業邏輯的想法:大量客製化(mass customization)。工業革命以來,「便宜」的代價就是「一模一樣」 — 唯有標準化才能量產。但如果生產是由數位檔案直接驅動數控機

具,那麼「做一千個一樣的」和「做一千個都不一樣的」,成本其實差不多。Cache 與 Lynn 都看到了這一點:數位製造,有機會終結「標準化」的暴政,讓每一件都能獨一無二,卻又不必更貴。

這個「非標準」的願景,後來在 2003 年龐畢度中心一場叫《Architectures non standard(非標準建築)》的展覽裡被正式命名與展示,集結了 Lynn、Cache、NOX、Oosterhuis 等人。它宣告:標準件、重複、模矩,不再是建築的鐵律;數位,讓「連續變化的、非標準的」量產成為可能。這條線,會在第五章「數位製造」裡真正落地。

這套理論還悄悄改寫了「設計者是誰」。如果設計的對象是一條生成函數、一套規則,那設計者的工作,就從「決定最終的樣子」,變成「設計那套生成的邏輯,再從結果裡挑選」。設計者退後了一步 — 從「造形的神」,變成「定義可能性空間的人」。這個主體位置的轉變,是整個數位建築最深、也最少被察覺的革命;它會在參數化、演算法、乃至今天的 AI 時代,被一再推得更遠。

這套華麗的理論,當然也不是沒有反對者。最常見的批評是:許多建築師其實是「先做了漂亮的曲面,再回頭找 Deleuze 來背書」 — 哲學成了形式的化妝品,而不是真正的驅動力。確實,當「摺疊」「平滑」「objectile」變成競圖簡報裡的咒語,它們也就慢慢失去了鋒利。但平心而論,正是這套理論,讓 1990 年代的曲面不只是「好看」,而擁有了能被認真討論、被嚴肅對待的思想厚度。

回頭看,1990 年代的摺疊理論,做了一件超越它自己時代的事:它在工具還很笨拙的時候,就先把「設計即生成規則」「物件即變體之族」「連續地容納差異」這些觀念,在哲學層次上想透了。它等於替還沒到來的參數化與演算法時代,先寫好了理論的劇本。

但理論再美,終究要面對一個樸素的問題:這些連續、流動、非標準的形體,真的蓋得起來、住得進去嗎? 1990 年代末,確實有一小批勇敢的案子,把 blob 從螢幕與哲學,推進了真實的雨水與重力之中。我們就用它們,來為這一章收尾。

第一批 blob 蓋起來:從紙上到真實

1990 年代的數位建築,大部分活在螢幕、競圖與理論裡。但這個十年的尾巴,有一小批案子真的蓋了起來 — 它們既證明了連續曲面的可能,也誠實地暴露了它的代價。

最具代表性的,是荷蘭的水館。1997 年,在荷蘭的人工島 Neeltje Jans 上,蓋起了一對關於「水」的展館;其中的「淡水館」(FreshH2O eXPO),由 Lars Spuybroek 的事務所 NOX 設計。

它徹底拋棄了牆、地板、天花的分別:整個空間是一條連續扭轉的管狀曲面,地會變成牆、牆會變成頂,人得半攀半走地穿過。更驚人的是它會「回應」 — 館內佈滿感測器,訪客的移動會即時觸發燈光、聲音、投影甚至噴霧的變化。它不只是一個 blob 造型,而是一個會感知、會反應的活空間。

看到這裡你大概會會心一笑:這不正是 Cedric Price 的 Fun Palace、Gordon Pask 的控制論夢想,隔了三十多年、終於用數位技術蓋出來的版本嗎?水館把 1990 年代的兩條線 — 連續曲面的「形」,與互動回應的「控制論」 — 縫進了同一座真實的建築。旁邊的「鹹水館」(Salt Water Pavilion)則由 Kas Oosterhuis 設計,同樣在探索可變、互動的空間。

另一條路,走得比較「可住」。荷蘭的 Ben van Berkel(與 Caroline Bos 的 UNStudio)在 1998 年完成了莫比烏斯住宅(Möbius House)。它的概念來自莫比烏斯環那條「只有一個面」的曲線:兩個人一天的動線 — 工作與休息、白天與夜晚 — 像莫比烏斯環一樣交纏、循環,再被翻譯成混凝土與玻璃的空間。形式不再是裝飾,而是一個「圖解」(diagram)被空間化的結果。

van Berkel 把這套「從圖解生成」的方法,發展成 UNStudio 的招牌。對他們,設計始於一個抽象的組織圖解 — 一條動線、一種拓撲、一個莫比烏斯環 — 再讓建築從這個圖解長出來。這跟上一節 Lynn 的「力場生成」異曲同工,都是「設計者定義一套生成邏輯,再讓形式長出來」。後來賓士博物館(2006)那兩條交纏上升的雙螺旋動線,就是這套圖解方法最成熟的展現。

如果要選一個「把摺疊真正蓋出來」的代表作,那會是橫濱港大棧橋(2002)。它出自一對年輕建築師 — Foreign Office Architects(FOA),也就是 Farshid Moussavi 與 Alejandro Zaera-Polo。他們在 1995 年的國際競圖中,用一個激進的概念勝出。

他們不把碼頭設計成一棟「建築」,而是設計成一片「摺疊的地景」:沒有立面、沒有正門,整座航廈是一張連續起伏、折來折去的表面,屋頂是可以漫步的公園,坡道把人流像水一樣導進導出。建築與地面、室內與室外的界線,被摺疊抹去了 — 這正是 Deleuze「連續地容納差異」在都市尺度上的實現。橫濱證明了:摺疊不只能做小住宅或展館,也能撐起一座龐大的公共基礎設施。

把這幾個案子擺在一起,會看到一些共同點:它們大多不大(除了橫濱)、都極耗心力、也都在跟「到底怎麼蓋」纏鬥。雙曲面、連續轉折、沒有重複構件 — 每一處都比蓋方盒子貴上數倍、難上數倍。這些先驅案,是用驚人的努力與預算「硬」蓋出來的;它們證明了 blob 可行,卻也暴露:若沒有更聰明的辦法去「合理化」這種複雜,連續曲面注定只能是少數人的奢侈品。

而這,正是下一個十年要解決的問題。1990 年代解放了形式,卻把「複雜如何被合理地建造」這個難題留在了原地。如果每一塊板、每一根桿都不一樣、都要手工協調,那再美的形式也無法規模化。建築界需要一種方法:讓複雜的形式,能從一套清楚的「規則與參數」裡長出來 — 於是每一個構件雖然不同,卻都能被同一套邏輯自動算出、自動加工。

這種「用規則與參數駕馭複雜」的方法,就是參數化(parametricism)。它不是要拋棄 1990 年代的連續曲面,而是要給它一副「理性的骨架」 — 讓 blob 不再只是力場的偶然或哲學的隱喻,而是一套可控、可改、可大量生產的系統。而讓這套方法從少數高手的專利、變成人人可用的,是 2000 年代一個改變了遊戲規則的小工具。

回望 1990 年代:它是數位建築「青春期」的十年。它叛逆(反抗直角與盒子),它狂熱(沉迷曲面與哲學),它也笨拙(蓋得貴、用得難)。但它做成了一件根本的事 — 它讓建築界相信:形式可以是連續的、流動的、被生成的,而且真的蓋得出來。形,被解放了。

這個十年也悄悄改變了建築教育。從哥倫比亞的無紙工作室開始,動畫軟體、NURBS 建模、生成式思考,一間接一間進了世界各地的設計課;台灣的建築系,也在 1990 年代末、2000 年代初陸續接上了這股潮流。一整代學生開始相信:設計不只是畫平面與立面,更是去定義一套能長出形體的邏輯。這個信念的轉變,其實比任何一棟 blob 都更深遠 — 因為它重寫的,是「建築師該怎麼想」這件事本身。

接下來的問題,是替這份自由找到紀律。當 David Rutten 在 2007 年釋出一個叫 Grasshopper 的小外掛,讓任何學生都能「拉節點、定規則、生成形體」時,1990 年代少數天才的實驗,就要變成一整個世代的共同語言。形式的解放之後,迎來的是參數化的時代。

來源

1. Folding in Architecture (AD, 1993) — Greg Lynn ed.
https://en.wikipedia.org/wiki/Greg_Lynn
2. The Fold (Deleuze book) — Wikipedia
[https://en.wikipedia.org/wiki/The_Fold_\(Deleuze_book\)](https://en.wikipedia.org/wiki/The_Fold_(Deleuze_book))
3. Deconstructivism — Wikipedia
<https://en.wikipedia.org/wiki/Deconstructivism>
4. Frank Gehry — Wikipedia
https://en.wikipedia.org/wiki/Frank_Gehry
5. Guggenheim Museum Bilbao — Wikipedia
https://en.wikipedia.org/wiki/Guggenheim_Museum_Bilbao
6. CATIA — Wikipedia
<https://en.wikipedia.org/wiki/CATIA>
7. Non-uniform rational B-spline (NURBS) — Wikipedia
https://en.wikipedia.org/wiki/Non-uniform_rational_B-spline
8. Greg Lynn / Animate Form — Wikipedia
https://en.wikipedia.org/wiki/Greg_Lynn
9. Bernard Cache (objectile) — Wikipedia
https://en.wikipedia.org/wiki/Bernard_Cache
10. Marcos Novak (liquid architecture) — Wikipedia
https://en.wikipedia.org/wiki/Marcos_Novak
11. Lars Spuybroek / NOX — Wikipedia
https://en.wikipedia.org/wiki/Lars_Spuybroek
12. Foreign Office Architects (Yokohama) — Wikipedia
https://en.wikipedia.org/wiki/Foreign_Office_Architects

參數化的時代

2000s

2000 年代,1990 年代解放出來的形式,終於找到了駕馭它的方法 — 參數化。設計者不再直接畫形狀,而是定義一套「參數 + 關係」的規則,讓形體自己長出來、又能被精確建造。而一個免費的小外掛 Grasshopper,把這套方法交到了每一個學生手裡。

上一章結尾留了一個難題:1990 年代解放了形式,卻沒解決「複雜如何被合理地建造」。

這一章談 2000 年代怎麼回答它 — 從「參數化是什麼」這個常被誤會的概念講起,到 Grasshopper 如何讓它平民化、Schumacher 如何把它升格成一場運動、ZHA 與一群工程師如何把它蓋成真實的地標。

參數化是什麼:從「畫形狀」到「定義關係」

上一章結尾留了一個問題:1990 年代解放了形式,卻沒解決「複雜如何被合理地建造」。2000 年代的答案,叫參數化(parametric design)。但在談工具與宣言之前,得先把「參數化到底是什麼」講清楚 — 因為它太常被誤會成「就是做曲面」,其實它的核心,是一種完全不同的設計思維。

參數化設計的核心,其實 1963 年的 Sketchpad 就埋下了:設計者不直接畫出最終的形,而是描述元素之間的「關係」與「規則」,再讓電腦去算出滿足這些關係的形。差別只在於,到了 2000 年代,這套思維終於有了成熟、好用的工具,能在真實的專案裡跑起來。

舉個最簡單的例子。傳統畫法:你畫一道樓梯,要一階一階畫,改層高就得全部重畫。參數化做法:你定義一條規則 —— 「階高固定 17 公分,階數 = 樓層高度 ÷ 階高,踏面隨之排列」。之後你只要改一個數字(樓層高度),整道樓梯就會自己重算、重排。你設計的不是「這一道樓梯」,而是「一台會生成樓梯的機器」。

把這個例子放大,就是參數化的三件事:參數(可調的數值,如高度、角度、數量)、關聯(參數與元素之間的依存規則),以及自動重算(改一個,牽動全部)。這套「關聯式幾何」(associative geometry)讓設計變成一個活的、會連動的系統 — 牽一髮,動全身。

這跟上一章 Greg Lynn 的「力場生成」有微妙、卻關鍵的差別。Lynn 是把幾何丟進一個物理模擬裡,讓它被「吹」成某個樣子 — 結果常常難以精確控制。參數化則更像「寫一套明確的規則」:每一個形,都能追溯到一條清楚的邏輯。它把 1990 年代那種詩意、卻有點任性的生成,變成了可控、可解釋、可修改的工程。

有趣的是,把參數化認真帶進建築的,不是學院的理論家,而是工程與大型實務的需求。2000 年代初,Robert Aish 在 Bentley 公司開發了 GenerativeComponents(GC)——一套讓建築師能建立「關聯式模型」的軟體。它最早的重度使用者,是 Foster + Partners 這類得處理大量複雜幾何的大型事務所:當你要設計一片有上千片、每片都不同的玻璃帷幕,手工畫是不可能的,你只能寫一套規則,讓它自己生成。

2001 年,Aish 與 Foster 的 Hugh Whitehead、KPF 的 Lars Hesselgren 一起發起了 SmartGeometry ——一個聚集前沿建築師、工程師與程式設計師的社群,每年辦工作坊,專門鑽研「如何用關聯式、參數化的方法,駕馭複雜幾何」。SmartGeometry 等於是參數化建築的「武林大會」,GenerativeComponents、乃至後來的 Grasshopper,許多關鍵想法都在這裡被磨出來。

關聯式模型背後,有一個優雅的結構:它其實是一張「有向圖」——每個元素都記得自己「依賴誰、又被誰依賴」。改動上游,變化會像水一樣順流而下,自動更新所有下游。這跟 Sketchpad 的「約束」、跟程式設計裡的「相依關係」,本質上是同一件事。參數化建築,某種意義上,就是把「寫程式」的思維,正式請進了設計的核心。

為什麼是 2000 年代?因為三件事同時到位了:電腦夠快、3D 建模軟體(尤其是 Rhino)夠普及,而 1990 年代的 blob 已經製造出對「複雜形式」的龐大需求,卻苦無「合理建造它」的方法。參數化正好補上這一塊——它讓 1990 年代那些難以掌控的曲面,變成可以被精確定義、修改與分工的系統。需求與工具,又一次在對的時間相遇。

參數化最深的改變,還是那個老主題:設計者的角色。當你設計的是「規則」而不是「形狀」,你就得先想清楚「什麼該是變數、什麼該是關係、什麼該被算出來」。這逼著建築師像程式設計師一樣思考——把設計「拆解」成一套清楚的邏輯。對很多人,這是解放(形式有了理性的依據);對另一些人,這是束縛(設計變得像在填一張會自動計算的試算表)。這個張力,貫穿了整個參數化時代。

也因此,參數化從一開始就有兩張臉。一張是「工程的」:用規則去馴服複雜、讓難蓋的形式變得可建造、可估價(這是 GenerativeComponents、SmartGeometry 那群工程師在乎的)。另一張是「表現的」:用參數去生成前所未有的造型、追求一種新的美學(這是後來 Patrik Schumacher 與 ZHA 高舉的)。同一套技術,一邊指向務實,一邊指向風格——這兩條線的拉扯,構成了這一章後面的故事。

所以,參數化不是一種「樣式」,而是一種「方法」:把設計從「畫出一個結果」,轉成「定義一套生成結果的系統」。它是 Sketchpad 那顆種子,在 2000 年代終於長成的大樹。

但只要這套方法還鎖在 GenerativeComponents 這類昂貴、難學、給大公司用的軟體裡,它就只能是少數人的武器。真正讓參數化「燎原」、讓每一個建築系學生都能上手的,是 2007 年一個免費的小外掛——它的名字,叫 Grasshopper。

Grasshopper:把演算法設計平民化

如果要選一個「改變了整個世代」的數位建築工具,那幾乎毫無疑問是 Grasshopper。它不是最強大的,也不是最早的,但它做到了一件別的工具沒做到的事:讓「演算法設計」從少數高手的祕技,變成人人都能玩的日常。

在 Grasshopper 之前,想做參數化、演算法設計,你得會「寫程式」—— RhinoScript、VBScript,或 GenerativeComponents 那類專業軟體。這道門檻,把絕大多數建築師擋在門外:他們有空間的直覺,卻未必想學程式語言的語法、分號與除錯。參數化於是停在一小群「會寫 code 的建築師」手裡。

舞台是 Rhinoceros(Rhino)。這套由 Robert McNeel 公司在 1998 年推出的 NURBS 建模軟體,因為相對便宜、好上手、又能精確處理自由曲面,在 2000 年代成了建築系與中小事務所的最愛。Rhino 的普及,等於替參數化準備好了肥沃的土壤——就差一個能讓人「不寫 code 也能定義規則」的介面。

這個介面,來自一位叫 David Rutten 的年輕程式設計師。2007 年,他在 McNeel 內部開發了一個 Rhino 外掛,最初叫「Explicit History」,後來改名 Grasshopper。它的革命性在於:你不用打一行字,而是在畫布上拖出一個個「電池」(運算元件)——這個是「點」、那個是「直線」、再接一個「陣列」——用線把它們串起來,資料就沿著線流動、生成幾何。

這叫「視覺化程式設計」(visual programming)。它把寫程式的那套邏輯——輸入、運算、輸出——變成了看得見、可以用拖拉操作的圖。對視覺思考的建築師來說,這簡直是量身訂做:你不必記語法,只要像接水管一樣把元件接起來,就能定義一套生成幾何的規則。拉一個滑桿,整個模型即時跟著變——Sketchpad 那個「改一個、動全部」的夢想,第一次變得這麼直覺、這麼好玩。

Grasshopper 引爆的速度是驚人的。因為它免費、內建在大家本來就在用的 Rhino 裡、又這麼直覺,短短幾年內,它就從一個小外掛,變成全球建築系的「第二母語」。學生們瘋狂分享「定義」(Grasshopper 檔案)、外掛與教學;一個龐大的社群在網路上自發長了出來。到 2010 年代初,「會不會 Grasshopper」,幾乎成了數位設計能力的代名詞。

Grasshopper 讓什麼成為可能?幾乎是一切 1990 年代只能用昂貴軟體做的事:陣列、漸變、鑲嵌、面板化、找形、最佳化……全都變成可以拖拉組裝的家常便飯。一個大二學生,現在能在一個下午,做出十年前要博士生寫好幾週程式才做得出的幾何。它把參數化的門檻,從「會寫程式」一口氣降到了「會接電池」。

更關鍵的是它的「外掛生態」。Grasshopper 開放讓人開發外掛,於是各種強大的能力被掛了上來:Kangaroo(物理找形)、Galapagos(演化最佳化)、Ladybug(環境分析)、Weaverbird(網格細分)……Grasshopper 本身像一塊主機板,全世界的開發者替它插上各種功能卡。這個生態系,讓它的能力遠遠超出 Rutten 一個人能寫的範圍——它變成了一個平台。

「平民化」有它輝煌的一面,也有它的陰影。輝煌的是:演算法設計不再是菁英的特權,任何人都能參與、實驗、創造。陰影則是:當人人都能拉出複雜的圖案,網路上很快充斥著「看起來很參數化、卻沒什麼道理」的作業——漂亮的鑲嵌表皮、隨機的開孔、為複雜而複雜。工具太好用,有時反而讓人忘了問「為什麼」。

Grasshopper 也徹底改寫了建築教育。設計課裡開始出現「運算設計」「參數化設計」的課程;「邏輯」「資料流」「演算法」這些原本屬於資訊系的詞,進了建築系的評圖現場。一整代學生(包括今天台灣各建築系的學生)學會了一件事:在設計一個東西之前,先問「它的生成邏輯是什麼」。這個思維習慣的養成,比任何一個花俏的作業都更重要。

回到那條貫穿全書的線。Gehry 用電腦「實現」手作;Lynn 用電腦「模擬」生成;而 Grasshopper 讓設計者能「明確地編寫」生成的規則——它把「設計即定義系統」這件事,變得透明、可讀、可分享。你可以打開別人的 Grasshopper 定義,一個元件一個元件地看懂他「是怎麼想的」。設計的邏輯,第一次變得像開源程式一樣,可以被閱讀與傳承。

Grasshopper 的歷史意義,不在於它發明了參數化(它沒有),而在於它「普及」了參數化。它把一個原本屬於少數天才與大公司的方法,交到了每一個學生手裡——而當一個強大的方法變成人人可用,它就不再只是工具,而成了一種共同的語言、一種看待設計的方式。

當一套方法強大到變成一整個世代的共同語言,總會有人想替它寫下「宣言」、把它升格成一個「主義」。2008 年,Zaha Hadid 的合夥人 Patrik Schumacher,就做了這件既大膽又惹議的事——他宣布:參數化不只是方法,它是繼現代主義之後,屬於我們這個時代的「風格」。

Schumacher 與參數主義宣言:一場運動的命名

到了 2000 年代末,參數化已經是事實上的潮流。但它還缺一樣東西:一個名字、一面旗,以及一套說它「不只是流行、而是時代必然」的論述。把這件事做到極致(也因此惹來最多爭議)的,是 Patrik Schumacher。

Schumacher 是 Zaha Hadid 的長期合夥人——如果說 Zaha 是事務所的藝術靈魂,Schumacher 就是它的理論大腦與管理引擎。他不只想做出漂亮的參數化建築,更要替整個運動,建立一套完整、自治、雄心勃勃的理論。

2008 年,在威尼斯建築雙年展上,Schumacher 發表了《Parametricist Manifesto(參數主義宣言)》。他的主張石破天驚:參數化不是一種技術、也不只是一種風格選項,而是繼現代主義之後、足以定義我們這整個時代的「新風格」(epochal style)。現代主義屬於工業時代;參數主義,屬於後福特、網路與資訊的當代。

這是個極大的宣稱。在 Schumacher 的敘事裡,1990 年代以來的解構、摺疊、blob,都只是過渡;它們要在參數主義裡,才找到歸宿與成熟。他甚至刻意把它寫成大寫的 Parametricism,當成一個與「現代主義 Modernism」平起平坐的歷史時期。這種野心,在後現代以來「什麼都行、沒有大敘事」的建築界,顯得格外突出 — 也格外刺眼。

Schumacher 還給了參數主義一套近乎教條的「規則」。負面表列:避免剛硬的幾何基本量(純粹的方塊、圓柱)、避免簡單的重複、避免不相關元素的生硬並置。正面表列:所有形式都該是參數化可塑的、所有系統都該彼此關聯連動、要讓元素隨場域條件平順地「分異」(differentiate)。一句話:萬物互相關聯、平滑分異、沒有突兀的接縫。

他更把這套邏輯往上推到城市尺度,提出「參數化都市主義」(Parametric Urbanism):整座城市不該是一塊塊分區的拼貼,而該像一個有機體,街道、量體、密度都隨彼此與地形「連續地分異」。這其實是 1990 年代「連續地容納差異」那個 Deleuze 式理想,被放大到都市規劃的版本。

Schumacher 的理論企圖,在他 2010–2012 年的兩大卷《The Autopoiesis of Architecture(建築的自我生產)》裡達到頂點。他借用系統理論,把「建築」本身當成一個會自我界定、自我演化的溝通系統來分析。無論你同不同意,這套著作的規模與野心,在當代建築理論裡都極為罕見 — 它是要替參數主義,寫一部哲學等級的根本大法。

宣言為什麼重要?因為「命名」本身就是一種力量。當散落各處的參數化實驗,被收攏進一個有名字、有教條、有歷史定位的「主義」,它就從一堆技巧,變成了一面可以追隨、也可以反對的旗幟。一夕之間,建築師彷彿必須選邊:你是參數主義者,還是不是?這種凝聚力(與排他性),正是任何「運動」成形的關鍵。

但這套宏大敘事,也立刻招來猛烈反擊。最常見的批評是:它太傲慢了。把一種還很年輕、爭議很大的取向,自封為「定義時代的唯一風格」,被許多人視為一種理論上的霸權 — 彷彿不做參數化,就是落伍。在一個珍視多元的時代,這種「獨尊一種風格」的姿態,讓很多人反感。

更實質的批評是:參數主義會不會只是把 1990 年代「形式淪為風格」的老問題,用更華麗的理論重新包裝了一次?當「平滑分異」「系統關聯」變成生產漂亮曲面的藉口,它和它所批評的「任性造型」,差別到底在哪?批評者說,很多掛著參數主義之名的作品,骨子裡仍是形式先行、論述後補。

2016 年 Zaha Hadid 驟逝後,Schumacher 接掌事務所,卻在那年的世界建築節上,發表了一系列極端自由市場的政治主張 — 主張廢除社會住宅、全面私有化公共空間。這番言論引爆眾怒,也嚴重損害了他個人、乃至「參數主義」的公共形象。一場原本關於形式與技術的運動,因為旗手的政治立場,蒙上了難以擺脫的陰影。這也提醒我們:建築的「主義」,從來不只是美學,它總牽連著價值與政治。

平心而論,Schumacher 的功過都很巨大。他的宣言把參數化推上了理論的高度、給了它歷史的座標,功不可沒;但他過度的宣稱與爭議的政治,也讓「參數主義」這個詞,在許多人耳裡,漸漸從前衛變成了某種傲慢的代名詞。一個運動最好的理論家,有時也會是它最大的負擔。

但理論的爭吵,擋不住作品的說服力。當 Schumacher 在紙上論戰時,他與 Zaha 的事務所,正一座接一座地,把參數主義那些流動、無縫、彷彿從地面長出來的形體,蓋成了全球城市的地標。要看參數化「長成真實建築」是什麼樣子,沒有誰比 Zaha Hadid 更有資格示範。

ZHA:把參數化推上世界舞台

如果參數化在 2000 年代有一張臉,那是 Zaha Hadid 的臉。她與她的事務所 ZHA(Zaha Hadid Architects),把參數主義那套流動、無縫的形式語言,變成了從巴庫到北京、從羅馬到廣州的一座座真實地標。

但 Zaha 的崛起一點都不順。1980 年代,她以一系列爆炸性、碎裂的繪畫聞名(沒錯,就是上一章那個解構展裡的她),被視為才華洋溢、卻「只會畫、不會蓋」的「紙上建築師」。1983 年她贏了香港 The Peak 的競圖,卻沒能蓋成;這種「得獎無數、卻蓋不出來」的處境,持續了將近十年。

轉折,從 1993 年的德國維特拉消防站(Vitra Fire Station)開始 — 她終於有了第一棟建成的作品。而真正讓她的形式「流動」起來的,正是 2000 年代的數位工具:當參數化讓那些動態、連續的曲面變得可以精確設計與建造,Zaha 腦中那些一直畫得出、卻蓋不出的形體,終於落了地。技術,解放了她的才華。

ZHA 並不是靠 Zaha 一個人徒手畫。事務所內部有一支高度專業的運算設計團隊(後來成形為 ZHA CODE),專門用 Grasshopper、GenerativeComponents 這類工具,把 Zaha 的造型意圖,轉譯成可控的參數模型、再拆解成可建造的構件。藝術直覺(Zaha)、理論論述(Schumacher)與運算工程(CODE 團隊)三位一體 — 這正是參數化時代,一間頂尖事務所該有的樣子。

這套機器最完美的展演,是 2012 年巴庫的 Heydar Aliyev 文化中心。整棟建築像一塊被掀起的大地,牆、地、頂、天花連成一片沒有接縫、沒有轉角的連續曲面,白色的表皮如波浪般起伏。它把「連續地容納差異」從哲學口號,變成了一萬多片量身訂做的面板,精確地拼成一個整體。它幾乎就是參數主義的「示範作」:流動、無縫、彷彿天生如此。

同一年,ZHA 在北京完成了 Galaxy SOHO — 四個流動的卵形量體,以連續的天橋彼此相連,像一座白色的人造峽谷。2000 年代的中國,正以驚人的速度與預算,成為這類大膽形體最重要的試驗場;ZHA、OMA 等事務所的許多參數化地標,都在這裡落地。但 Galaxy SOHO 也引來爭議 — 文資團體批評它無視老北京的胡同肌理,粗暴地植入了一個外星般的物體。

Heydar 與 Galaxy 只是其中兩個。2000 年代末到 2010 年代,ZHA 像一台地標製造機:羅馬的 MAXXI 當代美術館(2010)、廣州大劇院(2010)、倫敦奧運水上運動中心(2012)……每一座都帶著那個一眼就能認出的「Zaha 簽名」:流動、速度感、彷彿凝固的運動。參數化第一次證明,它不只能做學生作業或競圖效果圖,而能撐起世界級的公共建築。

ZHA 的作品,建立了一種「立刻可辨識」的風格 — 這在當代建築裡其實很稀有。看到那種無縫流動、彷彿從地面長出、充滿動勢的白色曲面,你幾乎不會認錯。但這份「高辨識度」也是雙面刃:它讓 ZHA 成為品牌,卻也讓批評者說,這套語言不論蓋在巴庫、北京還是倫敦,看起來都差不多 — 它真的回應了在地,還是只是到處複製同一種炫目的「未來感」?

對 ZHA(乃至整個明星參數化)最尖銳的批評,圍繞著「權力」與「脈絡」。它的客戶,常是石油國家、威權政府或巨型開發商 — 那些付得起、也想要一座驚世地標來彰顯國力或財力的人。於是有人質疑:參數主義那套「流動、無縫、宏偉」的美學,會不會在不知不覺中,成了權力與資本最順手的化妝品?以及:當建築忙著當一件奪目的雕塑,它對基地、氣候、街廓的責任,還剩下多少?

這類批評在 2010 年代越來越尖銳,也越來越關乎倫理。當頂尖事務所滿世界飛、為出價最高者打造地標,有人開始追問:這些奪目的形式,究竟是為了誰?Zaha 本人也曾因對海外工地勞動條件的發言而引發爭議。這些爭論未必有標準答案,但它們把一個問題擺上了檯面:當形式的雄心越來越大,對人、對社會的責任,絕不該被一併放大地遺忘。

2016 年,Zaha Hadid 因心臟病驟逝,得年僅 65。她是第一位獲得普立茲克獎(2004)的女性建築師,在一個長期由男性主導的行業裡,殺出了一條血路。她過世後,事務所由 Schumacher 接手,至今仍持續運作、持續產出那種辨識度極高的形體。Zaha 留下的,不只是幾十座地標,更是一個證明:數位工具,能讓最大膽的形式想像,真正在世界各地站立起來。

ZHA 的意義,在於它讓參數化「可信」了。在 Zaha 之前,流動的曲面多半活在學生作業、競圖板與哲學論文裡;在她之後,沒有人能再說「那種東西只是電腦遊戲、根本蓋不出來」。她用一座座真實的、巨大的、得獎的建築,替參數化背了書 — 哪怕代價是爭議纏身。

但這裡藏著一個容易被華麗外表蓋過的事實:Heydar 那片無縫的曲面、Galaxy 那些流動的量體,真正困難的從來不是「畫出來」,而是「算出來、拆出來、蓋出來」。在 Zaha 的造型背後,是一整群工程師與幾何學家,默默解決著「一萬片都不一樣的板子到底怎麼做」這種要命的問題。下一節,我們就去看這群幕後英雄 — 沒有他們,參數化永遠只能是一張漂亮的渲染圖。

把複雜變得可蓋:結構、幾何與幕後工程師

上一章留下一個沒解決的難題,這一節要正面回答它:那些流動、連續、每一塊都不一樣的形體,到底怎麼蓋?答案不在明星建築師身上,而在一群很少上鏡的人 — 結構工程師與幾何學家。沒有他們,參數化永遠只是一張漂亮的渲染圖。

問題的核心,叫「合理化」(rationalization)。電腦裡一個光滑的自由曲面,要蓋出來,得先被拆解成一片片真實、做得出來的構件 — 而且最好是平的(平板比雙曲板便宜太多)、最好能重複(模具越少越省),還要撐

得住力。如何把一個連續曲面,聰明地切成成千上萬片「可製造、又拼回去仍然順」的板子,本身就是一門深奧的學問。

這門學問,在 2000 年代被正式命名為「建築幾何」(architectural geometry)。數學家 Helmut Pottmann 與同事 2007 年的同名鉅著,系統性地研究:如何用平板逼近曲面、如何讓網格的每一格都接近平整、如何處理可展曲面……他還共同創立了 Evolute 公司,把這些演算法變成工具,專門替難搞的自由曲面「找出最省、最可蓋的拆法」。幾何,從畫圖的工具,變成了解決營造難題的鑰匙。

這裡有個微妙、卻關鍵的分野:「事前合理化」與「事後合理化」。事後合理化,是先做出你想要的造型,再回頭硬把它拆成可蓋的板——常常很痛苦、很浪費。事前合理化,則是一開始就用「做得出來的幾何」(例如可展曲面、平面四邊形網格)來生成造型——形式從一開始就「內建」了可建造性。參數化的高手,追求的正是後者:讓「美」與「可蓋」在同一套規則裡同時被滿足。

在實務界,把這套「結構 × 幾何 × 運算」玩到出神入化的,是工程師 Cecil Balmond。他在工程顧問公司 Arup 內部成立了「先進幾何小組」(AGU),專門和最前衛的建築師合作,解決那些「看起來不可能」的結構。他與伊東豐雄合作的 2002 年蛇形藝廊(Serpentine Pavilion),用一套看似隨機、其實是嚴謹演算法生成的線條,既是結構、又是花紋——完美示範了「結構即裝飾、規則即形式」。

Balmond 也是 OMA 那座驚世駭俗的北京 CCTV 大樓(那個環狀懸臂的「大褲衩」)背後的結構大腦之一。他在《Informal》一書裡主張:結構不必是僵硬、對稱、可預測的;它可以是「非正式的」、不規則的、由演算法生成的——只要算得夠精確。這套思想,讓結構工程從「建築師畫好之後才來撐住」,變成了「一開始就參與生成形式」的夥伴。

在日本,結構工程師 佐佐木睦朗 走的是另一條同樣深刻的路。他發展出「敏感度分析」與「擴展演化結構最佳化」等方法——簡單說,就是讓電腦像演化一樣,反覆微調形狀,自動找出「用最少材料、達到最佳結構效率」的曲面。他與伊東豐雄、SANAA、磯崎新等人合作,把這種「電腦找形」的自由曲面結構,做成了一座座真實的建築。

伊東豐雄 2001 年的 仙台媒體中心,正是這條路的里程碑(結構即由佐佐木睦朗操刀)。它沒有傳統的柱與牆,取而代之的,是十三根像海草、像水中氣泡上升般的鋼管束——它們同時是結構、是管道、是動線、是光井。結構不再隱藏在牆裡,而成了空間的主角。仙台證明了:運算式的結構思維,能造出一種前所未見、卻又輕盈合理的空間。

這種「複雜結構 × 運算」的能力,在 2008 年北京奧運達到一次集體高潮。鳥巢(北京國家體育場,Herzog & de Meuron + Arup + 艾未未)那看似隨機交織的鋼結構,其實是嚴謹計算的結果;水立方(北京國家游泳中心,PTW + Arup)的泡泡表皮,則直接取自一個數學難題——Weaire-Phelan 結構,一種「如何用最少表面積、把空間分割成等體積泡泡」的最佳解。兩座建築都向世界宣告:複雜,可以是理性的、是算得出來、也蓋得起來的。

讓建築師、工程師與程式設計師能真正一起解這些問題的,正是前面提過的 SmartGeometry 社群。在它每年的工作坊裡,Robert Aish、Hugh Whitehead 這些人,帶著 GenerativeComponents,和結構工程師、幾何學家一起動手,把「如何合理化一個自由曲面」從個案的苦工,變成可以傳授、可以共享的知識。參數化之所以能成熟,靠的不只是明星的造型,更是這種跨領域的協作文化。

這群幕後工程師,其實也悄悄修正了參數化的方向。當 Patrik Schumacher 那邊高談「風格」時,這邊的人更在乎「性能」:結構效率、材料用量、可建造性、成本。他們讓參數化不只關於「長得怎樣」,更關於「撐得住、蓋得起、用得久」。這條「性能導向」的線(其實上承萌芽期 ABACUS Strathclyde 的設計評估),會在下一章的演算法最佳化、與第五章的數位製造裡,變成主角。

所以,參數化真正成熟的標誌,不是又多了一座流動的地標,而是「複雜」終於變得「可控、可算、可蓋」。明星建築師負責讓人驚嘆,而這群工程師與幾何學家,負責讓驚嘆「站得起來」。少了任何一邊,參數化都只是半套。

有趣的是,這種「用幾何規則駕馭複雜形體」的智慧,其實一點都不新。早在一百多年前,巴塞隆納就有一位建築師,沒有電腦,卻已經在用繩子、重物與嚴謹的幾何,做著本質上一模一樣的事。他的名字是 Antoni Gaudí,而他那座還沒蓋完的教堂,正等著數位時代的人,用參數化把它接著蓋下去。

老教堂與新方法:Gaudí、聖家堂與參數化的遺產

把時間倒回十九世紀末。1882 年動工的巴塞隆納 聖家堂(Sagrada Família),它的建築師 Antoni Gaudí,在一個連電燈都還稀奇的年代,卻已經在做最徹底的「參數化」設計 — 只是他的電腦,是繩子、沙袋與鏡子。

Gaudí 最著名的工具,是「懸鏈模型」:他把繩子按柱與拱的位置垂掛,在繩上掛滿裝著鉛彈的小袋當作荷重;繩子在重力下自然垂成的曲線(懸鏈線),反過來(上下顛倒)看,就是一個「純受壓、不需要扶壁」的最佳拱形。他在模型下方放一面鏡子,從鏡中看到的,就是未來教堂的形狀。這是一台不折不扣的「類比計算機」 — 用物理本身,去求解最有效率的結構形式。

不只如此。Gaudí 晚年沉迷於「直紋曲面」(ruled surfaces) — 雙曲面、雙曲拋物面、螺旋面這些,看起來彎彎曲曲、其實都能用一根根直線生成的曲面。對他,這既是上帝的幾何、也是聰明的營造:彎曲的造型,卻能用直的木條、直的石材紋路去施工。聽起來耳熟嗎?這正是上一節「建築幾何」在追求的「用簡單元件逼近複雜曲面」 — Gaudí 在一百年前,就憑直覺與虔誠,走到了那裡。

問題是,Gaudí 1926 年因車禍驟逝,只留下燒毀大半的模型碎片與少量圖紙。他那套深奧的幾何邏輯,大多藏在他的腦中與那些石膏模型裡。一座建立在「複雜幾何規則」上的教堂,卻失去了掌握規則的人 — 此後幾十年,後繼者只能緩慢地、痛苦地,試著拼湊、猜測 Gaudí 到底想怎麼蓋。

轉機,來自數位。1979 年起,年輕的紐西蘭建築師 Mark Burry 加入了聖家堂團隊。他做了一件關鍵的事:用電腦,去「重新發現」Gaudí 的幾何邏輯。他意識到,Gaudí 那些直紋曲面、那些比例與規則,本質上就是一套參數化的系統 — 而這,正好是 CAD 與參數化軟體最擅長的。

Burry 與團隊用上了當時最先進的工具(包括 Gehry 那邊也在用的航太級軟體),把 Gaudí 殘存的幾何規則,重建成一套活的參數模型。一旦規則被數位化,那些失落的部位 — 中殿的拱頂、高聳的尖塔 — 就能依照 Gaudí 自己的邏輯被精確地推導、生成出來,再交給 CNC 切石機去加工。聖家堂的營造,於是從「猜測大師的意圖」,變成了「執行大師的演算法」。

這是數位建築史上最美的一個迴圈:一座十九世紀、用繩子與重力「類比計算」出來的教堂,在二十一世紀,被用 NURBS 與參數模型「數位計算」著接續完成。Gaudí 與 Burry,隔著一個世紀,做的其實是同一件事 — 用一套嚴謹的幾何規則,去生成一個複雜而合理的形式。聖家堂於是成了一個活生生的證明:參數化不是電腦發明的新玩意,它是一種古老的設計智慧,只是終於等到了配得上它的工具。

聖家堂也漂亮地總結了這一章:參數化的核心,從來不是「做曲面」,而是「用規則生成形式」。Gehry 的手作、Lynn 的力場、Schumacher 的宣言、ZHA 的地標、Balmond 的結構、Gaudí 的懸鏈 —— 看似各異,骨子裡是同一件事:設計者定義一套邏輯,讓形式從邏輯裡長出來。這,就是參數化時代留給建築最深的遺產。

回頭看參數化的爭議,答案其實漸漸清楚了。Schumacher 想把它變成一種「風格」(大家都做流動曲面),這個主張多半失敗了 — 沒有人接受「參數主義是唯一的時代風格」。但參數化作為一種「方法」,卻徹底贏了:今天幾乎沒有一間像樣的事務所、沒有一個建築系,不用 Grasshopper、不談參數與規則。風格會過時,方法卻沉澱了下來,成了基本功。

也因此,到 2010 年代中期,「參數化」漸漸不再是一個炫目的標籤,而變成了像「會用 CAD」一樣理所當然的基本能力。當一項技術成熟到「沒人再特別提它」,它才算真正贏了。參數化今天的低調,恰恰是它徹底勝利的證據 —— 它已經溶進了建築設計的血液裡。

但參數化也有它的天花板。它擅長的是「調整」一套已經定好的規則(拉滑桿、改參數),設計者始終握著那套規則。可是,有沒有可能讓設計「更自主」一點 —— 不只是調參數,而是讓電腦自己「演化」「生長」、自己探索出設計者沒想到的解?這個更激進的念頭,把我們帶向下一章。在 AA Emergent Technologies(湧現科技)等研究裡,「湧現」「演化」「生成」正取代「參數」,成為新的關鍵字。

另一條延伸,則指向「製造」。當參數模型能精確算出每一個構件,下一步自然是:讓這些資料,直接去驅動機器把它們做出來。Balmond 的結構、Burry 的切石、Pottmann 的板材 —— 都已經站在「設計到製造」那道門前。這條線,會在第五章「數位製造」裡,由機器手臂與 3D 列印,真正推開。

2000 年代,是數位建築「轉大人」的十年。它替 1990 年代狂野的形式,找到了理性的骨架;它把演算法設計,從天才的祕技,變成一代人的基本功;它也讓我們看清:複雜,可以是有道理的。形式被解放(1990s),又被

賦予了紀律(2000s)。接下來,設計者要把更多的主導權,交給演算法本身——讓電腦,從「執行規則的工具」,慢慢變成「生成規則的夥伴」。

來源

1. Parametricism — Wikipedia
<https://en.wikipedia.org/wiki/Parametricism>
2. Grasshopper 3D — Wikipedia
https://en.wikipedia.org/wiki/Grasshopper_3D
3. Rhinoceros 3D — Wikipedia
https://en.wikipedia.org/wiki/Rhinoceros_3D
4. GenerativeComponents — Wikipedia
<https://en.wikipedia.org/wiki/GenerativeComponents>
5. Smartgeometry — Wikipedia
<https://en.wikipedia.org/wiki/Smartgeometry>
6. Patrik Schumacher (Parametricist Manifesto) — Wikipedia
https://en.wikipedia.org/wiki/Patrik_Schumacher
7. Zaha Hadid — Wikipedia
https://en.wikipedia.org/wiki/Zaha_Hadid
8. Heydar Aliyev Center — Wikipedia
https://en.wikipedia.org/wiki/Heydar_Aliyev_Center
9. Galaxy SOHO — Wikipedia
https://en.wikipedia.org/wiki/Galaxy_SOHO
10. Architectural geometry / Helmut Pottmann — Wikipedia
https://en.wikipedia.org/wiki/Architectural_geometry
11. Cecil Balmond — Wikipedia
https://en.wikipedia.org/wiki/Cecil_Balmond
12. Sendai Mediatheque (Toyo Ito / Mutsuro Sasaki) — Wikipedia
https://en.wikipedia.org/wiki/Sendai_Mediatheque
13. Sagrada Família — Wikipedia
https://en.wikipedia.org/wiki/Sagrada_Fam%C3%ADlia
14. Antoni Gaudí — Wikipedia
https://en.wikipedia.org/wiki/Antoni_Gaud%C3%AD

演算法成為設計者

2005 - 2015

2005 到 2015 年,設計者再退一步:從「調整參數」,走向「編寫會自己生成、甚至自己演化的程序」。形,不再是畫出來的,而是一段演算法跑出來的結果 — 常常連設計者自己,都意想不到。

上一章結尾說,設計者要把更多主導權交給演算法。這一章,就是這件事真正發生的故事。

我們會看到:當寫程式變得人人可及、算力變得便宜,設計者如何從「調一個模型」,走向「編寫一段會生成、會演化的程序」 — 並向大自然偷師,讓設計像植物生長、像鳥群飛行、像細胞分裂一樣,自己長出來。

從參數到演算法:設計者退後一步

上一章結尾說,設計者要把更多主導權交給演算法。這一章,就是這件事發生的故事。如果參數化是「定義一套規則,然後拉滑桿調整」,演算法設計則更進一步:設計者寫下一段「程序」,按下執行,讓電腦自己跑出結果 — 而且結果,常常會超出設計者原本的想像。

這兩者的差別,微妙卻關鍵。參數化像是「填一張會自動計算的表格」:模型的結構是你定好的,你只是在調整裡面的數值。演算法設計則像是「寫一份食譜或一套遊戲規則」:你描述的是「該怎麼做」的過程 — 重複、判斷、遞迴、隨機 — 然後讓這個過程自己跑,長出你沒法預先畫出、甚至沒法預料的形。

再往前一步,是「生成式設計」(generative design)。它的關鍵字是「自主」與「湧現」:系統不只執行你的規則,還可能引入隨機、回饋與演化,產生出設計者意料之外的結果。設計者於是越來越像一個「園丁」或「立法者」 — 你播下種子、訂下規則,然後看著一座你沒有逐一設計過的花園,自己長出來。

把這個轉變講得最清楚的,是哈佛的 Kostas Terzidis。他在 2006 年的《Algorithmic Architecture(演算法建築)》裡主張:演算法不只是工具,而是一種新的設計「媒介」與「思維」。重點不再是「我想要什麼形狀」,而是「我要設計一個怎樣的程序,讓形狀自己出現」。設計的對象,從名詞(形)徹底變成了動詞(生成的過程)。

這帶來一個讓很多建築師不安的問題:如果形是電腦「跑」出來的,那作者到底是誰?Terzidis 甚至挑釁地指出,演算法能產生人腦「想不到」的解 — 因為人有偏見、有習慣,而演算法可以窮舉、可以隨機、可以探索我們直覺會跳過的可能。這既令人興奮(突破人的侷限),也令人焦慮(設計者還剩多少主控權)。這個張力,是整章的核心。

為什麼是 2005 到 2015 這十年?因為兩件事到位了。其一,Grasshopper 與 Processing 這類工具,讓「寫一段生成程序」變得人人可及。其二,個人電腦的算力,終於足以在合理時間內,跑完那些重複上百萬次的演算法。當「寫規則」變簡單、「跑規則」變快,演算法設計就從少數人的實驗,變成了一場運動。

這場運動其實有兩條平行的路。一條是「視覺化」的:在 Grasshopper 裡用電池組裝邏輯(上一章談過)。另一條是「寫程式碼」的:真的打開一個程式編輯器,用 Processing、Python 寫出生成的程序。前者直覺、好上手;後者更自由、更有威力。許多最前沿的演算法建築師,最終都從拉電池,走向了寫 code。

演算法設計其實建立在幾個簡單的「基本動作」上:重複(把一件事做一千次)、條件判斷(如果…就…)、遞迴(自己呼叫自己,像碎形)、隨機(擲骰子帶入變化)、回饋(把結果再餵回去)。光是這幾個積木的組合,就能生成出驚人複雜的圖案與形體。複雜,不必來自複雜的規則;它常常來自簡單規則的「大量重複」。這是演算法設計最深的洞見之一。

這個洞見並不新。回想第一章:1970 年代 George Stiny 的形狀文法(用規則生成設計)、John Frazer 的演化式建築(讓設計像生物一樣演化) — 他們其實早就在做演算法設計了,只是當時沒有工具能跑。2005 年後的這一代,等於是拿著終於成熟的工具,去實現前輩在四十年前就寫下的構想。又一次「思想先行、工具殿後」。

演算法設計還打開了一扇新門:「設計即搜尋」。如果你能寫一個程序,生成成千上萬種方案,又能寫一個標準去「評分」,那電腦就能自己去「搜尋」出最好的那一個 — 這就是最佳化與演化演算法,我們會在後面細談。設計於是從「畫一個答案」,變成「定義一個問題,再讓電腦去搜尋答案」。

這場運動也帶來了一種一眼可辨的新美學:Voronoi 圖的細胞網格、反應擴散的斑紋、群集的流動軌跡、碎形的自相似……這些原本是科學圖像的東西,變成了 2010 年前後建築與設計的視覺標誌。它們美、它們「有機」、它們「看起來很演算法」 — 但也很快氾濫成一種套路,這個我們最後再談。

總之,從參數到演算法,是設計者「再退一步」:從「調整一個模型」,退到「編寫一個會生成模型的程序」,再退到「定義一個目標、讓系統自己演化出答案」。每退一步,電腦的自主性就多一分,設計者的角色就更像一個「系統的設計者」,而不是「形狀的作者」。

但要「寫程序」,你總得先會寫 code。對絕大多數沒受過程式訓練的設計師與藝術家來說,這曾是一道高牆。2001 年,兩個 MIT 的學生做了一個小東西,把這道牆推倒了 — 它的名字,叫 Processing。

創意編程:Processing 與一代人的程式啟蒙

如果說 Grasshopper 讓建築師「不用寫 code 也能玩演算法」,那 Processing 做的是相反、卻同樣重要的事:它讓沒有資工背景的藝術家與設計師,真的學會了「寫 code」。

Processing 誕生於 2001 年的 MIT 媒體實驗室,出自 John Maeda 門下兩位學生 — Casey Reas 與 Ben Fry。他們的目標很單純:做一個讓藝術家、設計師能輕鬆上手的程式環境,讓「寫程式」像「畫素描」一樣,成為創作者探索想法的日常媒介。他們把這種實踐,叫做「創意編程」(creative coding)。

Processing 的魔法,在於它把門檻降到最低:打開軟體,寫幾行字,按下執行,畫面上立刻出現一個會動的圖形 — 沒有複雜的設定、沒有嚇人的樣板程式碼。這種「即時看到視覺回饋」的爽感,讓無數對程式心生畏懼的人,第一次體會到寫 code 的樂趣。它把程式設計,從工程師的專業,變成了創作者的畫筆。

它的設計哲學,全寫在用詞裡:Processing 把每一個程式都叫做一張「sketch(素描)」。這個隱喻很重要 — 它告訴使用者:寫程式不必嚴肅、不必完美,你就大膽地塗、快速地試、不停地改。對習慣用草圖思考的設計師來說,這簡直是把程式設計「翻譯」成了他們的母語。

Casey Reas 本人就是最好的示範。他是一位以程式為媒材的藝術家,作品常是一套簡單規則生成的、不斷變化的抽象圖像。對他,程式碼不是達成某個預定畫面的手段,而是一種「描述行為」的語言 — 你描述元素該怎麼互動,作品就在執行中自己長出來。這與演算法建築的精神完全一致:設計「過程」,而非「結果」。

Processing 的影響,是「養出了一整代人」。2000 年代到 2010 年代,全世界的設計學院、藝術學院、建築系,紛紛用 Processing 當作學生程式啟蒙的第一站。許多今天的數位設計師、互動藝術家、創意技術人,他們寫的第一行有意義的程式碼,就是在 Processing 裡畫的一個圓。它默默地,替整個創意產業做了一次程式掃盲。

Processing 還帶起了一整個「創意編程」的生態。想要更高效能、做即時互動與大型裝置的,轉向了 C++ 的 openFrameworks(2005);想要在瀏覽器裡、用網頁技術創作的,則有了 p5.js(2014) — 由 Lauren McCarthy 發起,把 Processing 的精神帶進 JavaScript 與開放網路,讓任何人打開瀏覽器就能寫、能分享。這條線,讓創意編程從桌面走向了網路與行動裝置。

Processing 與 Grasshopper,代表了演算法設計的兩種性格。Grasshopper 是「視覺化」的 — 適合空間思考、不必碰語法,但被框在它的節點與 Rhino 的世界裡。Processing 是「文字的」 — 你得真的寫 code,門檻稍高,但自由度與威力大得多,而且你學到的是「真正的程式設計」,能遷移到任何地方。很多人從 Grasshopper 入門,最後為了突破限制,走向了寫程式。

為什麼設計師該會寫 code?因為工具總有邊界。當你只會用現成軟體,你就只能做出「那個軟體預設能做的東西」。一旦你會寫程式,你就能創造工具本身沒提供的功能、自動化重複的苦工、實現任何你想得到的邏輯。Processing 給設計師的,不只是一個軟體,而是一種「不再被工具侷限」的自由 — 這在工具快速更替的數位時代,是最保值的能力。

創意編程也長出了自己獨特的文化:開源分享、線上社群、把寫程式當成一種表演(live coding)、用演算法做出「每天一張」的生成藝術…… 它和傳統工程的「嚴謹、保密、目標導向」很不一樣,更接近藝術的

「玩耍、分享、過程導向」。這種文化,深深影響了後來的生成藝術、互動裝置,乃至今天的生成藝術與 AI 創作。

對建築而言,Processing 的意義是「鬆土」。它讓一整代設計學生不再害怕程式,養成了「遇到問題,寫段 code 來解」的習慣與膽量。當這些人進入建築實務,他們就能跳出 Grasshopper 的舒適圈,用 Python、用真正的程式,去處理更大、更怪、更個人的演算法問題。Processing 不直接生產建築,但它生產了「敢寫 code 的建築人」。

Processing 的歷史地位,和 Grasshopper 是一對:Grasshopper 讓演算法設計「平民化」,Processing 讓程式設計本身「創意化」。一個降低了做參數化的門檻,一個降低了學寫程式的門檻。兩者合起來,才真正把「運算」這件事,徹底交到了設計世代的手裡。

有了會寫 code 的人、有了能跑程式的工具,接下來的問題是:要寫什麼樣的演算法?最迷人、也最豐饒的靈感來源,不是別的,正是大自然本身 — 植物怎麼生長、鳥群怎麼飛、斑馬身上的紋路怎麼形成。下一節,我們看建築師如何向自然偷師演算法。

向自然學演算法:L 系統、群集、反應擴散、Voronoi

上一節結尾說,演算法最豐饒的靈感來自大自然。這不是浪漫的比喻 — 而是一個深刻的洞見:大自然從不「畫」出最終的形,它只是「執行生長的過程」。一棵樹沒有藍圖,它只是反覆遵循「長一節、分個叉」的規則;一片葉脈、一群飛鳥、一身斑紋,都是簡單規則大量重複後「湧現」出來的。設計者一旦意識到這點,自然就成了演算法取之不盡的教科書。

這套思維有個名字:形態發生(morphogenesis)。它和傳統的「仿生」(biomimicry)不同 — 仿生是「模仿自然的樣子」(把建築做成貝殼狀),形態發生則是「模仿自然生成的過程」(用貝殼生長的規則,去長出建築)。前者抄結果,後者學方法。2000 年代的演算法建築師,大多追求的是後者。AA Emergent Technologies(湧現科技)等研究,正是把這套「向過程學習」的思路,變成系統性的設計方法。

最早被建築界廣泛採用的自然演算法,是 L 系統(L-system)。它由生物學家 Aristid Lindenmayer 在 1968 年提出,用一套簡單的「改寫規則」來描述植物的生長:把一個符號,反覆替換成更複雜的符號串,再翻譯成分枝的幾何。短短幾條規則,就能長出一棵以假亂真的樹、一叢蕨類。對建築師,L 系統是生成分枝結構、樹狀柱、複雜碎形立面的利器 — 它把「生長」直接變成了可執行的程式。

另一個迷人的自然演算法,是群集智能(swarm)。1986 年,Craig Reynolds 寫了一個叫「boids」的程式:每隻虛擬鳥只遵守三條超簡單的規則 — 別撞到鄰居、跟上鄰居的方向、往群體中心靠 — 結果,成千上萬隻 boids 就自己飛出了像椋鳥群那樣壯觀、有機、不可預測的集體流動。沒有人指揮,秩序卻自己湧現。建築師用它來生成流動的人群動線、分布的構件、會自我組織的立面 — 群集教會我們:複雜的整體,可以從簡單的個體規則中自己長出來。

L 系統與群集,共同指向演算法設計最核心的一個概念:湧現(emergence)。湧現,就是「整體大於部分之和」——一個別元素遵循簡單規則,卻在互動中產生出單看任何一個元素都預測不到的複雜行為與形態。對設計者,這既迷人又謙卑:你不再從上而下「設計」那個複雜整體,而是從下而上「培養」它——你只負責寫好個體的規則,然後等待整體自己浮現。

第三個自然演算法,藏在斑馬與豹的皮膚裡。1952 年,電腦科學之父 Alan Turing 在過世前發表了一篇談「形態發生」的論文,提出一個驚人的猜想:動物身上的斑點與條紋,可能來自兩種化學物質「反應」與「擴散」時的相互作用——一個會聚集、一個會散開,兩者拉扯,就自己長出了斑紋。這套「反應擴散」(reaction-diffusion)模型,後來成了生成有機紋理、孔洞、分區圖案的經典演算法。一個關於老虎紋路的數學猜想,六十年後成了建築表皮的生成器。

第四個,是你一定看過、卻未必知道名字的:Voronoi 圖。給定空間裡一堆點,Voronoi 圖把空間切成一塊塊區域,每塊區域都「最靠近」其中一個點——結果就是那種蜂巢、龜裂泥地、長頸鹿斑紋、蜻蜓翅膀般的細胞網格。它在自然界無所不在(因為它常是「最有效率的分割」),也因此成了 2010 年前後最氾濫的演算法美學——從立面開孔、結構分佈到家具紋理,到處都是 Voronoi。

這些自然演算法之所以重要,不只因為它們「好看」,更因為它們常常「有效率」。自然界的形態,是億萬年演化篩選下、用最少材料達成最大效能的結果:一片葉脈的分布,是輸送養分的最佳網絡;一塊骨頭的孔隙,是承重與輕量的最佳平衡。當建築師借用這些生成規則,他借來的不只是美感,還有藏在裡面的「效率」。這條線,直通下一節的「最佳化」,與第五章的材料與結構。

這些演算法能在 2010 年前後爆紅,Grasshopper 的外掛生態功不可沒。想做反應擴散、Voronoi、群集?社群早就寫好了現成的元件,拉進來、接上去就能跑。自然演算法於是從學術論文裡的數學,變成了學生下午就能玩的積木。這當然降低了門檻,卻也埋下一個問題:當生成一個 Voronoi 表皮只要拖三個電池,它還承載著任何「關於自然、關於效率」的思考嗎?還是只剩下一個時髦的花紋?

這就帶出了對這波美學最尖銳的批評:「演算法媚俗」(algorithmic kitsch)。當 Voronoi、反應擴散、碎形變成一鍵生成的視覺套路,它們原本承載的「向自然學習生成過程」的深意,就被掏空了,只剩下一層「看起來很科學、很有機、很未來」的皮。批評者說:這跟 1990 年代「為曲面而曲面」的 blob,其實是同一個病——只是換了一批更時髦的圖案。形式與意義的落差,又一次出現。

但認真的人,把這條路走得很深。AA 的「湧現科技與設計」碩士,系統性地研究自然的形態發生與材料系統,把它們連到真實的結構與製造;Achim Menges 等人(我們下一章會細談)更把「材料自己的行為」也納進這套生成邏輯。對他們,自然演算法不是花紋,而是一套關於「如何用最少資源、長出最有效形態」的嚴肅方法論。

向自然學演算法,本質上是一次設計哲學的位移:從「我來設計這個形」,變成「我來設計一個像自然那樣的生成過程,讓形自己長出來」。它最好的時候,讓建築獲得了自然般的效率與生機;它最壞的時候,淪為一層時髦的演算法花紋。差別,從來不在用了哪個演算法,而在「你到底想透了沒有」。

不過,光是「讓形自己長出來」還不夠。長出來的那一個,是最好的嗎?如果我能生成一千種變體,有沒有辦法讓電腦自己,從這一千種裡,挑出、甚至「演化」出最好的那一個?這就要談演算法設計最強大、也最像魔法的一面 — 演化與最佳化。

演化與最佳化:讓電腦自己找答案

上一節結尾問:生成了一千種變體,怎麼挑出最好的?答案,是把達爾文請進電腦。演算法設計最強大的一招,叫「演化」與「最佳化」 — 你不直接設計答案,而是定義「什麼叫好」,然後讓電腦自己去演化、去搜尋出最好的解。

這背後是一個觀念的大翻轉:「設計即搜尋」(design as search)。想像所有可能的設計方案,構成一個巨大的「解空間」 — 一棟高樓的每種量體、每種開窗,都是這空間裡的一個點。傳統設計,是設計師憑經驗,在這空間裡「挑」幾個點來畫。最佳化則是:定義一個目標(例如「日照最多、結構最省」),讓電腦有系統地在整個解空間裡「搜尋」,找出最好的那個點。

最迷人的搜尋方法,是「演化演算法」(genetic algorithm)。它直接模仿生物演化:先隨機生成一群方案(第一代),用一個「適應度函數」(fitness function)替每個方案打分;留下分數高的,讓它們「交配」(混合彼此的特徵)、「突變」(隨機微調),生出下一代;如此一代代篩選下去,方案會像生物一樣,自己「演化」得越來越好。設計者不設計形,設計者設計「什麼叫好」,然後當演化的旁觀者。

這個想法最震撼的早期示範,來自藝術家 Karl Sims。1994 年,他用演化演算法,在電腦裡「養」出一群會游泳、跳躍、競爭的虛擬生物 — 沒有人設計牠們的形狀與動作,牠們完全是演化出來的,有些長得詭異、卻又合理得驚人。Sims 證明了:演化不只能最佳化數值,還能創造出設計者完全想不到的「形態與行為」。這對建築師是個強烈的暗示 — 也許最好的設計,不是想出來的,而是演化出來的。

把這套魔法交到一般設計師手裡的,又是 Grasshopper。2010 年前後,David Rutten 在 Grasshopper 裡內建了一個演化求解器,取名 Galapagos(加拉巴哥) — 正是啟發達爾文的那群島嶼。用法簡單到驚人:你指定一群「要被調整的滑桿」(基因)、一個「要被最大化或最小化的數值」(適應度),按下執行,Galapagos 就會一代代演化那些滑桿,自己爬向最佳解。突然之間,「演化設計」也變成人人能玩的東西了。

整套魔法的關鍵,全在那個「適應度函數」 — 你怎麼定義「好」。這也是它最危險的地方。電腦極其聽話、也極其愚蠢:你叫它最大化採光,它可能給你一棟全是玻璃、夏天熱死人的房子;你叫它結構最省,它可能給你一個沒人想住的怪形。電腦會精確地給你「你要求的」,而不是「你真正想要的」。最佳化的成敗,從來不在演算法,而在「你有沒有把問題問對」。這逼著設計者,把模糊的設計意圖,翻譯成精確的數學目標 — 而這翻譯本身,就是最難的設計。

真實的設計,從來不只一個目標:你要採光好、又要不過熱、又要結構省、又要造價低 — 而這些目標常常彼此衝突。於是有了「多目標最佳化」:它不給你唯一的最佳解,而是給你一整排「各有取捨」的好方案

(所謂的 Pareto 前緣) — 這個採光最好但較貴、那個最便宜但稍暗。電腦負責找出所有「沒有一項能再改善、而不犧牲其他」的方案,最後的「價值取捨」,還是交還給人。Grasshopper 的 Octopus 等外掛,把這種多目標演化,也帶進了日常。

最佳化還有一種更「物理」的親戚:找形(form-finding)。1960 年代,Frei Otto 用肥皂膜、用懸鏈模型,讓物理自己「算」出最省力的形(第一章提過);而 Daniel Piker 的 Grasshopper 外掛 Kangaroo,把這套物理找形數位化了 — 它在電腦裡模擬張力、重力、彈力,讓一張數位的網,自己鬆弛、垂落、找到力學上的平衡形態。Gaudí 的懸鏈、Otto 的肥皂膜,到這裡變成了即時、可玩的數位模擬。找形,是「讓自然律當設計師」的最純粹形式。

把「最佳化」「找形」連起來看,你會發現它接上了一條很老的線:第一章 1969 年 ABACUS Strathclyde 那條「用電腦評估設計效能」的路。當年只能「評估」一個既有方案好不好;現在,電腦能反過來「生成並演化」出最好的方案。從被動的評估,到主動的搜尋 — 性能導向設計,在這十年真正長出了牙齒,也成了永續、綠建築設計能大量運用運算的根基。

演化最佳化還帶著一層哲學餘味。它意味著:設計者承認自己「不知道最好的答案長什麼樣」,而願意把「發現」的工作,交給一個盲目卻有耐心的搜尋過程。這是一種謙卑 — 也是一種冒險。你可能得到一個絕妙、自己永遠想不到的解;也可能得到一個技術上「最佳」、卻毫無靈魂的怪物。電腦能告訴你「哪個最佳」,卻永遠不能告訴你「最佳是否值得」。

也因此,最佳化有它清楚的天花板。它擅長回答「可量化」的問題(採光、結構、造價),卻對建築裡最重要的那些「不可量化」的東西 — 氛圍、意義、美、人的感受 — 無能為力。一棟把所有數字都最佳化的建築,完全可能是一棟沒人愛的建築。最佳化是強大的僕人,卻是糟糕的主人:它該服務於人的判斷,而不是取代它。

演化與最佳化,把「設計即搜尋」推到了極致:設計者從「畫答案的人」,變成「定義問題、設定目標、然後評判結果的人」。電腦負責在浩瀚的可能性裡搜尋,人負責決定「要搜尋什麼、以及哪個結果值得」。這是人機分工最清楚、也最深刻的一種樣貌 — 它預演了今天我們與 AI 協作時,同樣要面對的那條界線。

理論談夠了。現在,讓我們看看真正把演算法推到極致的那些「工匠」 — 他們用程式生成出人手永遠畫不出的繁複,再用數位機具,把這些繁複真的做了出來。就從 Michael Hansmeyer 那些數百萬個面的柱子說起。

演算法的工匠:從 Hansmeyer 到一代實踐者

理論與工具都備齊了,接下來該看「人」。2005 到 2015 年間,冒出了一批可以稱為「演算法工匠」的設計師。他們不只把演算法當概念談,而是當成一門「新的手藝」,用它做出人手永遠畫不出的繁複 — 然後,再用數位機具,把這些繁複真的做了出來。

把這門手藝推到最極致的,是 Michael Hansmeyer。他用一種叫「細分」(subdivision)的演算法 — 規則簡單到不可思議:把一個面切成更多面,再把切出來的面繼續切,如此遞迴下去。幾輪之後,一根原本樸素的柱子,就長出了數百萬個面、密到肉眼分辨不清的繁複表面。那是一種人類絕對無法手繪、卻又奇異地讓人聯想到巴洛克與自然的「演算法裝飾」。

但 Hansmeyer 最厲害的,是他證明了這些繁複「做得出來」。他的「細分柱」(2010)是把幾千片雷射切割的紙板,一層層堆疊組裝起來的;2013 年,他與 Benjamin Dillenburger 合作的「數位怪誕」(Digital Grotesque),更直接用 3D 列印,印出一整個由兩億六千萬個表面構成的砂岩石窟房間。演算法生成的無限細節,第一次以這種駭人的密度,變成了你能走進去、用手摸到的真實空間。

Hansmeyer 的作品,翻轉了一個現代主義的禁忌:裝飾。現代主義說「裝飾即罪惡」,因為裝飾意味著昂貴的手工。但當裝飾是演算法生成、數位機具製造的,它就「免費」了 — 生成一百萬個細節和生成一個細節,對電腦來說一樣容易。演算法因此讓建築重新有資格談「繁複」與「裝飾」,只是這一次,它不來自工匠的手,而來自規則的遞迴。

如果 Hansmeyer 追求繁複的「表面」,Marc Fornes 追求的是繁複的「結構」。他的工作室 THEVERYMANY 用自寫的演算法,生成由成千上萬片薄鋁板拼成的、色彩鮮豔的自承重殼體 — 薄得像紙,卻靠著精密計算的條紋與彎折,自己站了起來。他的 nonLin/Lin Pavilion 等作品證明:演算法不只能做花紋,還能精打細算地分配材料,讓「結構」與「裝飾」再一次合而為一。

另一群人,把「湧現」推進到了結構本身。Roland Snooks(與 Kokkugia)發展出「行為演算法」:他不直接設計形體,而是設計一群「會自己行動的小代理」,讓它們像群集般互動、沉積,自己長出建築的形與結構。對他,形不是被畫出來的,而是無數微小決策「協商」出來的結果 — 這把第一章 Nicholas Negroponte「機器作為協作者」的夢,推進到了「一群微型機器協作生成建築」的版本。

還有更多。Alisa Andrasek(biothing)把設計推向極端的資料密度,生成出像高能物理場、像神經網絡般繁複的運算場域;Aranda\Lasch 則回到最基本的「單元聚集」,用演算法研究晶體、準晶與圖案如何從簡單規則中生長。這些人各有路數,卻共享一個信念:設計師的工作,是寫出那套「生成的規則」,而不是畫出最終的物。

把這套「演算法工藝」做得最普及、最貼近生活的,是 Nervous System 工作室。兩位創辦人用反應擴散、Voronoi 圖、生長模擬等演算法,生成珠寶、燈具、家具,再用 3D 列印製造、在網路上販售 — 他們甚至做出能整件 3D 列印、印出來就能穿、會自己摺疊收合的「動力學洋裝」。他們完美示範了上一章 Bernard Cache 的「objectile」與「大量客製化」:每一件都不同、卻又能規模化生產。演算法設計,從建築的尺度,一路滲進了日常的物件。

工具這邊也在演化。原本給好萊塢做特效的 Houdini,因為它徹底的「程序化」本質(在 Houdini 裡,幾乎一切都是可回溯、可修改的節點網絡),在 2010 年代被建築運算圈快速採用,用來處理 Grasshopper 難以

應付的大規模、複雜程序化幾何。又一次,建築師從娛樂工業借來了更強的武器 — 從動畫軟體、到遊戲引擎、到 Houdini,這條「挪用」的線從沒斷過。

把這些人放在一起看,會浮現一個關鍵詞:工藝(craft)。我們習慣把「數位」與「手工」對立,但這群人證明了相反 — 演算法是一門新的工藝。它一樣需要長年的鑽研、一樣講究分寸與品味、一樣在「規則」與「材料」之間反覆推敲。差別只在於,他們的「鑿子」,是程式碼;他們的「材料」,既是幾何,也是雷射切割機與 3D 印表機。

這一節其實藏著一個關鍵的依存關係:這些演算法工匠的作品,沒有一件能脫離「數位製造」而存在。Hansmeyer 的細部要 3D 列印或雷射堆疊、Fornes 的鋁板要數控切割、Nervous System 的洋裝要 3D 列印 — 演算法負責「生成」那無法手繪的繁複,而數位機具負責把它「做出來」。少了後者,前者永遠只是螢幕上的圖。這,正是下一章「數位製造」要登場的原因。

當然也有批評。當「生成過程」本身成了賣點,有些作品陷入一種「過程崇拜」:設計師熱衷於展示「這是用多複雜的演算法長出來的」,卻說不清楚「它為什麼該長成這樣、它對使用者有什麼意義」。繁複,變成了一種炫技;演算法,變成了一張「我很前衛」的證書。最好的演算法工匠,和最空洞的演算法炫技,用的常常是同一套工具 — 差別,永遠在那個老問題:你想透了沒有。

不過,這群工匠多半還停在「生成幾何」。有沒有可能,把演算法推得更遠 — 不只生成形狀,而是生成「活的、會生長的材料」本身?在這個十年的尾聲,有人開始用真菌、用細胞,把「生成」從螢幕裡,帶進了生命本身。

從生成到生長:演算法走向物質與生命

演算法設計這十年,大部分時間在「生成幾何」 — 在電腦裡長出形狀,再用機器做出來。但在 2010 年代中期,有人開始問一個更激進的問題:能不能讓演算法,不只生成「形狀」,而是生成「活的、會自己生長的材料」?

2014 年,David Benjamin 與他的事務所 The Living,在紐約 MoMA PS1 的庭院裡,蓋了一座叫 Hy-Fi 的塔。乍看,它是一座優雅的、由白色磚塊砌成的有機曲塔 — 但那些磚,不是燒出來的,是「長」出來的。

那些磚的原料,是農業廢棄的玉米稈,加上蘑菇的菌絲體(mycelium)。把它們填進模具,菌絲會像樹根一樣在幾天內生長、纏繞,把鬆散的玉米稈「黏」成一塊堅固的磚 — 不需要高溫燒製、不耗能、零碳。塔的展期結束後,這些磚被打碎、堆肥,完全回歸土壤。Benjamin 把運算設計(塔的形與結構是演算法生成的)與「生物製造」(磚是生物長出來的)接在了一起。

Hy-Fi 標誌著一個觀念的躍遷:從「生成設計」到「生長設計」。前面所有的演算法,生成的都還是「指令」 — 告訴機器去切、去印、去組裝一個被動的材料。而 Hy-Fi 暗示了另一種可能:設計者設定條件(養

分、模具、環境),讓「生命自己」去生長出材料與形體。設計者於是更像一個「培育者」,而材料,從死的,變成了活的。這條「生物 × 運算」的線,會在第五章 Neri Oxman 的作品裡,綻放得更燦爛。

與此同時,演算法設計的另一支,走向了完全相反的方向 — 大型工業。2010 年代中期,Autodesk 等公司把「生成式設計」(generative design)變成了商業軟體的賣點:你輸入限制與目標(這個零件要承受多少力、用多少材料、怎麼製造),軟體就用演算法,自動生成出成千上萬種、往往長得像骨骼或珊瑚的「有機」方案,供你挑選。

最有名的例子,是 2015 年空中巴士的「仿生隔艙」:用生成式設計長出的飛機隔板,結構像骨骼般鏤空,比傳統設計輕了將近一半 — 省下的重量,意味著每年省下大量燃油與碳排。生成式設計於是從建築學院的前衛實驗,變成了汽車、航太、工業設計裡實實在在省錢省料的工具。演算法,正式從「藝術」走進了「產業」。

但「人人都能生成」也帶來了一個弔詭。當生成一個複雜方案變得這麼便宜、這麼快,「生成」本身就貶值了。網路上充斥著海量「演算法生成」的圖案與形體,美則美矣,卻越來越難打動人 — 因為大家都看得出來,那只是「跑了個程式」。當供給無限,稀缺的就不再是「複雜」,而是「判斷」:在無限的生成結果裡,知道「哪一個值得、為什麼」,成了比生成本身更珍貴的能力。

回顧這整個「演算法的十年」,它最大的成就與最大的隱憂,其實是同一件事:它讓「生成複雜」變得太容易了。成就是:設計者獲得了前所未有的生成能力,能探索人腦想不到的形態與解。隱憂是:當複雜唾手可得,形式與意義的落差也被放到最大 — 一個 Voronoi 表皮、一根百萬面的柱子,到底是深刻的設計,還是時髦的噪音?這個問題,演算法本身永遠回答不了。

這十年留下兩條清楚的伏筆。第一條,指向「製造」。我們一再看到:演算法生成的繁複,只有在能被「做出來」時才有意義 — Hansmeyer 要 3D 列印、Fornes 要數控切割、Benjamin 要菌絲生長。演算法與製造,是一體兩面。當設計的複雜度被演算法解放,營造的能力就必須跟上。下一章「數位製造」,講的就是機器手臂與 3D 印表機,如何把這些演算法真正「蓋」出來。

第二條伏筆,指向「學習」。注意這整章的姿態:設計者「寫下」規則,電腦「執行」規則。無論多自主的演化、多複雜的湧現,那套規則,始終是人寫的。但有沒有可能,連「規則」本身,都讓電腦從海量的資料裡「自己學」出來?當這個問題被認真提出,我們就從「演算法時代」,跨進了「人工智慧時代」。生成式設計與最佳化,正是今天生成式 AI 最直接的觀念祖先。

從第三章到第四章,設計者的角色一路位移:參數化時代,他是「規則的編寫者」;演算法時代,他更像「目標的設定者」與「結果的評判者」。電腦的自主性一步步增加,人退到了越來越上游的位置 — 不再管「怎麼做」,只管「要什麼、以及哪個好」。這條「人步步後退、機器步步向前」的軌跡,在下一個 AI 世代,會被推到一個令人既興奮又不安的新極端。

2005 到 2015,是演算法真正「成為設計者」的十年。它把設計從「畫形狀」,徹底變成了「編寫生成形狀的過程」;它讓建築師向自然、向演化、向生命偷師;它也第一次,讓電腦能產出人想不到的東西。設計,從一門關於「結果」的藝術,變成了一門關於「過程」的藝術。

但這一整章的演算法,有一個共同的、未竟的渴望:它們都想「離開螢幕」。一根百萬面的柱子、一座菌絲塔、一件生成的洋裝——它們真正的高潮,都不在生成的那一刻,而在「被製造出來、能用手觸摸」的那一刻。當設計的複雜被演算法徹底解放,壓力就全壓到了營造端:我們到底要怎麼,把這些電腦裡的繁複,變成磚、變成鋼、變成真實的房子?這個問題,把我們帶向機器手臂與 3D 印表機的時代——數位製造。

來源

1. Kostas Terzidis, Algorithmic Architecture — Wikipedia
https://en.wikipedia.org/wiki/Kostas_Terzidis
2. Generative design — Wikipedia
https://en.wikipedia.org/wiki/Generative_design
3. Processing (software) — Wikipedia
[https://en.wikipedia.org/wiki/Processing_\(software\)](https://en.wikipedia.org/wiki/Processing_(software))
4. Casey Reas — Wikipedia
https://en.wikipedia.org/wiki/Casey_Reas
5. p5.js — Wikipedia
<https://en.wikipedia.org/wiki/P5.js>
6. L-system — Wikipedia
<https://en.wikipedia.org/wiki/L-system>
7. Boids (Craig Reynolds) — Wikipedia
<https://en.wikipedia.org/wiki/Boids>
8. Reaction–diffusion system (Turing) — Wikipedia
https://en.wikipedia.org/wiki/Reaction%E2%80%93diffusion_system
9. Genetic algorithm — Wikipedia
https://en.wikipedia.org/wiki/Genetic_algorithm
10. Karl Sims — Wikipedia
https://en.wikipedia.org/wiki/Karl_Sims
11. Michael Hansmeyer / Digital Grotesque — Wikipedia
https://en.wikipedia.org/wiki/Michael_Hansmeyer
12. Houdini (SideFX) — Wikipedia
[https://en.wikipedia.org/wiki/Houdini_\(software\)](https://en.wikipedia.org/wiki/Houdini_(software))
13. The Living / Hy-Fi — Wikipedia
[https://en.wikipedia.org/wiki/The_Living_\(architecture_firm\)](https://en.wikipedia.org/wiki/The_Living_(architecture_firm))

從設計到製造

2010s

2010 年代,電腦終於也改變了「建造」。機器手臂進了建築學校,3D 印表機印出整座橋 — 設計的數位檔案,不再是給人看的圖,而是直接驅動機器的生產指令。形,從螢幕走進了真實的磚、鋼與混凝土。

前面四章,電腦改變的都是「設計」。這一章,電腦終於也改變了「建造」。

數位製造(digital fabrication)講的,是電腦裡的幾何,如何不經過人手的「翻譯」,直接變成真實的材料 — 並且,讓前面所有演算法生成的繁複,第一次有了被蓋出來的可能。

file-to-factory:設計檔案就是生產指令

上一章結尾,所有的演算法都擠在同一道門前,渴望離開螢幕。這一章,門打開了。數位製造(digital fabrication)講的,就是電腦裡的幾何,如何不經過「翻譯」,直接變成真實的磚、鋼、木與混凝土。

先看傳統的營造,有一道危險的鴻溝。建築師畫圖(一套符號與尺寸),營造者讀圖、再用人手與經驗把它「翻譯」成實體。這道翻譯充滿損耗:看錯、做不到、現場將就。形越複雜,翻譯就越容易出錯 — 這正是 1990 年代 blob 蓋得那麼痛苦的根本原因。

數位製造的核心觀念,叫「從檔案到工廠」(file-to-factory):設計的數位檔案,不再是「給人看的圖」,而是「給機器執行的指令」。同一個模型,直接驅動雷射切割機、CNC 銑床、3D 印表機或機器手臂,把幾何一比一地做出來。中間沒有人「翻譯」,也就沒有翻譯的損耗。設計與製造,第一次接成了一條連續、無縫的數位鏈。

這個想法其實不新。第一章我們就看過:1964 年的 IBM DAC-1,已經能把設計輸出去驅動數控工具機;Antoni Gaudí 用直紋曲面,讓彎曲的形能用直工具施工;第二章 Frank Gehry 的 CATIA,讓鈦板從模型直接輸出加工。file-to-factory,是這條「設計即製造」的伏線,在 2010 年代終於成為建築主流的時刻。差別在於:這一次,連「機器手臂」這種通用、可程式的工具,都進了建築的工具箱。

數位製造大致分三類。「減法」:從一塊材料切掉多餘的(CNC 銑、雷射切割、水刀)。「加法」:一層層堆疊出來的(3D 列印)。「塑形」:把材料彎折、成型(數控彎管、熱塑)。再加上「組裝」:讓機器手臂按數位指令,把構件一件件精確地拼起來。這幾種動作的組合,幾乎能對應到所有的營造工法 — 只是現在,執行者從人的手,換成了讀檔案的機器。

其中最關鍵的角色,是「機器手臂」(robotic arm)。它原本是汽車廠裡焊接、噴漆的工業機器人,動作精準、不知疲倦。但它對建築最迷人的地方,是它的「通用性」:同一支手臂,換個工具頭、改段程式,今天能砌磚、明天能切割、後天能 3D 列印、大後天能組裝木構。它不是一台「專用機器」,而是一個「可程式的通用工匠」——這正好對上了數位設計「每個案子都不一樣」的需求。

還記得第二章 Bernard Cache 的「大量客製化」、那個「做一千個不同的和做一千個相同的一樣便宜」的夢嗎?數位製造,就是讓這個夢落地的引擎。傳統工廠靠「模具」量產,所以一致才便宜;數位機具直接讀檔案,改一個參數就做一個不同的件,完全不需要換模。於是「每一片都不同」的非標準營造,第一次在經濟上變得可行。1990 年代的理想,2010 年代才有了實現它的工廠。

file-to-factory 還悄悄改變了建築師的權力。第二章提過,Gehry 說數位模型讓建築師能重新當「營造大師」;到了數位製造,這變得更徹底:當設計檔案直接驅動工廠,設計與製造之間的承包商、製圖員、翻譯者一層層被壓縮,建築師(或他的數位模型)重新握住了從設計到實體的全鏈條。這既是賦權,也是重擔——你不能再說「那是施工的問題」,因為製造的邏輯,已經內建在你的設計裡了。

這帶來一個深刻的轉變:設計與製造,不再是先後的兩件事,而是同一件事的兩面。當你知道「這會被機器手臂砌出來」,你設計的方式就會不一樣——你會順著機器的動作、材料的脾氣去設計。製造的限制與可能,從設計的「下游」,變成了設計的「起點」。這就是接下來幾節,Gramazio Kohler、Achim Menges、Philippe Block 這些人共同在做的事:讓「怎麼做」回到「設計什麼」的核心。

值得注意的是,推動這一切的主力,不是事務所,而是「學校」——特別是蘇黎世的 ETH 與斯圖加特大學。為什麼?因為把工業機器人、把實驗性的製造方法引進建築,需要大量的研究、試錯與不計成本的探索,這是商業事務所負擔不起的。於是 2010 年代的數位製造,主要是在大學的研究室裡,以一座座「研究亭」(research pavilion)的形式,被一年一年推進的。

「研究亭」成了這個時代最重要的一種建築類型。它通常很小、是臨時的、蓋在校園或雙年展裡——但它是一個完整的「實驗」:把一個新的演算法、一種新材料、一套新的機器人工法,完整地走一遍「設計→模擬→製造→組裝」。它不必擔心法規、預算、業主,可以純粹地問:「這樣做,行不行?」十年下來,這些亭子累積的知識,正一點一點滲進真實的建築。

所以,數位製造補上了數位建築最後、也最硬的一塊拼圖。從萌芽期到演算法,我們一直在談電腦如何改變「設計」;而數位製造,是電腦終於也改變了「建造」。當設計檔案就是生產指令,「想得到」與「做得出來」之間那道存在了幾千年的鴻溝,第一次被填平了。

而把這條路走在最前面、第一個把工業機器手臂搬進建築學校的,是蘇黎世兩位教授。他們不只用機器人蓋東西,還提出了一個影響深遠的觀念——在數位的時代,「材料」與「資訊」可以重新合而為一。

機器人進了學校:Gramazio Kohler 與數位物質性

2005 年,蘇黎世聯邦理工學院(ETH)做了一件當時看來有點瘋狂的事:他們把一支工業機器手臂,搬進了建築系。推動這件事的,是兩位教授 — Fabio Gramazio 與 Matthias Kohler(事務所 Gramazio Kohler)。這支手臂,後來改變了整個建築界對「製造」的想像。

為什麼說瘋狂?因為工業機器手臂屬於汽車廠、屬於生產線,跟「設計創意」八竿子打不著。把它放進以手繪、模型、美學訓練為主的建築系,在當時像是把一台堆高機開進畫室。但他們看到了別人沒看到的:這支手臂不是一台笨重的生產機器,而是一個「能直接執行設計檔案」的通用工具 — 它能把建築師腦中的演算法,直接變成磚與牆。

他們的成名作,是 2006 年瑞士的 Gantenbein 酒莊立面。他們讓機器手臂,把兩萬多塊磚,每一塊都以電腦算好的、獨一無二的角度砌上牆 — 整面牆於是浮現出一幅由光影構成的、像葡萄籃又像像素畫的圖案。這種「每塊磚角度都不同」的牆,人手幾乎不可能精準砌出,但對讀檔案的機器手臂,只是照著指令轉動而已。這是史上第一次,工業機器人在真實建築上「砌」出了人手做不到的東西。

從這件作品,他們提出了一個影響深遠的觀念:「數位物質性」(digital materiality)。長久以來,我們把「數位」(虛擬、資訊、螢幕)和「物質」(真實、材料、手感)當成對立。但他們主張:當資訊能直接驅動材料的擺放,數位與物質就融合了一 — 每一塊磚的位置,既是「資料」,也是「實體」。建築的物質性,從此可以被「編程」。

Gramazio Kohler 的研究,核心是探索這支「通用工匠」還能做什麼。除了砌磚,他們讓機器手臂編織、堆疊、組裝木構,甚至控制無人機。其中「飛行組裝建築」(2011)讓一群四軸無人機,合作把六百塊磚塊堆成一座六公尺高的塔 — 把「機器營造」從固定的手臂,推向了會飛的、群集的機器人。又一次,我們看到第四章的「群集」,在真實的營造裡現身。

他們也把機器人推出實驗室、推向工地。「就地製造」(in-situ fabrication)讓機器人直接在工地現場作業;「網模」(Mesh Mould)則用機器人現場彎折鋼絲,做出一個既是鋼筋、又是模板的網,再灌入混凝土 — 一舉解決了「複雜混凝土結構需要昂貴模具」的老問題。這些研究,都指向同一個目標:讓數位製造,從做漂亮的小亭子,走向能蓋真實房子的工法。

機器人要進入建築,還缺一個「翻譯»:建築師習慣用 Grasshopper,而機器人說的是工業控制的語言。2010 年前後,「建築機器人協會」開發了 KUKA|prc 等外掛,讓設計者能直接在 Grasshopper 裡,用他們熟悉的方式,編寫機器手臂的動作。這就像 Grasshopper 當年降低了參數化的門檻一樣 — KUKA|prc 降低了「指揮機器人」的門檻,讓機器人製造,從少數實驗室的特技,變成更多學校與事務所玩得起的工具。

把機器手臂理解成「通用工匠」,其實翻轉了工業革命的邏輯。工業革命用「專用機器 + 標準化」取代了工匠,代價是一致、單調、去技能化。而建築機器手臂是「通用 + 可程式」的:它能像工匠一樣,對每一個

構件做出不同的、回應性的動作 — 只是這份「手藝」,現在寫在程式裡。某種意義上,數位製造想找回的,正是工業化弄丟的那種「每一件都用心對待」的工匠精神,只是改用機器來實現。

但這也帶出一個尖銳的問題:勞動。當機器手臂能砌磚、能組裝,那砌磚的工人呢?支持者說,機器人接手的是危險、重複、傷身的粗活,讓人去做更需要判斷的工作;批評者則擔心,這又是一輪以「效率」為名的去技能化與失業。這個關於「機器取代工匠」的老爭論,在建築工地上,正以新的形式重演 — 而它沒有簡單的答案。

Gramazio Kohler 在 ETH 的研究室(Gramazio Kohler Research),培養與影響了一整代數位製造的研究者,蘇黎世也因此成為全球機器人建造的聖地之一。他們證明了一件事:機器人不是要把建築變得更冷、更工業,反而可能讓建築重新擁有一種「被精心製作」的、近乎手工的豐富 — 只是這雙「手」,是讀著演算法的機械臂。

「數位物質性」這個觀念,其實是對整個數位建築前半段的一次溫柔修正。1990 年代,大家沉迷於螢幕裡的虛擬曲面;Marcos Novak 甚至想讓建築乾脆留在資訊空間裡。而 Gramazio Kohler 把焦點拉回了「物質»:再炫的演算法,最終都要落實成一塊有重量、有紋理、會老化的真實材料。數位的意義,不在於取代物質,而在於讓我們能以前所未有的精度與自由,去「編排」物質。

Gramazio 與 Kohler 把機器手臂帶進學校,做的遠不只是一個新工具的示範。他們重新定義了「製造」在設計裡的位置:製造不再是設計完之後的事,而是設計的一部分;材料不再是被動的填充,而是能被資訊精確編排的、活的媒介。這套「數位物質性」的思維,替接下來所有的數位製造研究,定了調。

如果說 Gramazio Kohler 探索的是「機器人能怎麼做」,那麼斯圖加特的另一群人,問的是一個更根本的問題:形,能不能不要由建築師決定,而是由「材料自己的脾氣」決定?他們向甲蟲、向海膽學習,做出了這十年來最美的一批研究亭。

仿生與材料計算:Menges、ICD/ITKE 與研究亭

上一節結尾的問題是:形,能不能由「材料自己的脾氣」決定?把這個問題鑽研得最深、最美的,是德國斯圖加特大學的兩個研究所 — Achim Menges 的 ICD(計算設計研究所)與 Jan Knippers 的 ITKE(結構研究所)。

他們的核心觀念,叫「材料計算」(material computation),或者說「材料即運算」。傳統設計是「形式決定材料»:建築師先想好造型,再挑材料去實現它,材料是被動的奴僕。Menges 反過來:讓「材料的行為」來決定形式。一片膠合板會往哪個方向彎、一束纖維在張力下會排成什麼、一塊木頭乾燥時會怎麼翹 — 把這些材料「本來就會做的事」算進設計,讓形式從材料的天性中長出來。

為什麼這條路一定要「向自然學」?因為大自然從不浪費。自然界沒有「先設計形狀、再硬塞材料」這回事——一根骨頭、一片葉、一個貝殼,它的形、它的材料、它的結構、它的生長,全是同一個過程裡長出來、且彼此完美整合的。Menges 與 Knippers 的研究,本質上就是想學會自然這種「形態、材料、結構、製造一體成形」的智慧,再用演算法與機器人把它實現。這是第四章「向自然學演算法」,在材料與製造層次的延伸。

他們把這套研究,具體化成一個延續至今的傳統:ICD/ITKE 研究亭(Research Pavilion)。從 2010 年起,他們幾乎每年蓋一座——每一座都鎖定一種自然原型、一種材料系統、一種機器人工法,完整地實驗一遍。這些亭子小巧、輕盈、繁複得驚人,成了全球數位製造研究最受矚目的「年度發表」。

看幾個例子就懂了。2012 年的亭子,師法龍蝦外殼的纖維結構,用機器手臂把碳纖維與玻璃纖維,一圈圈纏繞成輕到不可思議、卻很堅固的殼。2013-14 年的亭子,學的是甲蟲翅鞘(elytra)的雙層纖維構造。還有師法海膽骨板的木構亭、師法水蜘蛛水下氣囊的膜結構亭……每一座,都是把一種生物的「建造智慧」,翻譯成一套可運算、可機器製造的建築系統。

其中最迷人的工法之一,是「無核纏繞」(coreless filament winding)。傳統做纖維構件要先做一個昂貴的模具(核),把纖維纏上去。Menges 的團隊不要模具——他們讓兩支機器手臂在空中對接,直接把浸了樹脂的纖維,一根根憑張力「編織」在空中,纖維自己拉成最有效率的路徑。沒有模具、沒有廢料,結構直接從纖維的張力中浮現。這簡直是把蜘蛛結網,變成了機器人的舞蹈。

ICD/ITKE 真正的貢獻,是把整條鏈「閉合」了:生物原型 → 演算法生成 → 結構模擬 → 機器人製造 → 組裝 → 回饋修正,環環相扣成一個完整的循環。在他們的流程裡,你無法說「這是設計、那是工程、那是製造」——它們是同一件事。這也是「材料計算」這個詞的深意:形與材料,在同一個運算裡同時被決定。

這裡要特別點出 Jan Knippers 與 ITKE 的角色。這些亭子之所以不只是漂亮的雕塑,而是真正「站得住、算得準」的結構,靠的是背後嚴謹的工程。Menges 提供計算設計與仿生的想像,Knippers 確保它在力學上成立、在製造上可行。又一次,我們看到第三章那個教訓:最前沿的數位建築,從來不是建築師一個人的獨秀,而是設計與工程的深度共舞。

這些研究也慢慢走出校園。2019 年德國園藝博覽會(BUGA)的兩座亭子——一座纖維、一座木構——是 ICD/ITKE 多年研究的集大成,尺度更大、更接近真實的公共建築。它們證明:這套「仿生 + 材料計算 + 機器人製造」的方法,不只能做一次性的實驗,也開始能撐起人們真的會走進去使用的空間。

材料計算還有一層越來越重要的意義:永續。當形式順著材料的天性生成,你往往能用「最少的材料」達到「最強的結構」——因為你不再跟材料對抗,而是順勢而為。在一個營建業碳排驚人的時代,這種「用智慧省材料」的思路,從一種美學追求,變成了一種倫理責任。Menges 近年也越來越往「木構」與生物基材料走,正是這個方向。

但也有清醒的批評,常被戲稱為「研究亭症候群」:這些亭子美則美矣,卻大多很小、是臨時的、用昂貴的客製纖維與大量人力研究堆出來的 — 它們離「能規模化、能蓋成你我住的房子」,還有很長一段路。批評者問:十年蓋了十幾座驚豔的亭子,但其中有多少,真的改變了一般人蓋房子的方式?這個「實驗室與工地之間的鴻溝」,是整個數位製造領域共同的痛點。

儘管如此,Menges 與 ICD/ITKE 立下的標竿是清楚的:他們示範了一種設計的最高整合 — 讓形態、材料、結構、製造,像在自然裡那樣,從同一個運算過程中一起長出來。這是「材料計算」獻給數位建築最深的一課:最好的形式,不是被加諸於材料的,而是從材料的天性中,被「算」出來的。

ICD/ITKE 向「纖維」與「生長」學習;而在蘇黎世,另一位研究者把目光投向了最古老的材料 — 石頭,以及一個最古老的問題:不靠鋼筋,光靠「形狀」,一塊石頭能撐多遠?他的答案,美得讓人屏息,也把我們帶回了 Antoni Gaudí 的懸鏈。

用幾何省材料:Block 的薄殼、Armadillo Vault 與 Striatus

上一節說到石頭。在蘇黎世 ETH,Philippe Block 與他的 Block Research Group 鑽研的,是一個聽起來很復古、卻極其當代的問題:如何不靠鋼筋,光憑「形狀」,讓最廉價、最古老的材料 — 石頭、磚、無筋混凝土 — 撐起大跨度的結構?

他們的核心信念,可以濃縮成一句話:「以幾何換材料」(strength through geometry)。現代營造習慣用大量的鋼與鋼筋混凝土去「硬扛」各種力 — 簡單粗暴,但鋼與水泥的碳排極高。Block 反其道而行:如果一個結構的「形狀」設計得夠聰明,讓它內部只有「壓力」、沒有會把脆性材料拉裂的「拉力」,那它就能用很少的材料、甚至用不能受拉的石頭,撐起驚人的跨度。形狀,可以取代材料。

聽起來耳熟嗎?這正是第三章 Antoni Gaudí 懸鏈模型、與第四章 Frei Otto、Kangaroo 找形那條線的直系後裔。Gaudí 用繩子和重力,「類比」地找出純受壓的拱形;Block 則用電腦,把這套「找形」數位化、精確化、立體化。他發展的「推力網絡分析」(Thrust Network Analysis),能為任意的三維曲面,算出力如何在其中流動、以及怎樣的形狀能讓這些力全部維持在「壓力」狀態。Gaudí 的繩子,變成了 Block 的演算法。

推力網絡分析強大在哪?它把十九世紀的「圖解靜力學」(用畫圖來分析力)推進到了三維、且電腦化。設計師可以即時地探索:這片殼如果這樣彎、那樣開洞,力會怎麼重新分布?哪一種形狀,能讓整個結構都處於穩定的純受壓?這讓「設計一個只靠壓力站立的殼」,從少數天才的直覺,變成了可運算、可調整、人人能學的方法。

這套方法最攝人心魄的展演,是 2016 年威尼斯建築雙年展的 Armadillo Vault(穿山甲拱)。它由 399 塊石灰岩,乾砌而成 — 沒有鋼筋、沒有水泥砂漿,塊與塊之間只靠精確的幾何彼此頂住。它跨距達 16 公尺,最薄處卻只有 5 公分,薄得像蛋殼。它靠的,純粹是「形狀」算得夠準,讓每一塊石頭的重量,都恰好順著拱的

幾何,變成壓住彼此的力。它像一個來自未來的中世紀拱頂 — 用最尖端的運算,復興了最古老的營造智慧。

為什麼在 2016 年,要費這麼大勁去蓋一個「石頭拱」?因為它是一個關於「未來」的論證。Block 想證明:面對氣候危機,營建業不能再無止盡地揮霍鋼與水泥。Armadillo Vault 說的是:如果我們願意把「運算」花在「形狀」上,我們就能用低碳、在地、甚至可拆解回收的材料(石頭),蓋出又輕又強的結構。這是把最高的數位技術,用在最樸素、最環保的目的上。

這套思路的最新一步,是 2021 年的 Striatus 印製石橋。BRG 與 Zaha Hadid 的運算團隊(ZHA CODE)合作,用 3D 列印混凝土,印出一座無鋼筋的拱橋。關鍵在於:混凝土被「一條條」地印在精確計算的方向上,讓力順著列印的紋理流動,整座橋因此只靠壓力站立、不需鋼筋,還能被拆開、材料重新利用。Striatus 等於把 Block 的「以幾何換材料」,和最新的「3D 列印混凝土」結合 — 古老的拱、最新的列印,在一座橋上相遇。

順帶一提,「3D 列印混凝土」(construction 3D printing)本身,正是這十年數位製造最受矚目的方向之一:用巨大的機器手臂或龍門機,像擠牙膏一樣一層層堆出牆與結構,省模板、省人力、能做複雜形狀。但它也有個大問題 — 列印的混凝土通常不能放傳統鋼筋,結構上很受限。而 Block 的「純受壓幾何」,恰恰是這個問題的漂亮解答:既然不能用鋼筋抗拉,那就用「形狀」讓結構根本不產生拉力。Striatus 正是這兩者的完美聯姻。

Block 的工作,標誌著「結構工程師」在數位建築裡角色的再一次提升。從第三章的 Cecil Balmond、佐佐木睦朗,到這裡的 Block,我們看到一條清楚的線:在數位時代,最有趣的建築創新,越來越多來自「結構」與「材料」,而不只是「造型」。當形狀可以被精確計算,結構邏輯就能直接變成空間的美 — 力的流動,本身就是建築。

把 Menges 的「材料計算」與 Block 的「以幾何換材料」放在一起,會看到 2010 年代數位製造一個共同的、越來越響亮的主題:永續。早期的數位建築(blob、參數化地標)常被批評為奢侈、浪費、為形式而形式;而這一代研究者,正努力把運算的威力,從「做出更炫的形」,轉向「用更少的資源、做出更負責任的建築」。數位,從炫技的工具,開始長出倫理的重量。

當然,同樣的批評也適用:Armadillo Vault 是個雙年展裝置,Striatus 是座小步橋。它們是壯麗的「概念證明」,但要讓「以幾何換材料、低碳營造」成為工地的日常,還需要克服法規、產業慣性與技術成熟度的重重關卡。這些研究指出了一個正確、迷人的方向,但路還很長 — 理想已經被清楚地畫出來了,接下來,是漫長的落地。

Block 與 BRG 給數位建築的禮物,是把它和一個更大的命題接了起來:地球。當運算能讓我們「以智慧換材料」,數位製造就不只是關於酷炫與精確,更是關於「如何用最少的代價,蓋出該蓋的東西」。從 Gaudí 的繩子,到 Block 的演算法,跨越一個世紀的「找形」,最終指向的,是一個更輕、更省、更該被我們追求的未來。

Menges 學纖維、Block 算石頭,而在大西洋彼岸的 MIT,有一位設計師走得更遠、也更詩意:她不只想用機器製造材料,還想讓設計回到「材料如何在自然中生長」的層次 — 結合蠶、結合菌、結合多材質列印。她的名字,是 Neri Oxman。

列印一棟房子:Oxman 的材料生態與 3D 列印的疆界

Achim Menges 用機器人纏纖維、Philippe Block 用幾何省石頭;而在 MIT,Neri Oxman 把數位製造推向了一個更詩意、也更激進的疆界 — 她不只想「製造」材料,還想讓設計回到「材料如何在自然中生長與分布」的層次。

Oxman 在 MIT 媒體實驗室的研究群叫「中介物質」(Mediated Matter),她提出的概念是「材料生態」(Material Ecology)。傳統製造,把材料當成均質、被動的東西 — 一塊鋼到處都一樣硬。但自然不是這樣:一根骨頭,密的地方密、鬆的地方鬆;一片葉子,該硬的地方硬、該軟的地方軟。材料的「性質」,在自然裡是連續變化、隨需求分布的。Oxman 想用數位製造,做到同樣的事。

她最招牌的工具,是「多材質 3D 列印」:在同一次列印裡,讓材料的軟硬、透明、顏色連續地漸變。想像一根樑,受力大的地方自動變硬、不受力的地方變軟變透 — 不是用好幾種零件拼起來,而是一體成形、性質連續變化。這把上一節 Menges「讓材料行為決定形式」的理想,推進到了「連材料的內部性質,都能被逐點設計」的程度。

她最美的作品之一,是 2013 年的 Silk Pavilion(蠶絲亭)。團隊先用機器手臂,在一個半球骨架上鋪出絲線的底圖;然後 — 這是神來之筆 — 放上 6500 隻活蠶,讓牠們在這個骨架上,把空隙一點點吐絲填滿。最後的亭子,是「機器人 + 活體蠶」共同織出來的。Oxman 在這裡示範了一種驚人的協作:設計者搭好舞台、定好規則,讓「生命本身」成為製造的一部分。這是第四章 David Benjamin 菌絲塔那條「生物製造」線,最優雅的延伸。

Oxman 的探索一路擴張:她做過能列印「透明玻璃」的印表機,讓玻璃也能像塑料一樣被自由塑形;也做過用蝦殼裡的甲殼素、用纖維素做成的「Aguahoja」展亭 — 它們能在雨水中慢慢分解、回歸自然,挑戰「建築必須恆久」的執念。對 Oxman,終極的設計,是像自然一樣:生長、服務,然後優雅地回到物質循環裡。

同樣在 MIT,Skylar Tibbits 的「自組裝實驗室」則探索另一個科幻般的方向:「4D 列印」。它的意思是,你列印出來的東西,還會隨時間「自己變形」 — 印一根扁平的桿,泡進水裡,它會依預先「程式」好的方式,自己摺成一個立體的形。材料本身,被寫入了「該怎麼變」的指令。Tibbits 想像的未來是:有一天,材料能自己組裝、自己修復 — 製造,不再需要工廠,而是內建在材料裡。

把尺度拉回到「蓋房子」,最受矚目的,是「建築尺度 3D 列印」(construction 3D printing)。這個夢其實很早:1990 年代末,USC 的 Behrokh Khoshnevis 就提出「輪廓工藝」(Contour Crafting),設想用巨大的

噴頭,像擠牙膏一樣一層層「印」出整面牆、甚至整棟房子,他甚至想用在月球、火星上就地取材印出基地。

二十多年後,這個夢部分成真了。像 ICON 這樣的公司,已經能用巨型 3D 印表機,在一兩天內印出一棟小屋的混凝土牆,並把它用在平價住宅、災後重建上。支持者說,這能大幅降低營造的時間、人力與成本,是解決全球住房短缺的希望 — 它也確實省了模板、能做出曲牆。

但現實也很骨感。目前的「列印房子」,絕大多數只是「印牆」 — 樓板、屋頂、門窗、水電,還是得用傳統方法。而且如前一節所說,印出來的混凝土很難放鋼筋,結構受限;列印速度、材料、品質、法規,每一關都還在磨。「印一座城市」的科幻願景,離我們還很遠。3D 列印房子,目前更像是一個「很有潛力、但還在長大」的孩子。

也有人把 3D 列印,推向「在地」與「環保」而非「科幻」。加州的 Ronald Rael(Emerging Objects)用黏土、用泥土、用鹽、用回收材料來 3D 列印 — 他關心的不是「印得多快多大」,而是「印得多在地、多低碳、多有文化」。他用泥土印出的牆,既古老又前衛,提醒我們:數位製造的未來,不一定是冰冷的高科技,也可以是回到泥土、回到地方的。

回到藝術的一端,Benjamin Dillenburger(就是和 Michael Hansmeyer 合作「數位怪誕」那位)持續探索 3D 列印的「極致細節」:用砂質列印,印出傳統工藝永遠做不出的、繁複到極點的裝飾構件。如果說工業 3D 列印追求「快與便宜」,這條線追求的是「繁複與精緻」 — 兩者共享同一台印表機,卻指向完全不同的價值。數位製造,從一開始就同時是「產業工具」與「藝術媒介」。

把這一節放在一起看,3D 列印與材料製造,正在模糊幾條古老的界線:設計與生長(Oxman 的蠶)、製造與生命(菌絲、甲殼素)、人造與自然(可分解的建築)。最激進的願景是:有一天,我們不再「製造」建築,而是「培育」它 — 像種一棵樹那樣,讓建築從程式好的材料與生命中,自己長出來。我們離那裡還很遠,但方向已經被指了出來。

Oxman 在 MIT 養蠶、Block 在蘇黎世砌石、Menges 在斯圖加特纏纖維 —— 數位製造的地圖,看起來像是歐美頂尖學府的專利。但這張地圖,其實正在迅速擴張。在上海,在更多地方,有人把這些技術帶回了自己的土地;而一個更尖銳的問題,也正等著我們:這一切,真的改變了一般人蓋房子的方式嗎?

製造的版圖與批評:從蘇黎世到上海

前面五節,數位製造的舞台幾乎都在歐美頂尖學府 — 蘇黎世、斯圖加特、MIT。但這張地圖正在迅速擴張,而其中最重要的一塊新版圖,就在我們旁邊:亞洲。

在上海,同濟大學的 袁烽(Philip F. Yuan)是把機器人製造帶進華語世界最重要的推手之一。他的事務所(Archi-Union / Fab-Union)用機器手臂砌磚、3D 列印混凝土,做出一系列既前衛、又帶著東方營造意識的

作品。2019 年,他的團隊在上海印出了一座約 26 公尺長的 3D 列印混凝土步行橋 — 當時世界最長的同類橋之一。

袁烽的意義,不只在於技術,更在於「脈絡」。他證明了數位製造不必是西方的舶來品 — 它可以和中國的磚作傳統、和東方的空間意識結合,長出自己的樣子。對台灣與華語地區的建築教育,這是個重要的提醒:我們不必只當技術的追隨者與消費者,也可以把這些工具,接到自己的材料、工法與文化脈絡裡,做出屬於這裡的數位建築。

與此同時,在蘇黎世,數位製造也終於跨出了「研究亭」,蓋出了第一棟真正能住人的房子。2019 年,ETH 把多年的研究成果,集合進一棟三層樓的實驗住宅 DFAB House:它的彎曲承重牆是機器人現場用「網模」做的、樓板是 3D 列印模具灌出來的、木構是機器手臂組裝的……好幾種數位製造工法,第一次在同一棟「真實、有人住」的建築裡協同登場。

DFAB House 是一個重要的里程碑,因為它回答了那個一直懸著的批評:「這些技術,只能做小亭子嗎?」它證明了不 — 數位製造可以整合起來,蓋出符合法規、能住、能用的真實建築。從一座座一次性的研究亭,到一棟能住人的房子,這一步走了將近十五年。

但誠實地說,批評依然成立,而且很重要。第一個批評,是「實驗室與工地的鴻溝」。我們這一章看到的所有驚豔之作 — 纖維亭、石頭拱、蠶絲亭、列印橋、DFAB House — 幾乎全是大學或頂尖機構,用大量研究經費、人力與時間堆出來的「示範」。它們證明了「可能」,卻還遠遠談不上「普及」。你我家附近正在蓋的房子,九成九仍是用幾十年沒變的方式蓋的。

第二個批評更尖銳:這一切,真的讓蓋房子變得更好、更便宜、更普及了嗎?在一個全球都鬧住房危機的時代,數位製造最該證明自己的地方,正是「能不能又快又省又好地,蓋出人人住得起的房子」。3D 列印住宅在這個方向上有些進展,但整體而言,這十年數位製造的光環,更多落在炫目的展演,而非平凡的、巨量的、人們真正需要的住宅上。技術的浪漫,與民生的迫切之間,還有很大的落差。

第三,是勞動與永續的兩面性。機器人取代工人的疑慮(前面談過)依然存在;而「永續」的旗幟下,也有需要誠實面對的帳:養一個機器人實驗室、跑一台大型印表機,本身也耗能;「可分解的建築」「低碳的石拱」很美,但它們目前的規模,對營建業天文數字般的碳排,還只是杯水車薪。方向是對的,但別把「示範性的環保」,誤當成「已經解決了問題」。

該怎麼持平地看這十年?或許這樣說:數位製造在 2005–2020 年間,完成的是「證明與奠基」,而不是「普及與革命」。它證明了「設計什麼,就能造出什麼」在技術上成立;它奠定了機器人、3D 列印、材料計算的方法基礎;它也把「永續」「在地」「生物」這些更大的命題,接進了數位建築。至於把這些變成你我日常的房子 — 那是還在進行、也最該被期待的下一步。

但無論如何,數位製造補上了一塊關鍵的拼圖。回顧整個故事:萌芽期讓電腦進入設計;1990 年代解放了形式;2000 年代用參數化馴服了複雜;演算法讓設計會自己生成;而數位製造,終於讓這一切「離開螢幕、變成

實體」。從 Ivan Sutherland 的光筆,到機器手臂砌的磚,「想得到」與「做得出來」之間那條線,被完整地接了起來。設計與製造,重新合而為一。

然而,這裡有一個諷刺。我們花了五章談的這些 — 炫目的形式、聰明的演算法、精密的機器人 — 其實都是建築界裡最前沿、最少數的「尖端」。真正讓全世界絕大多數建築,每天得以被設計、協調、興建、管理的,是一套遠遠沒這麼浪漫、卻重要無比的東西:資訊的整合。當你的房子要讓建築、結構、機電、水電、營造、業主幾十個人協作,光有漂亮的形與酷炫的機器人是夠的 — 你需要一個大家能共用的「資訊模型」。這,就是下一章 BIM 的主題。

2010 年代,是數位「製造」的十年。它讓建築學會了用機器人的精準、3D 列印的自由、與材料的智慧去建造;它讓設計師的想像力,第一次不再被「人手做不做得出來」所綁架;它也讓數位建築,從一場關於「形式」的遊戲,長出了關於「材料、結構、永續」的重量。電腦,終於不只會畫,也會蓋了。

但接下來,我們要把鏡頭,從這些光鮮的前沿,轉向一個樸實、龐大、卻撐起整個產業的東西。它不性感、不上雜誌封面,卻是今天全世界蓋房子真正的骨幹 —— 它叫 BIM。如果說前五章是數位建築的「藝術史」,那麼下一章,是它的「基礎建設史」。

來源

1. Digital modeling and fabrication — Wikipedia
https://en.wikipedia.org/wiki/Digital_modeling_and_fabrication
2. File-to-factory / Branko Kolarevic, Architecture in the Digital Age
https://en.wikipedia.org/wiki/Digital_architecture
3. Gramazio & Kohler — Wikipedia
https://en.wikipedia.org/wiki/Gramazio_%26_Kohler
4. Robotic arm / Robots in architecture — Wikipedia
https://en.wikipedia.org/wiki/Robotic_arm
5. Achim Menges / ICD Stuttgart — Wikipedia
https://en.wikipedia.org/wiki/Achim_Menges
6. Philippe Block / Block Research Group — Wikipedia
https://en.wikipedia.org/wiki/Philippe_Block
7. Armadillo Vault (Beyond Bending, 2016) — Block Research Group
https://en.wikipedia.org/wiki/Block_Research_Group
8. Neri Oxman / Mediated Matter — Wikipedia
https://en.wikipedia.org/wiki/Neri_Oxman
9. Contour crafting (Behrokh Khoshnevis) — Wikipedia
https://en.wikipedia.org/wiki/Contour_crafting
10. Philip F. Yuan / Archi-Union — Wikipedia
https://en.wikipedia.org/wiki/Philip_F._Yuan

BIM 與整合工作流

1990s - 2010s

前五章是數位建築的「藝術史」 — 形式、演算法、製造。這一章是它的「基礎建設史」。BIM 不上雜誌封面,卻是今天全世界蓋房子真正的骨幹:它把建築從「一堆給人看的圖」,變成「一個所有人共用的資料庫」,讓設計、結構、機電、營造、業主,第一次能圍繞同一個模型協作。

前五章,我們看的是數位建築最耀眼的部分 — 流動的形式、聰明的演算法、精密的機器人。但這些,影響的其實是少數前衛建築。

真正讓全世界「幾乎每一棟」稍具規模的建築都被數位化的,是一套樸實得多、卻重要得多的東西:BIM。這一章,談這個不性感、卻撐起整個產業的骨幹。

BIM 是什麼:從畫圖,到建一個資料庫

前五章,我們看的是數位建築最耀眼的「藝術史」 — 流動的曲面、聰明的演算法、精密的機器人。這一章,我們轉向它樸實、龐大、卻撐起整個產業的「基礎建設史」。它的名字,是 BIM(Building Information Modeling,建築資訊模型)。它不上雜誌封面,卻是今天全世界蓋房子,真正的骨幹。

首先要破除一個常見的誤解:BIM 不是「用 3D 畫房子」。你可以用 3D 軟體畫一棟漂亮的房子,那不是 BIM;你也可以只畫 2D、卻是在做 BIM。BIM 的關鍵,不在於「3D」,而在於那個 I — Information(資訊)。BIM 的本質,是把建築,從「一堆圖」,變成「一個資料庫」。

在 BIM 裡,你畫的不是「一條代表牆的線」,而是放下「一道牆」這個物件 — 而這道牆知道自己是什麼:它多高多厚、用什麼材料、防火時效多久、造價多少、誰負責、什麼時候施工。每一扇門、每一根樑、每一台空調,都是一個帶著一身資料的智慧物件。整棟建築,於是成了一個結構化的、可查詢、可計算的資料庫。

這帶來一個革命性的後果:單一資料源(single source of truth)。傳統上,平面、立面、剖面、明細表是分開畫的 — 改了平面,你得記得手動去改其他所有圖,漏改就出錯。BIM 裡,它們全是同一個模型資料的不同「視圖」:改一處,所有圖面、所有明細,自動同步更新。聽起來耳熟嗎?這正是第一章 Sketchpad「主件/實例」、Charles Eastman 1974 年「建築資料庫」的夢,終於成真。

沒錯,BIM 不是 2000 年代的新發明,它是一個等了三十年的老構想。第一章我們談過,Charles Eastman 早在 1974 年就提出 Building Description System,主張把建築存成資料庫 — 當時太超前,無人問津。直到

電腦夠強、軟體夠成熟,這個構想才以 BIM 的名字復活。Eastman 後來也成了《BIM Handbook》(與 Rafael Sacks 等人合著)的作者,從「被忽視的先知」,變成了這個領域公認的奠基者。

而把這個構想,第一個做成能用的軟體的,其實在更早、更意想不到的地方。1980 年代,在鐵幕後的匈牙利,Gábor Bojár 創立了 Graphisoft,在當時還很弱的個人電腦上,做出了 ArchiCAD — 他稱之為「虛擬建築」(Virtual Building)。早在「BIM」這個詞出現前十幾年,ArchiCAD 就已經在做 BIM 的事:用帶資料的物件,蓋一棟虛擬的、資訊化的建築。BIM 的歷史,比那個詞本身,老得多。

至於「BIM」這個詞本身,是在 2000 年前後才被推廣開來的 — 評論家 Jerry Laiserin 是把它帶進主流討論的關鍵人物之一。一個術語的普及很重要:當散落的「虛擬建築」「建築資料模型」等說法,被收攏成一個響亮、好記的「BIM」,整個產業才有了一面共同的旗,知道大家在談的是同一件事。(這和第三章 Patrik Schumacher 替參數化命名,是同一種「命名的力量」。)

把 BIM 推向全球主流的,是 Revit。它 2000 年問世、2002 年被 Autodesk 收購,核心是「參數化的建築模型」:所有構件都帶著參數與關聯,改一個,牽動全部。Revit 把「關係系統」的思維(第三章參數化的核心),從做造型,搬到了做「一棟完整、可施工的建築」。它後來幾乎成了 BIM 的代名詞 — 這既是它的成功,也埋下了後面我們會談的「壟斷」問題。

這裡要說一句可能讓設計取向的人不舒服的話:對整個營建產業,BIM 的影響,遠遠大於前五章所有炫目的形式探索。blob、參數化地標、3D 列印,影響的是少數前衛建築;而 BIM,影響的是「幾乎每一棟」稍具規模的建築 — 醫院、學校、辦公樓、機場。數位真正、大規模地改變建築這個「產業」,主要不是透過形式,而是透過 BIM 這套不起眼的資訊基礎建設。

BIM 還悄悄改變了「一張圖」的意義。傳統的圖,是給人看的「再現」;BIM 的模型,是給機器算的「資料」。從這個模型,你能自動算出用了多少混凝土、抽出某層的逃生動線、模擬日照與能耗、產出施工順序…… 建築資訊,第一次變得可被查詢、被計算、被分析。建築,從一種「視覺的再現」,變成了一種「可運算的知識」。

但 BIM 不是魔法。一個常被忽略的真相是:BIM 模型的價值,完全取決於「裡面的資訊有多好」。如果建模的人隨便填、亂歸類,那它就只是一個「很重、卻沒用的 3D 模型」 — 業界戲稱 GIGO(垃圾進、垃圾出)。BIM 把「畫圖」變成了「建構資訊」,這其實是更費心、更需要紀律的工作。它的威力,藏在那些看不見的、被認真維護的資料裡。

所以,BIM 的本質,是一次「從再現到資訊」的躍遷:建築不再被存成一疊給人看的圖,而被存成一個給人與機器共用的、結構化的資料庫。這是 Eastman 1974 年的夢,等了三十年,終於落地。而一旦建築變成了「一個大家共用的資料庫」,它就解鎖了 BIM 真正的超能力 — 讓所有人,圍繞同一個模型,協作。

因為說到底,蓋一棟房子,從來不是一個人的事。建築師、結構技師、機電工程師、營造廠、業主…… 幾十、幾百個人,各自帶著自己的專業與圖紙。傳統上,他們各畫各的,到工地才發現「你的管線撞到我的

樑」。BIM 最大的承諾,就是讓這群人,第一次能站在同一個模型前面,一起工作。

一個模型,所有人協作:整合工作流

BIM 把建築變成一個共用的資料庫;而它真正改變產業的地方,是讓所有人能圍繞這個資料庫「協作」。這聽起來理所當然,但在傳統營造裡,協作其實是一場災難。

想像傳統的流程:建築師畫好圖,丟給結構技師;結構技師畫他的圖,丟給機電工程師;機電畫他的,再丟給營造廠。每個人各畫各的一套圖,彼此靠「對圖」來協調 — 而人總會漏。於是最經典的災難,是到了工地才發現:結構的樑,正好擋在機電的風管要走的位置。這時候,要嘛改設計、要嘛現場硬改,都是時間與金錢的巨大損失。

BIM 的第一個殺手級應用,就是解決這個:碰撞檢測(clash detection)。當建築、結構、機電的模型疊在同一個空間裡,電腦能自動找出所有「打架」的地方 — 這根樑撞到那根管、這道牆卡到那台設備 — 在「電腦裡」就抓出來,而不是等到「工地上」才付出代價。光是這一項,就替大型專案省下了難以計數的時間、金錢與糾紛。

協作還能加上「時間」與「金錢」這兩個維度。把施工進度排程連進模型,就是 4D BIM:你能像看動畫一樣,模擬這棟樓會怎麼一層層蓋起來,提前發現「這個吊車到不了那個位置」之類的問題。再把造價連進來,就是 5D:改一個設計,造價即時跟著算。建築模型,於是不只描述「空間」,還描述了「時間」與「成本」。

為什麼要這麼費勁,在設計早期就把所有資訊整合進來?HOK 的 Patrick MacLeamy 用一條著名的「MacLeamy 曲線」講清楚了:在設計越早期,改一個決定的成本越低、影響越大;到了施工階段才改,代價極其高昂。BIM 的整合,就是要把大量的決策與協調「前移」到設計早期 — 趁改動還便宜的時候,把問題都解決掉。這條曲線,成了整個 BIM 與整合設計運動的理論口號。

但這裡有個障礙:傳統的合約,把建築師、營造廠擺在「對立」的位置(你出錯我賺錢),大家防著彼此,誰要先掏心掏肺地共享資訊?於是出現了「整合專案交付」(IPD):一種新的合約模式,讓業主、設計、營造從一開始就綁在同一條船上、共擔風險、共享利益,以鼓勵真正的協作。這點出 BIM 最深的一課:它表面是技術,骨子裡是「人與制度」。

這正是 BIM 最常被誤解的地方。很多人以為「買了 Revit = 導入了 BIM」,結果失敗了 — 因為他們只換了軟體,卻沒改變「流程」與「協作的文化」。真正的 BIM,是一套關於「誰、在什麼時候、共享什麼資訊、負什麼責任」的流程重整。軟體只是載體;沒有協作流程的改變,再貴的 BIM 軟體,也只是一台昂貴的繪圖機。Phil Bernstein 這類同時懂實務與科技的人,一直在強調這件事。

支撐這種協作的,是一整個工具生態。Navisworks、Solibri 用來整合各專業模型、跑碰撞檢測;Tekla Structures 專做鋼結構;BIM 360 等「雲端共同資料環境」(CDE),讓全球各地的團隊能在同一個雲端模型上即時協作。建築的協作,從「交換一疊疊圖紙」,變成了「共同編輯一個雲端的資料庫」— 有點像建築界的共筆文件。

整合協作也攪動了一池「責任」的渾水。當大家共用一個模型,出了錯,是誰的責任?那道沒被抓到的碰撞,是建模的人、檢查的人、還是軟體的錯?當設計與營造的界線變模糊,傳統那套「各司其職、各負其責」的法律框架就開始鬆動。BIM 帶來的協作紅利是巨大的,但它也逼著整個產業,重新協商「責任」與「權利」該怎麼分— 這個過程,至今仍在進行。

對建築師,BIM 是一把雙面刃。一方面,它讓建築師有機會重新成為專案的「資訊核心」與協調者,提升專業地位;另一方面,BIM 也讓建築變得更技術、更需要大量前期的資訊投入,讓一些建築師覺得自己從「設計師」,變成了「資料管理員」。「我到底是在設計建築,還是在填一個巨大的資料庫?」— 這個焦慮,和第三章參數化帶來的焦慮,是同一種。

但 BIM 整合工作流的願景,確實動人:建築、結構、機電、營造、業主,幾十種專業,第一次能站在同一個「會即時同步、會自己抓錯」的模型前面,一起把一棟複雜的建築想清楚、再蓋出來。它把營造這個出了名「破碎、對立、充滿損耗」的產業,推向一種更整合、更協作的可能。這個願景能實現多少,因案、因人、因制度而異— 但方向,是清楚的。

如果說前五章的數位,給了建築「新的形式與新的造法」,那 BIM 給的,是「新的協作方式」。它沒那麼性感,卻可能是這整個故事裡,對「一棟房子如何被一群人做出來」影響最深的一章。它把分散在幾十個人腦中、圖中的資訊,匯流成一個共用的真相。

但一個「所有人共用的資料庫」,立刻引出一個尖銳的問題:這個資料庫,屬於誰?它用誰的格式?如果全世界的建築資料,都鎖在一兩家公司的軟體裡,會發生什麼事?BIM 的故事,於是不只是技術,還是一場關於「開放與壟斷」的角力。

開放還是封閉:IFC、openBIM 與 Autodesk 的版圖

上一節結尾的問題很尖銳:一個「所有人共用的資料庫」,到底屬於誰?用誰的格式?這聽起來像枯燥的技術細節,其實是 BIM 時代最關鍵的權力問題之一。

先想一件事:一棟建築,可能要站五十年、一百年。但軟體呢?軟體版本幾年一換,公司會倒、格式會被淘汰。如果一棟醫院的全部資訊,只存在某家公司某個版本的私有格式裡,那二十年後,當這個格式不再被支援,這些資訊還讀得到嗎?這不是科幻— 它是每個管理大量建築資產的政府與業主,真實的惡夢。建築資料的「壽命」,必須比任何一家軟體公司都長。

為了解決這個,業界發展出「開放格式」:IFC(Industry Foundation Classes,工業基礎類別)。它是一種中性的、不屬於任何一家公司的 BIM 資料格式 — 就像 PDF 之於文件、JPEG 之於圖片。理論上,不管你用哪家軟體,都能把模型匯出成 IFC,讓別人用別的軟體打開。它由一個叫 buildingSMART 的國際非營利組織維護,是 BIM 資料能跨軟體、跨國界、跨時間流通的基礎。

圍繞 IFC,buildingSMART 推動了一整套「openBIM」的理念:主張 BIM 的協作,應該建立在開放、中性、公開的標準上,而不是被綁在某一家廠商的封閉生態裡。openBIM 的精神,和開放網路、開源軟體是一脈相承的:基礎建設層級的東西,應該是公共財,而不是私產。

但現實,遠比理念骨感。BIM 的世界,長期由一家公司高度主導:Autodesk。它的 AutoCAD 壟斷了製圖時代,它的 Revit 又幾乎壟斷了 BIM 時代,再加上 Navisworks、BIM 360 等一整套產品線 — 從畫圖、建模、協調到雲端協作,一家公司幾乎包了。當一個產業的數位命脈,高度集中在一家公司手裡,問題就來了。

最大的問題,叫「廠商鎖定」(vendor lock-in)。當全行業都用 Revit、你的協力廠商都用 Revit、你的員工只會 Revit,你就很難不用 Revit — 即使它漲價、即使它創新變慢。你的資料、流程、人才,都被綁在那個生態裡,換不掉。2020 年,一群全球頂尖的大型事務所,甚至公開發表致 Autodesk 的聯名信,抗議它「價格越漲越高、產品卻越來越停滯」 — 這正是壟斷的典型病徵。

於是,BIM 世界有一場持續的拉鋸:開放 vs 封閉。一邊是 buildingSMART、IFC、openBIM,以及越來越多的開源工具,主張資料自由與互通;另一邊是商業巨頭的封閉生態,用無縫的整合體驗與市場慣性,把使用者留在牆內。這場拉鋸沒有終局,但它的走向,深刻影響著每一個建築人:你的作品、你的資料,究竟是被你自己掌握,還是被鎖在別人的軟體裡?

在這場角力裡,一個關鍵的力量是「政府」。因為政府是最大的建築業主(蓋了無數公共建築),又最在乎資料的長期保存與廠商中立。於是許多國家開始「強制要求」:公共工程必須交付 BIM、而且常常要求是開放的 IFC 格式。英國 2016 年的 BIM 強制令、新加坡、北歐諸國……都用政府的採購力量,去推動 BIM 普及與資料開放。政策,成了塑造 BIM 版圖的一隻看不見的手。

這件事和台灣也切身相關。台灣的公共工程,近年也逐步要求導入 BIM;這意味著,對在地的建築人來說,「會不會 BIM」「懂不懂 IFC 與開放標準」,正從加分項,變成基本門檻。理解這場「開放 vs 壟斷」的角力,不只是國際趨勢的旁觀 — 它直接關係到本地事務所的生存、議價能力,與專業自主。

當協作搬上雲端,「資料治理」也成了新課題。「共同資料環境」(CDE)是一個專案所有資訊匯集、控管、分享的雲端空間 — 但它也帶來新問題:誰能存取?資料存在誰的伺服器上、在哪個國家?出了資安事件誰負責?專案結束後,這些資料歸誰?當建築的全部資訊都數位化、雲端化,「資料主權」就從一個科技業的詞,變成了建築業也必須面對的問題。

也許你會想:這些格式、壟斷、政策,跟「設計」有什麼關係?關係很大。你最珍貴的設計與知識,如果全鎖在一個你無法掌控、可能漲價、可能停滯的封閉系統裡,你的專業自主就是脆弱的。懂得選擇開放的格

式、保有匯出與遷移的能力、不把雞蛋全放在一個籃子裡 — 這在數位時代,和「會設計」一樣,是一種專業的成熟。工具的自由,最終關乎創作的自由。

所以 BIM 不只是技術,它還是一張關於「誰擁有建築資料」的政治版圖。開放標準與商業壟斷的拉鋸、公共政策的介入、資料主權的焦慮 —— 這些不性感的議題,卻決定了數位建築這套「基礎建設」,最終是會成為自由流通的公共財,還是少數巨頭的收費高速公路。這是每個數位時代的建築人,都該有意識去參與的一場角力。

說了這麼多 BIM 的「產業」與「政治」,你可能會懷念前幾章那個充滿創意的運算世界。好消息是:那個世界(Grasshopper、演算法)和這個世界(Revit、BIM),長年各過各的、甚至互不相通;但在這十年,它們終於開始,架起了一座座橋。

BIM 遇上運算設計:Dynamo、Rhino.Inside 與兩個世界的橋

數位建築,長年其實是「兩個世界」。一個是「設計運算」的世界:Rhino、Grasshopper、演算法、做造型、玩生成 — 自由、實驗、屬於設計前期(第三、四章)。另一個是「BIM」的世界:Revit、資料、協調、施工 — 嚴謹、務實、屬於落地的整個產業(這一章)。長久以來,這兩個世界,幾乎不相往來。

這道鴻溝很真實。你在 Grasshopper 裡生成了一個漂亮的參數化表皮 — 然後呢?要把它變成能施工的 BIM 模型,過去常常得「重做一遍」:把幾何匯出、再到 Revit 裡手動重建成帶資料的牆與板。設計運算的成果,進不了 BIM 的流程;BIM 的資料,也回不到設計運算裡調整。兩邊各有一套工具、一套邏輯、甚至一群互看不順眼的人。

為什麼會這樣?因為兩個世界的「關注點」根本不同。Grasshopper 在乎的是「幾何與生成的自由」,它不關心一道牆的防火時效或造價;Revit 在乎的是「資訊與可施工」,它討厭你做出它的物件庫裡沒有的怪形狀。一個要自由,一個要規矩;一個服務想像,一個服務落地。它們的隔閡,某種意義上,正是「設計」與「營造」那道古老鴻溝的數位版。

第一座重要的橋,是 Dynamo。2011 年前後出現的它,本質上就是「Revit 版的 Grasshopper」:一樣是拉節點、寫視覺化程式,但它操作的對象,是 Revit 裡帶資料的 BIM 物件。有了 Dynamo,你能用演算法去自動化那些 BIM 裡最煩人的苦工 — 自動替幾百個房間編號、自動依規則佈置成千上萬個構件、自動從試算表生成模型。參數化的威力,第一次進到了 BIM 的核心。

Dynamo 開啟了一個叫「計算式 BIM」或「BIM 自動化」的領域。BIM 模型雖然聰明,但建模的過程充滿重複勞動;Dynamo 與 Python 腳本,讓建築師能把這些重複「寫成程式自動跑」。一件原本要實習生點三天滑鼠的事,一段腳本幾分鐘跑完。這不只省時 — 它讓設計師在 BIM 裡,也能保有「用規則思考」的那種運算式設計的精神,而不只是當一個手動的建模員。

更徹底的橋,是 2020 年的 Rhino.Inside.Revit。它做了一件近乎魔法的事:讓整個 Rhino 與 Grasshopper,直接「跑在 Revit 裡面」。於是你可以在熟悉的 Grasshopper 裡做自由的參數化設計,而生成的結果,即時變成 Revit 裡帶資料的 BIM 物件 — 兩個世界,第一次能在同一個畫面裡無縫對話。設計運算的自由,與 BIM 的資訊,終於能即時地、雙向地流通。

還有一條走「開放」路線的橋:Speckle。它是一個開源的 AEC 資料平台,想做的是「建築資料的 Git」 — 讓設計資料能在 Rhino、Revit、Blender、各種軟體之間自由地流動、版本控制、協作,而不被任何單一軟體綁死。Speckle 呼應了上一節 openBIM 的精神:與其等大公司開恩讓彼此互通,不如自己建一套開放的管道。互通,從「廠商的恩賜」,變成「社群的自建」。

這些橋的意義,遠不只是技術上的方便。當「設計運算」與「BIM」打通,一件強大的事成為可能:你能用演算法,生成出「既自由、又可施工、又帶滿資料」的建築。前四章那些聰明的生成邏輯,不再止步於漂亮的螢幕造型,而能一路長成可協調、可估價、可興建的真實模型。生成設計與營造資訊,合流了。

退一步看,這座橋接起的,正是這本書的兩條主線。一條是「形式與生成」(從 Sketchpad、blob、參數化到演算法),關於設計的想像力;另一條是「資訊與營造」(從 Charles Eastman、file-to-factory 到 BIM),關於把建築真正做出來。數位建築的前半生,這兩條線常常是分開的 — 而 2010 年代這些橋,讓它們終於開始交織。一個成熟的數位建築人,需要同時站在這兩個世界裡。

但也別把這座橋想得太理想。會 Rhino.Inside、會 Dynamo、會寫 BIM 腳本的人,目前仍是少數菁英;大多數使用 Revit 的人,還是在手動建模。這道橋,目前更像是「高手專用的快速道路」,而非人人都走得上路的大路。設計運算與 BIM 的真正融合,和許多事一樣,理想清晰、落地緩慢。

這對建築教育是個明確的訊號。過去,「設計運算」(Grasshopper)與「BIM」(Revit)常被當成兩門不相干的課、甚至兩種不同性格的人在學。但這座橋說明:未來最有價值的建築人,是能同時駕馭「生成的自由」與「資訊的紀律」、能在兩個世界間自由穿梭的人。把這兩種能力縫起來,正是當代數位建築教育(包括台灣)該努力的方向。

Dynamo、Rhino.Inside、Speckle 這些橋,正在縫合數位建築長年的分裂:讓充滿創意的「設計運算」,與務實龐大的「BIM」,從兩個老死不相往來的世界,變成一個能彼此對話、互相餵養的整體。設計的想像,終於能無縫地,流向營造的現實。

理論談夠了。要看 BIM 與整合協作真正的威力,得去看它最艱難的戰場 — 一棟要協調上千人、上百個系統、高達六百多公尺的超高層。沒有 BIM,這種建築,根本不可能被蓋出來。我們去上海,看一棟會旋轉的塔。

蓋一棟超高層:BIM 的真實戰場

前面四節談了 BIM 的觀念、協作、政治與工具。但要真正體會它的威力,得看它最艱難的戰場——一棟複雜到沒有 BIM 就根本不可能蓋出來的建築。我們去上海,看一棟會旋轉的塔。

上海中心大廈,632 公尺,2015 年完工時是世界第二高樓。它由美國事務所 Gensler 設計,最醒目的特徵,是整棟樓像被人輕輕擰過一樣,從底到頂旋轉了大約 120 度,外面還罩著一層透明的、像衣服一樣鬆鬆套住內塔的雙層玻璃帷幕。它優雅、流動,看起來像一件巨大的數位雕塑。

但這個「扭轉」,不是為了好看——它是「算」出來的。超高層最大的敵人是風;工程師透過風洞測試與電腦模擬發現,這個旋轉、漸縮的造型,能讓風「抓不住」這棟樓,把風的側向荷載減少了約四分之一。光是這一項,就省下了可觀的結構鋼。這正是第四章「最佳化」的思維,用在了世界級的尺度上:形狀,是為了性能而生的。

現在想想要蓋這樣一棟樓的複雜度。一棟扭轉的、雙層皮的、六百多公尺的超高層,意味著:外帷幕上兩萬多片玻璃,幾乎每一片的形狀與角度都不同;裡面有結構、電梯(還是世界最快的之一)、機電、消防、給排水……幾十套系統,要在一個不斷旋轉的幾何裡彼此避讓、完美協調;工地上,成千上萬的工人,要在數年間精確地把這一切組裝起來。這種複雜度,遠遠超出任何人腦或傳統圖紙能掌握的極限。

讓這一切成為可能的,就是 BIM。整棟樓被建成一個龐大的、所有專業共用的資訊模型:那兩萬多片不同的玻璃,由參數化模型生成、編號、定位,再直接送去工廠加工(file-to-factory);各系統的碰撞,在電腦裡一輪輪被抓出來解決;施工順序用 4D 模擬一遍遍演練。沒有 BIM,要協調這種程度的複雜,幾乎是不可能的任務。上海中心,是 BIM 真正大顯身手的舞台。

這點出 BIM 一個關鍵的特性:它的價值,和專案的「複雜度」成正比。蓋一棟簡單的方盒子透天,BIM 的好處有限、甚至嫌它麻煩;但蓋一棟上海中心這種等級的龐然大物,BIM 就從「可有可無」變成「絕對必要」。這也解釋了 BIM 的普及路徑:它總是先在最大、最複雜、最貴的專案(機場、醫院、超高層)站穩腳跟,再慢慢往下滲透。複雜,是 BIM 最好的朋友。

把上海中心,和上一章那些精緻的「研究亭」並排,很有意思。研究亭是小巧的、實驗性的、探索「新形式新工法可不可能」的;上海中心則是龐大的、務實的、處理「巨大複雜度如何被協調」的。前者是數位建築的「前衛實驗」,後者是它的「重工業」。兩者都很數位,卻是數位建築完全不同的兩張臉——一張負責想像,一張負責扛起真實世界的重量。

上海中心也是一個時代與地理的標記。2000-2015 年,全球超高層的重心,明顯移到了亞洲——特別是中國與波灣。這波超高層熱潮,正好和 BIM 的成熟同步:沒有 BIM,這些越蓋越高、越蓋越複雜的塔,在工程協調上根本無解。某種意義上,是亞洲這波瘋狂的建設量體,把 BIM 從一個歐美的概念,逼成了全球營造的標準配備。

上海中心還示範了 BIM 與「模擬」的結合。除了風,還有日照、能耗、人流、消防逃生、結構地震反應……全都能在那個資訊模型上跑模擬、做最佳化。那 21 個「空中花園」中庭,既是社交與生態空間,也是調節雙層皮之間溫度的「熱緩衝」——是經過能耗模擬的設計決策。建築,越來越像一個在蓋之前,就已經被電腦反覆「試營運」過的系統。

要強調的是,上海中心的 BIM 成就,大多是「看不見」的。人們驚嘆它的高度與扭轉的造型,卻很少有人意識到,真正的奇蹟藏在那個沒人看得到的資訊模型裡——在那裡,幾十套系統、幾萬個構件、幾年的工期,被一絲不苟地協調成一個能準時、安全完工的整體。BIM 的偉大,正在於它的「隱形」:它讓極端的複雜,看起來毫不費力。

當然,這種等級的 BIM,也是極其昂貴的:需要龐大的專業團隊、強大的硬體、長時間的前期投入。它是「大專案的遊戲」。對中小型事務所、對一般的住宅與街屋,要不要、能不能用上這麼重的 BIM,一直是個現實的問題。這也預告了下一節要談的:BIM 那些「美好的承諾」,在真實世界裡,兌現得並不平均。

上海中心證明了 BIM 最硬核的價值:它讓人類能協調出,複雜到超越任何個人理解極限的建築。從 Sketchpad 一條會自己保持平行的線,到上海中心兩萬片各不相同、卻完美就位的玻璃——數位建築這條路,在這裡展現了它最務實、也最驚人的一面:不是讓建築更炫,而是讓「不可能協調的複雜」,變得可以被駕馭。

但上海中心落成、剪綵、啟用之後呢?那個耗費無數心力建起來的龐大資訊模型,難道就此封存、變成一堆沒人再看的檔案?還是說,它可以繼續活下去——變成這棟樓一輩子的「數位分身」?這個問題,把我們帶向 BIM 的現在與未來,也帶向它一些尚未兌現的承諾。

從模型到分身:BIM 的現在、未來與破滅的承諾

一棟建築落成,通常也是它的 BIM 模型「死亡」的時刻——交屋、結案,那個耗費無數心力的模型,常常就被封存進硬碟,再也沒人打開。但這其實是巨大的浪費:一棟樓要用幾十年,而關於它最完整的資訊,就鎖在那個沒人再看的模型裡。

於是出現了一個更大的願景:數位分身(digital twin)。它的意思是,讓 BIM 模型不要在交屋時死掉,而是繼續活著,變成這棟建築一輩子的「數位雙胞胎」——透過遍佈建築的感測器,即時反映實體的狀態:這層樓現在多少人、空調耗多少電、哪台設備快壞了。實體建築與它的數位分身,持續同步、互相對話。

數位分身把 BIM 的野心,從「設計與興建」,擴張到了建築的「一輩子」:設計 → 興建 → 營運 → 改修 → 拆除回收,整個生命週期,都建立在同一套持續演進的資料上。對一棟要用五十年的建築,興建只佔成本的一小部分,「營運」才是大頭——而數位分身,正是想用資料,把這最大、卻最被忽視的一段,管理得更聰明。建築,從一個「完工就定型的物」,變成一個「會被持續監測、調節、學習的系統」。

這條線,連到了「智慧建築」(smart building)與物聯網。當建築佈滿感測器與致動器,能感知環境與使用、並自動調節(燈光、空調、遮陽),它就成了一個活的、會回應的系統。讀到這裡你應該再次會心一笑:這不正是第一章 Cedric Price 的 Fun Palace、Gordon Pask 控制論建築的夢想嗎?只是這一次,讓建築「活起來」的,不是機械天車,而是 BIM、感測器與資料。控制論的老夢,在數位分身裡借屍還魂。

但說了這麼多美好願景,該誠實地潑點冷水了。BIM 是數位建築裡「被吹捧得最過頭、也最常令人失望」的一塊。過去二十年,無數白皮書許諾 BIM 會帶來無縫協作、零浪費、生命週期資料貫通……而真實世界裡,BIM 的落地,常常磕磕絆絆、遠不如宣傳。

第一,「無縫協作」常常是假的。即使有了 IFC,不同軟體間的資料交換,至今仍充滿損耗與痛苦;很多專案的「BIM 協作」,實際上是各做各的模型、再費勁地兜在一起。BIM 想消滅的「破碎」,很多時候只是換了個數位的形式繼續存在。理想中的「單一真相」,現實中常是「好幾個彼此矛盾的模型」。

第二,BIM 常淪為「打勾交差」。當政府或業主強制要求交付 BIM,很多團隊做的,只是為了過關而硬擠出一個模型 — 資訊隨便填、做完就丟,根本沒拿來真正協作或營運。這種「BIM 洗白」(BIM-washing),讓 BIM 淪為昂貴的官僚負擔,而非生產力工具。再加上前面說的 GIGO,一個填滿垃圾資訊的 BIM 模型,比沒有還糟。

第三,BIM 的紅利分配極不平均。它在大型專案、大型事務所大放異彩,卻把許多中小型事務所與工班遠遠拋在後面 — 他們沒有資源去養 BIM 團隊、去買昂貴的軟體與訓練。於是 BIM 也可能加劇產業的「數位落差」:強者越強,弱者越難跟上。一個本該提升整個產業的工具,有時反而擴大了鴻溝。

那麼,該怎麼持平看待 BIM?或許是:BIM 真實地、深刻地改變了「大型、複雜建築」的設計與興建方式 — 這點無庸置疑;但它「改變整個產業、貫通建築一生」的宏大承諾,至今只兌現了一部分,而且很不平均。它是強大的基礎建設,不是萬靈丹。它最大的問題,從來不是技術不夠好,而是「人與產業的習慣、制度、利益」改變得太慢。BIM 的瓶頸,是社會性的,不是技術性的。

拉遠來看,BIM 在這整個故事裡,扮演著一個關鍵卻常被低估的角色。前五章那條光鮮的「形式線」很迷人,但若沒有 BIM 這條樸實的「資訊線」,數位設計的成果,根本無法被一個龐大、破碎的產業真正接住、蓋出來。BIM 是數位建築沉默的骨幹 — 它不負責讓人驚嘆,它負責讓事情「真的運轉起來」。一部誠實的數位建築史,不能只寫那些上雜誌封面的曲面,也必須寫這條撐起整個產業的、不起眼的資訊基礎建設。

而 BIM 還留下了一份意想不到的遺產,正悄悄改變未來。想想 BIM 做了什麼:它把成千上萬棟建築,變成了結構化的、機器可讀的「資料」。而「大量結構化的資料」,正是人工智慧最需要的養分。當全世界累積了海量的 BIM 模型、營運資料、數位分身,這些資料,就成了訓練 AI「學會」建築的沃土。BIM 把建築數位化的同時,也無意間,替下一個時代鋪好了跑道。

2010 年代,是數位建築「建立基礎建設」的十年。它沒有 1990 年代的叛逆、沒有 2000 年代的華麗,卻做了一件最根本的事:它把建築,從一疊圖,變成了一個所有人共用、貫穿生命週期的資料庫。它把「資訊」

正式放進了建築的核心。它不浪漫,但它讓數位建築,終於長出了能撐起整個產業的骨架。

從 Sketchpad 到 BIM,六章走下來,有一條線索越來越清晰:設計者,一步步把更多的工作,交給了電腦 — 從畫圖、到生成、到協調。但無論交出多少,有一件事始終是人在做:「寫下規則」。電腦再聰明,規則,終究是人定的。然而 2022 年,這條最後的界線,也開始動搖了 —— 當機器,能從海量資料裡「自己學會」規則,我們就站到了一個全新時代的門口。那是最後一章,也是我們的此刻:人工智慧。

來源

1. Building information modeling — Wikipedia
https://en.wikipedia.org/wiki/Building_information_modeling
2. Charles Eastman / BIM Handbook — Wikipedia
https://en.wikipedia.org/wiki/Charles_Eastman
3. Autodesk Revit — Wikipedia
https://en.wikipedia.org/wiki/Autodesk_Revit
4. ArchiCAD / Graphisoft — Wikipedia
<https://en.wikipedia.org/wiki/ArchiCAD>
5. Integrated project delivery — Wikipedia
https://en.wikipedia.org/wiki/Integrated_project_delivery
6. Industry Foundation Classes (IFC) / openBIM — Wikipedia
https://en.wikipedia.org/wiki/Industry_Foundation_Classes
7. Autodesk — Wikipedia
<https://en.wikipedia.org/wiki/Autodesk>
8. Dynamo (Revit) / visual programming — Wikipedia
[https://en.wikipedia.org/wiki/Dynamo_\(software\)](https://en.wikipedia.org/wiki/Dynamo_(software))
9. Speckle (AEC data) — speckle.systems
<https://speckle.systems>
10. Shanghai Tower — Wikipedia
https://en.wikipedia.org/wiki/Shanghai_Tower
11. Digital twin — Wikipedia
https://en.wikipedia.org/wiki/Digital_twin
12. Building automation / smart building — Wikipedia
https://en.wikipedia.org/wiki/Building_automation

人工智慧的協作者

2018 -

2022 年,機器跨過了一條從未被跨越的線:它開始能從海量資料裡,自己「學會」創造 — 打一句話,就生成一張前所未見的圖。第一章 Nicholas Negroponte 在 1967 年問的那個問題 — 電腦能不能是建築的協作者 — 五十多年後,終於有了答案。只是,這個答案比他想像的,更驚人,也更麻煩。

前六章,設計者一步步把工作交給電腦,卻始終握著最後一樣東西:寫下規則。

這一章,連這條界線都動搖了 — 當機器能從資料裡自己「學」出規則、自己生成,我們站到了一個全新時代的門口。這是最後一章,也是我們的此刻;它會收束這六十年的所有伏線。

機器開始自己學:從寫規則,到學規則

六章走下來,有一條線索越來越清晰:設計者一步步把工作交給電腦 — 畫圖、生成、協調。但始終有一件事,牢牢握在人手裡:寫下規則。電腦再聰明,那套規則,終究是人定的。這一章,講的就是這條最後的界線,如何開始動搖。

前六章所有的「運算」,本質上都是同一件事:人類把知識,寫成電腦能執行的明確規則(畫線的規則、生成形體的規則、協調資訊的規則)。電腦很快、很準,但它不「懂」 — 它只是忠實地執行我們寫好的指令。人工智慧帶來的根本斷裂在於:機器,開始能從海量的資料裡,自己「學」出規則 — 那些連我們自己都說不清、寫不出來的規則。

這就是「機器學習」(machine learning)。傳統程式是「規則 + 資料 → 答案」(你寫規則,電腦套用);機器學習反過來:「資料 + 答案 → 規則」(你餵給它大量的例子,它自己歸納出背後的規則)。你不再教電腦「怎麼畫一張平面」,而是給它看十萬張平面,讓它自己「學會」平面長什麼樣子。知識,第一次不是被「編寫」進電腦,而是被「訓練」出來的。

做到這件事的,是「神經網路」與「深度學習」 — 一種鬆散地模仿大腦神經元的數學結構。給它夠多的例子與夠強的算力,它能自己調整內部數百萬個參數,直到能辨認貓、能翻譯語言、能生成圖像。2012 年前後,深度學習在影像辨識上的突破,點燃了這一波 AI 革命;十年內,它就從實驗室,衝進了包括建築在內的每一個領域。

把「生成」帶進深度學習的關鍵一步,是 2014 年 Ian Goodfellow 提出的「生成對抗網路」(GAN)。它的點子很巧妙:讓兩個神經網路對打 — 一個「生成器」拼命造假圖,一個「鑑別器」拼命抓假;在這場無止盡

的貓抓老鼠裡,生成器被逼著越造越逼真,最後能生成出以假亂真、卻又是全新的圖像。GAN 是第一個讓機器能「無中生有」地創造的深度學習模型 — 它很快就敲開了建築的大門。

大約 2018–2019 年,GAN 開始進入建築研究。研究者拿成千上萬張建築平面、立面去訓練 GAN,看它能不能「學會」並生成新的設計。這聽起來像第一章 George Stiny 形狀文法(用規則生成設計)的夢 — 但有個關鍵差別:形狀文法的規則,是人歸納、人寫的;GAN 的「規則」,是機器自己從資料裡學的,藏在它數百萬個參數裡,連造它的人都未必說得清。設計的「生成邏輯」,第一次變成了一個人類無法完全讀懂的黑盒子。

這個「黑盒子」,是 AI 與前六章所有運算最深的不同。Grasshopper 的定義、BIM 的模型,你都能打開,一個元件一個元件地看懂它「為什麼這樣」。但一個神經網路為什麼生出這張圖、而不是那張,沒有人能完整解釋 — 連模型自己也不會「說明理由」。我們獲得了前所未有的生成能力,代價是失去了「可解釋性」。這在追求邏輯與理由的建築裡,是個既誘人又不安的交換。

AI 不是 2022 年才有的,但 2022 年是它「引爆」的一年。為什麼?因為兩件事到位了:一是「擴散模型」這種新技術,讓生成的品質突然好到驚人;二是這些工具變得人人可用、甚至只要打字就行。又一次,我們看到那個熟悉的模式 — 強大的技術 + 極低的門檻 = 全民運動。和當年的 Grasshopper、Processing 一模一樣,只是這次來得更快、更猛。

把 AI 放進全書的脈絡,會看到一條清晰的軌跡。設計者的位置,一路在「後退」:從畫出形狀(製圖)、到描述關係(參數化)、到編寫生成過程(演算法)、到設定目標讓系統演化(最佳化)…… 每一步,人都把更多「怎麼做」交給機器,自己退向更上游。而 AI,是這條後退之路目前的終點:連「規則」本身,人都交出去了,只剩下最上游的兩件事 — 「要什麼」(下指令)與「哪個好」(做判斷)。

但 AI 這一步,和前面每一步,有質的不同。前面的後退,人交出的是「勞動」(畫圖太累)與「計算」(算不過來);而 AI 這一步,人開始交出的,是「創造」本身 — 那個我們一直以為是人類專屬、機器碰不到的領域。當機器能生成出連我們都覺得驚豔、意外、甚至「有創意」的東西,「設計到底是不是人類專屬」這個從沒被認真懷疑過的前提,第一次鬆動了。這既讓人興奮,也讓人焦慮 — 整章後面,都在處理這份興奮與焦慮。

對建築,AI 的衝擊正在好幾個層次同時展開:它能在幾秒內生成概念意象(下一節);能嘗試生成平面與方案(第三節);能被當成一種全新的創作媒材與美學(第四節);也逼出了關於作者、版權、偏見的尖銳難題(第五節)。這一章,我們一層層往下挖,最後回到一個六十年前就被問過、如今才真正有了答案的問題。

所以,AI 標誌著數位建築的一次根本轉向:從「人寫規則、機器執行」,到「機器從資料學規則、再生成」。電腦,從一個極其聽話的執行者,變成了一個會自己歸納、自己創造、卻又無法被完全理解的「協作者」。這個詞 — 協作者 — 我們在第一 chapter 就聽過,而它的提出者,等了五十多年,才終於等到機器配得上這個稱呼。

而讓全世界在一夕之間,都感受到這股衝擊的,是一個簡單到不可思議的動作:打一句話,按下 enter,幾秒後,一張前所未見的圖,出現在螢幕上。2022 年,建築師們第一次集體意識到:有些東西,真的不一樣了。

一句話生成一張圖:擴散模型與 Midjourney 衝擊

2022 年,對視覺創作的領域 — 包括建築 — 都是天翻地覆的一年。那一年,「文字生成影像」(text-to-image)的 AI,突然好到、也普及到,讓任何人只要打一句話,就能生成出一張精緻的圖。領頭的,是 Midjourney、Stable Diffusion 與 DALL·E。

想像那個時刻:你在輸入框裡打下「一座漂浮在雲端的、由珊瑚與玻璃構成的美術館,日落,電影感」,按下 enter,二十秒後,四張從沒存在過、卻精緻得像專業渲染的圖,出現在你眼前。對花了無數通宵學渲染、建模的建築人來說,這既是魔法,也是某種程度的驚嚇 — 一件原本要好幾天的事,現在一句話、幾十秒就完成了。

這背後的技術,叫「擴散模型」(diffusion model)。它的訓練方式很奇特:拿幾十億張「圖 + 文字說明」的配對,先教模型把一張清晰的圖,一步步加上雜訊、直到變成純粹的雪花;再讓它學會「逆轉」這個過程 — 從一團隨機的雜訊,一步步「去噪」,還原出一張清晰的圖。生成時,你給一句文字當「方向」,模型就從一團隨機雜訊裡,「去噪」出一張符合你描述的圖。它字面意義上,是從「純粹的隨機」裡,長出秩序。

於是一種新的技能誕生了:「提示工程」(prompt engineering)。你給的那句話(prompt),就是你對 AI 下的指令 — 而怎麼下得精準、下得有畫面,成了一門講究的學問。形容詞、風格、光線、鏡頭、藝術家的名字…… 每個詞都會把生成的結果,推向不同的方向。設計者的工作,於是多了一個奇特的新環節:不是畫圖,而是「用文字,把腦中的畫面,精確地描述給機器聽」。

對建築,擴散模型最直接的衝擊,在「發想階段」。傳統上,把一個模糊的概念變成一張能溝通的意象圖,既慢又貴;現在,建築師能在一個下午,生成上百張風格、氛圍各異的概念圖,快速地探索、篩選、激發靈感。它把「發想」的速度與廣度,放大了好幾個數量級。對概念設計、對競圖、對跟業主溝通「感覺」,這是實實在在的生產力。

但這裡有個關鍵的限制,必須說清楚:Midjourney 生成的,是「圖」,不是「建築」。它畫出一個美麗的立面,卻不知道那面牆能不能站、那根柱合不合理、那個空間能不能用 — 它學的是「建築看起來像什麼」,而不是「建築如何成立」。它是一個極其厲害的「效果圖機器」,卻對結構、機能、營造一無所知。把 AI 的美圖誤當成「設計」,是這個時代最常見、也最危險的誤會。

和 Voronoi 圖、和 blob 一樣,當生成變得容易,氾濫與貶值立刻跟上。社群媒體被海量的「AI 建築美圖」淹沒 — 漂浮的、流動的、長滿植物的、像生物又像外星的…… 它們一開始很驚豔,但看多了,你會發現它們有種詭異的「千篇一律」:同樣的炫光、同樣的氛圍、同樣的「AI 感」。當人人都能生成驚豔,驚豔本身,就不再稀奇。

這種「AI 感」從哪來?因為模型是從「整個網路的圖像」裡學的 — 它生成的,本質上是一種「網路視覺的平均值」:最常見、最討喜、最沒有稜角的那種美。它擅長給你「似曾相識的漂亮」,卻很難給你「前所未見的銳利」。這帶出一個深刻的問題,我們第五節會深談:當創作建立在「過去資料的平均」上,它會不會,正在悄悄地讓我們的想像力,越來越趨同?

AI 也重新攪動了「再現」在設計裡的角色。從手繪、到電腦渲染、到現在的 AI 生成,建築師「把腦中的想法變成一張說服人的圖」的方式,又變了一次。有人擔心,當 AI 能瞬間生成美圖,年輕設計師會不會失去「慢慢畫、在畫的過程中思考」的能力?畫圖從來不只是輸出結果,它也是一種思考的方式 — 當這個過程被外包給 AI,我們會不會也外包掉了某部分的「想」?

但換個角度,prompt 也可能是一種新的、有趣的設計語言。用文字去「描述」一個還不存在的空間,本身就是一種設計思考 — 它逼你把模糊的感覺,精煉成精準的語言。有些設計師開始把 AI 當成一個「會視覺化的對話對象」:我說一句、它回四張、我從中挑一個方向再修正…… 這種人與機器你來我往的「對話式設計」,聽起來是不是很耳熟?(對,又是 Nicholas Negroponte 那個提問的回聲。)

純靠文字,你很難精準控制構圖 — 這是早期 AI 圖最大的痛點(你要這個角度,它偏給你那個)。於是工具快速進化:像 ControlNet(2023)這類技術,讓你能用線稿、深度圖、甚至一個粗模型去「約束」生成 — 既要 AI 的豐富,又要設計師的控制。AI 正從一個「你許願、它隨機給」的拉霸機,慢慢變成一個「你能精準指揮」的工具。這個「控制權」的拉鋸,是 AI 進入專業設計流程的關鍵。

2022 年的擴散模型,給了建築一個威力驚人的「發想加速器」,也給了它一身新的麻煩:氾濫、同質、「把美圖當設計」的誤會。它最擅長的,是「生成看起來像建築的圖像」;而它最不會的,是「讓建築真正成立」。要讓 AI 從一個「畫效果圖的」,變成一個真正能參與設計的協作者,還需要把它帶進更深的設計流程裡 — 從生成「意象」,走向生成「方案」。

於是問題變成:AI 能不能不只畫漂亮的圖,而是真的幫忙「做設計」 — 生成可用的平面、合理的量體、甚至協助解決機能與法規?這條把 AI 從「美圖工具」推向「設計夥伴」的路,正是這幾年建築 AI 研究最熱鬧的戰場。

AI 進入設計流程:從生成意象,到生成方案

上一節結尾的問題是:AI 能不能不只畫漂亮的圖,而是真的「做設計」?2022 年的 Midjourney 給了我們驚人的「意象」,但意象不是建築。這一節,我們看一群人如何努力,把 AI 從「美圖機器」,推向真正能參與設計的「方案夥伴」。

走在最前面的探索之一,是 Stanislas Chaillou 2019 年在哈佛的 ArchiGAN。他做了一件比生成意象難得多的事:讓 AI 生成「公寓平面」。他把平面設計拆成三步 — 先生成基地輪廓、再在裡面配置房間、最後擺

進家具 — 每一步都用一個學過上千張真實平面的 GAN 來生成。這是一次認真的嘗試:不是畫一張看起來像建築的圖,而是生成一個有房間、有邏輯、能用的「平面」。

為什麼生成平面,比生成美圖難得多?因為平面有「對錯」。一張概念圖只要好看就行;但一個平面,房間要連得通、廁所要有管道間、走道要夠寬、要符合法規 — 它有無數「硬規則」要滿足。Midjourney 那種「學個大概的樣子」的本事,在這裡不夠用。ArchiGAN 的價值,正在於它逼著 AI 去面對建築「不只是好看、還要成立」的那一面。

在實務界,AI 生成「方案」最成熟的戰場,是「空間規劃」與「可行性評估」。像 TestFit 這樣的工具,能在幾秒內,針對一塊基地,自動生成出幾十種「放得下幾戶、停車夠不夠、賺不賺錢」的配置方案,讓開發商在前期快速試算。Autodesk 收購後推出的 Forma,則把日照、風、噪音的分析與生成,帶進都市與基地尺度的早期設計。這些工具不做美學,做的是又快又準的「理性方案探索」。

另一條路,是把 AI 直接「嵌進」建築師既有的軟體。像 Veras 這類工具,能讓你在 Revit、SketchUp 或 Rhino 裡做好一個粗模型,再一鍵用 AI 把它「渲染」成精緻的意象 — 既保有你對幾何、空間的控制(模型是你做的),又借用 AI 的視覺豐富度。這代表了一種務實的整合思路:不是讓 AI 取代設計,而是讓它接手設計流程裡「最花時間、最不需要創意」的那一段。

讓這種「控制」成為可能的關鍵技術,是 ControlNet(2023)。前面說過,純文字很難精準控制 AI 生成的構圖;ControlNet 讓你能用一張線稿、一張深度圖、或一個粗模型,去「框住」AI 的生成 — 你給骨架,AI 給血肉。對建築師,這是決定性的一步:它把 AI 從一個「你許願、它隨機給」的黑盒子,變成一個「你能精準指揮」的工具。設計師,重新拿回了方向盤。

在學院裡,一整批研究者正系統性地探索「AI × 建築」:Matias del Campo 談「神經建築」、把 AI 當成一種新的設計本體論;Kyle Steinfeld 在柏克萊把 AI 帶進設計教學;新加坡的 Immanuel Koh 研究「建築資訊」如何與機器學習結合;Daniel Bolojan、Andrew Witt 等人,則各自探索深度學習如何增強設計與幾何。AI 在建築裡,正從零星的炫技,長成一個嚴肅的研究領域。

但把這些加起來,一個清醒的共識浮現了:目前的 AI,是「副駕駛」,不是「自動駕駛」。它能加速發想、能跑出大量方案、能接手繁瑣的渲染與配置 — 但它不懂基地的文化、不懂法規的細節、不懂業主沒說出口的渴望、更負不起一棟建築安全與否的責任。它能生成一千個選項,卻不能替你決定「哪一個對」。AI 越強,那個「做最後判斷的人」,反而越重要。

你會發現,AI 生成方案,其實是第四章「設計即搜尋」的升級版。當年 Galapagos 用演化演算法,在你「定義好的」解空間裡搜尋;現在 AI 能從資料裡「學出」一個遠比你能定義的更廣、更豐富的解空間,再讓你在裡面探索。搜尋的「空間」,從人工定義的,變成了機器學習的 — 大了好幾個數量級。但那個古老的分工沒變:機器負責生成與搜尋,人負責定義問題與評判結果。

這對建築這個「職業」,當然帶來焦慮。如果 AI 能瞬間生成意象、配置平面、跑出方案,那畫圖的、做配置的、做表現圖的初階建築人,工作會不會消失?這是真實的擔憂。但歷史也許能給點安慰:從 CAD 到 BIM 到參數化,每一次工具的躍進,都消滅了某些「手藝」,卻也創造了新的角色(BIM 經理、運算設計師…)。AI 大概也一樣 — 它會淘汰某些任務,但會把人推向更上游:策略、判斷、整合、意義。

對台灣與華語地區的設計教育,這其實是個難得的「重新洗牌」時刻。AI 工具大多是新的、開放的、人人都能上手的 — 這意味著,我們不必像過去那樣,落後歐美幾十年才追上某套昂貴的軟體生態。在 AI 這條起跑線上,大家其實站得比以往任何時候都近。誰能更快地把 AI 想清楚、用得深度、接到自己的在地問題上,誰就能在這一波佔到位置。

所以,AI 正努力從「生成意象」,走向「生成方案」 — 從一個畫效果圖的,變成一個能跑配置、能被精準指揮、能當副駕駛的設計夥伴。它還遠不能取代建築師,但它正在快速地、不可逆地,改變建築師「做設計的方式」。重點不再是「要不要用 AI」,而是「怎麼用得聰明、用得判斷」。

但若只把 AI 看做一個「更快的工具」,其實小看了它。在最前衛的探索裡,AI 不只是用來「做設計」的工具,它本身,正成為一種全新的「創作媒材」 — 一種由資料、由「潛在空間」構成的、前所未有的材料。要懂這一點,我們得先認識 AI 最迷人、也最抽象的一個概念:潛在空間。

資料、潛在空間與新的美學

要真正理解 AI 對設計的意義,得認識一個抽象、卻關鍵的概念:潛在空間(latent space)。它聽起來很數學,但其實是 AI 時代最迷人的一個設計想法。

當一個模型「看」過幾十億張圖之後,它在內部建立了一個高維度的「地圖」 — 在這張地圖上,每一個「點」,都對應到一個它能生成的圖像。相近的圖,在地圖上靠得近(所有的「貓」聚成一區、所有的「拱」聚成一區);你在地圖上移動一步,生成的圖就跟著漸變一點。這張壓縮了「所有可能影像」的內部地圖,就叫潛在空間。它是模型把世界「學進去」之後,內心的那個宇宙。

這帶來一個全新的設計動作:在潛在空間裡「散步」。你不再「畫」一個設計,而是在這個由 AI 學出來的、浩瀚的可能性空間裡「移動、探索」 — 從這個拱,平滑地變形到那個穹頂;在「哥德」與「未來主義」之間,找到一個從沒人想過的中間點。設計,變成了一種「導航」:在一個比任何人腦都廣闊的可能性宇宙裡,尋找那些值得停下來的點。

聽起來耳熟嗎?第一章 George Stiny 的形狀文法,生成的是「一整類」設計;第二章 Bernard Cache 的 objectile,是「一族」變體;它們都在說:設計的對象,不是單一的物,而是一個「可能性的空間」。潛在空間,是這個古老想法的終極版本 — 只是這一次,那個「可能性空間」不是人用規則定義的,而是機器從整個人類的視覺文化裡「學」出來的,廣闊到難以想像。

把「潛在空間」當成藝術材料、玩得最淋漓盡致的,是藝術家 Refik Anadol。他拿巨量的資料——一座城市的影像、一間美術館的全部典藏、大自然的紀錄——去訓練模型,再讓模型在潛在空間裡漫遊,生成出不斷流動、變形、像在「做夢」的影像。他把這些投影到建築立面、灌進整個展廳(如他在 MoMA 的《Unsupervised》)。對 Anadol, AI 的潛在空間,是一種能流動、能雕塑的「顏料」。

Anadol 點出了 AI 時代一個深刻的轉變:資料,成了一種材料。過去建築的材料是磚、是混凝土、是光;在 Anadol 手裡,「一座城市累積的數百萬張照片」,本身成了可以被塑形的媒材。這呼應、也超越了第五章 Gramazio Kohler 的「數位物質性」——那時是「資訊驅動物質」,而現在,資訊(資料)本身,就是物質。建築的「材料庫」裡,多了一種看不見、卻無比豐沛的東西:資料。

AI 也帶來了一種前所未有的「美學」。潛在空間的「散步」,催生出一種獨特的視覺語言:流動的、漸變的、像融化又像生長的、介於兩種真實之間的「夢境感」。這種「AI 美學」——也叫「潛在空間美學」——已經開始影響建築的形式想像:不再是參數化那種精確的數學曲面,而是一種更模糊、更有機、更「不確定」的、彷彿從夢裡撈出來的形。

但這種美學,也藏著和前面每一波一樣的陷阱。「AI 感」很快會從新鮮,變成又一種套路——就像 Voronoi 圖、就像 blob。當每個人都在潛在空間裡撈那種「流動的夢境」,它就不再前衛,而變成一種廉價的時髦。AI 美學最有趣的地方,或許不是那個「一看就知道是 AI 做的」的表面風格,而是它背後那個「設計即導航可能性空間」的全新思維方式。

AI 不只能生成虛構,也能「捕捉真實」。NeRF(神經輻射場)與後來的「高斯潑濺」等技術,能用一支手機拍的一堆照片,重建出一個能自由穿梭的、逼真的 3D 場景。這模糊了「真實」與「虛擬」的界線:你能把一棟真實的老建築、一條真實的街,輕鬆地「掃」進電腦,變成可編輯、可被 AI 處理的資料。現實與數位之間那道牆,越來越薄。

把這些放在一起,你會看到 AI 在做一件比「畫圖」根本得多的事:它在把整個「視覺世界」——過去的所有建築、所有影像、甚至掃描進來的真實——壓縮進一個可被探索、可被生成、可被混合的「潛在空間」。設計師於是站在一個奇異的新位置上:他面對的,不再是一張白紙,而是一個已經「學過全人類視覺文化」的、無限的可能性之海。設計,從「無中生有的創造」,越來越像「在已知的海洋裡,航向未知的點」。

這帶來一種哲學上的暈眩。如果一切可能的形式,都已經「潛在地」存在於那個空間裡、等著被你「找到」,那麼設計,還算不算「創造」?還是只是「發現」?這個問題沒有答案,但它逼著我們重新想:在一個機器能生成幾乎一切的時代,人類設計師獨特的價值,到底是「生成」(機器更強),還是「選擇」與「賦予意義」(機器做不到)?這把我們直接帶向下一節最尖銳的問題。

潛在空間,是 AI 給設計最深刻的禮物,也是最大的概念衝擊:它把「所有可能的設計」,變成了一個你能漫步、能探索、能當顏料的真實空間。Refik Anadol 證明了資料能當材料、潛在空間能被雕塑;NeRF 證明了真實能被捕捉進來。AI 於是不只是一個工具,而是一種全新的、由資料構成的創作媒材——一種建築史上從未有過的「材料」。

但這個由「全人類視覺文化」訓練出來的潛在空間,藏著一個我們還沒問的問題:這些資料,從哪來?是誰的?它公平嗎?當我們從這個「海洋」裡撈出一個美麗的設計,那個設計,到底是誰的創作?AI 最迷人的能力背後,是這個時代最棘手、也最不容迴避的一組難題 — 作者、偏見,與版權。

作者是誰?偏見、版權與同質化

上一節結尾的問題,是這整個 AI 時代最尖銳、也最不容迴避的:當我們從那個「學過全人類視覺文化」的潛在空間裡,撈出一個美麗的設計,它,到底是誰的?這一節,我們面對 AI 那些光鮮能力背後的陰影 — 作者、版權、偏見,與同質化。

先問「作者是誰」。當一張 AI 圖完成,誰是它的作者?是打 prompt 的你?是寫出模型的工程師?還是那幾億張「被學習」的圖背後、成千上萬從未同意、也沒拿到一毛錢的藝術家與建築師?這不是哲學空談 — 美國著作權局已經裁定:純由 AI 生成、沒有實質人類創作的作品,不受著作權保護。法律,正艱難地試圖回答一個它從沒遇過的問題:沒有「人」的創作,還算創作嗎?

更激烈的戰場,是「訓練資料」。今天的生成模型,是用網路上幾十億張圖「餵」出來的 — 而這些圖,絕大多數是在創作者不知情、未同意、無補償的情況下被抓取的。於是訴訟接踵而至:Getty Images 告 Stability AI、一群藝術家集體控告 Midjourney 與 Stability…… 核心爭議是:模型「學習」海量受版權保護的作品、再生成相似的東西,這算是合理的「學習」(像人看畫展),還是大規模的「抄襲»?這個問題,法庭還在吵,而它的答案,將定義整個創意產業的未來。

對建築人,這件事很切身:你發表過的作品、你事務所的網站、你得獎的照片 — 很可能,都已經是某個模型的訓練資料了。當別人用 AI 生成出一個「有點像你風格」的設計,你會怎麼想?AI 模糊了「學習一種風格」與「盜用一個人的創作」之間的界線 — 而建築界,還幾乎沒開始認真討論這件事。

比版權更深、更隱蔽的問題,是「偏見」。模型從資料裡學,而資料反映了世界的不平等。網路上的建築影像,絕大多數是西方的、知名的、上鏡的、被大量拍攝與發表的;而廣大的、非西方的、庶民的、在地的營造智慧,在資料裡是稀少、甚至缺席的。於是當你叫 AI 生成「一棟美術館」「一個住宅」,它給你的,往往是那個「資料裡的主流」 — 一種西方的、雜誌封面式的「正常」。

這就導向一個對建築文化來說很危險的後果:同質化。如果全世界的設計師,都用同幾個、學了同一批「主流」影像的 AI 工具來發想,那大家撈出來的東西,會不會越來越像?AI 擅長給你「過去的平均」,而「平均」,正是地方性、獨特性、差異性的反面。一個本該百花齊放的世界建築,有沒有可能,正在被這些「學了同一個網路」的模型,悄悄地、溫柔地,推向一種全球一致的、似曾相識的美?

這對台灣、對所有非西方的建築文化,是個明確的警訊。如果我們只是被動地使用這些「內建西方審美」的工具,我們等於是在用別人的眼睛看自己。但反過來,這也是個機會:我們可以刻意地,用在地的、被主流

忽視的資料去訓練、去微調模型,讓 AI 學會我們自己的空間、材料與文化。AI 是會放大偏見、還是能搶救多元 — 取決於「誰來餵它資料、誰來提問」。這件事,華語世界的建築人不該缺席。

還有一個科幻般、卻真實的隱憂:「模型崩塌」。當網路上越來越多內容本身就是 AI 生成的,而下一代模型又拿這些 AI 內容來訓練 — 等於是 AI 在學 AI 的二手貨。研究發現,這會讓模型逐漸退化、失真、越來越貧乏,像影印件的影印件。這提醒我們:AI 的「創造力」,其實完全寄生於人類真實的創作之上。如果人停止創造、只剩 AI 互相餵食,這口井終會枯竭。人類原創的、真實的設計,比以往任何時候,都更珍貴。

別忘了還有一筆實體的帳:能源。訓練與運行這些龐大的模型,耗費驚人的電力與冷卻用水。在一個談永續、談碳排的時代(第五章 Philippe Block 用幾何省一塊石頭的碳),我們卻一邊用 AI 一秒生成一百張用過即丟的美圖 — 這中間的諷刺,值得每個用 AI 的設計人,誠實地放在心上。數位,從來不是真的「無重量」;它的重量,藏在看不見的資料中心裡。

把這些問題串起來,會看到一個清晰的結論:當「生成」變得無限、免費,真正稀缺、真正有價值的東西,就翻轉了。不再是「能不能做出來」(機器隨便做),而是「判斷」(哪個值得)、「意圖」(為什麼是這個)、「責任」(出了事誰扛)、與「意義」(它對人、對地方,意味著什麼)。這些,恰恰是 AI 給不了的 — 而它們,正是建築師這個專業,在 AI 時代真正的價值所在。

說這些,不是要反對 AI、也不是要回到手繪的鄉愁。AI 是強大、迷人、且不會走的工具,逃避它是愚蠢的。重點是「清醒地使用」:享受它的生成力,同時不被它的偏見綁架、不把它的美圖當設計、不外包掉自己的判斷與立場。對 AI 又愛又怕、卻有意識地駕馭它 — 這是這個時代的設計人,最該修練的成熟。

AI 的能力越驚人,它帶來的問題就越深:作者的消解、版權的混亂、偏見的放大、文化的同質、真實的稀釋。這些不是技術 bug,而是當「創造」被自動化之後,必然浮現的、關於「人」的問題。它們沒有標準答案 — 但身為使用這些工具的人,我們有責任,持續地問。

問到這裡,我們其實已經繞了一個大圈,回到了整本書最開始的地方。因為這些關於「機器能不能懂人、機器與人如何協作」的問題,並不是 2022 年才出現的 — 早在 1967 年,在第一章那隻沙鼠的箱子裡,就已經被問過了。最後一節,讓我們回到那個提問,看看六十年走了一圈,我們到底走到了哪裡。

回到 1967 的提問:六十年,一個循環

我們從一隻沙鼠,結束這趟旅程。第一章,1970 年,Nicholas Negroponte 的 MIT Architecture Machine Group 做了一件「失敗」的裝置 SEEK:一群沙鼠住在方塊堆裡,一支機械手臂試著「讀懂」牠們的意圖、替牠們重新排好被推亂的方塊。結果機器讀不懂沙鼠,越理越亂,當機收場。那隻在故障手臂前發抖的沙鼠,提出了一個當時無解的問題:機器,到底能不能真正理解、並協作於一個活生生的對象?

六十年後的今天,答案以一種 Negroponte 想像不到的方式,來了。今天的機器,真的能對話(ChatGPT)、真的能生成、真的能給你驚喜 — 它在某種意義上,終於配得上 Negroponte 當年那個大膽的詞:「協作者」。1967 年那個近乎科幻的提問,在 2022 年,成了我們每天打開電腦就在做的事。這條從「建築機器」到生成式 AI 的暗線,我們追了整整七章,終於合攏。

但這裡有個動人的反諷:SEEK 的問題,其實沒有被解決,只是被放大了。今天的 AI,生成力強大得驚人,卻依然「讀不懂」那隻沙鼠 — 它不懂你真正要什麼、不懂這塊地的文化、不懂一個空間對人的意義、它有偏見、它會胡謔、它負不起責任。我們造出了一個能言善道、卻不真正「理解」的協作者。SEEK 那隻沙鼠的困惑,換了一身華麗的外衣,又回到了我們面前。機器越像人,「機器到底懂不懂人」這個老問題,就越尖銳。

讓我們把這六十年,快轉一遍。電腦在建築裡的角色,一路在變:在 Sketchpad,它是讓你「畫」的筆;在參數化,它是替你「算關係」的引擎;在演算法,它是替你「生成」的程序;在數位製造,它是替你「建造」的機器手臂;在 BIM,它是讓眾人「協作」的資料庫;到了 AI,它成了會自己「學習與創造」的夥伴。畫 → 算 → 生成 → 建造 → 協作 → 學習 — 電腦,從一塊製圖板,一步步,走成了一個協作者。

而在這條軸線的另一端,是設計者不斷的「後退」。每一個時代,人都把更多的「怎麼做」交給機器,自己退向更上游:從畫線,到定義關係,到編寫規則,到設定目標,到最後 — 在 AI 時代 — 連「規則」都交了出去,只留下「要什麼」與「哪個好」。這趟旅程,可以讀成一個略帶不安的故事:人,在自己的創造物面前,一步步後退。

但也可以反過來讀,而我更願意這樣讀:每一次後退,人其實是被「解放」到更高的層次。當電腦接手了畫圖,人得以專注於關係;接手了計算,人得以專注於生成的邏輯;接手了生成,人得以專注於目標與判斷。機器負責「怎麼做」,人就更能專注於那個機器永遠回答不了的問題:「為什麼、為了誰、值不值得」。後退,不是撤退,而是把自己,釋放到更接近「意義」的位置。

這七章,也藏著幾個一再重演的旋律,值得記住。其一,「思想總是早於工具」:George Stiny 的文法、John Frazer 的演化、Luigi Moretti 的參數、Charles Eastman 的資料庫 — 都比能實現它們的工具,早了二三十年。其二,「每一波新工具,都先解放形式、再氾濫成套路、最後逼出意義的追問」:從 blob、到參數化、到 Voronoi 圖、到 AI 美圖,同一個故事演了四遍。其三,「工具會普及,門檻會崩塌」:Grasshopper、Processing、Midjourney,一次次把菁英的特技,變成全民的日常。讀懂這些旋律,你就不會被下一個新工具的炒作沖昏頭 — 因為你見過它的祖先。

還有一條貫穿全書的焦慮:形式與意義的落差。從 1990 年代「為曲面而曲面」的質疑,到今天「AI 美圖是不是空洞」的擔憂,這個落差從未消失。但或許,這正是答案本身:技術,永遠只能解決「形式」與「效率」;而「意義」,永遠是人的責任。每一代的數位工具,都讓「做出漂亮東西」變得更容易,也就讓「做出有意義的東西」這件人的工作,顯得更重要、更稀缺。工具越強,人的判斷,就越是那個不可取代的核心。

所以,如果你是正在讀這個網站的設計科系學生,這六十年的故事,最想對你說的,其實很簡單。你現在拚命學的某個軟體 — Rhino、Revit、Midjourney — 幾乎可以肯定,會在你執業的某一天過時,被新的東西取代,就像 AutoCAD 取代了鴨嘴筆、Grasshopper 收編了寫程式。所以,別把自己,只練成某個工具的熟練操作員。

真正會跟著你一輩子、不會過時的,是工具底下的那些東西:把設計看成「一套關係與規則」的思維(這從 Sketchpad 到今天都沒變)、面對任何新工具都能快速看穿它本質的能力、對「為什麼這樣設計」的執著、對人與土地的關懷,以及在無限的生成裡,判斷「什麼才值得」的品味與立場。工具是會換的詞彙,這些,才是不變的文法。

還有一件事。這整部數位建築史,主角幾乎都在歐美 — MIT、蘇黎世、倫敦。但故事還沒寫完,而續篇,正握在你手裡。AI 時代難得地把起跑線拉近了:工具開放、知識公開、社群全球同步。把這些工具,接到我們自己的氣候、我們的營造、我們的城市與文化裡 — 做出屬於台灣、屬於這裡的數位建築 — 這一頁,沒有人能替我們寫,只能由此刻、在這裡的我們,自己動手。

從 1963 年 Ivan Sutherland 那支在螢幕上畫下第一條線的光筆,到 2022 年你打進輸入框的那一句 prompt — 六十年,電腦從一塊冰冷的製圖板,變成了一個會與你對話的協作者。但 Negroponte 那個提問,最深的一層,其實從來不是「機器能不能協作」,而是:「協作,需要一個有話要說的人。」再強大的 AI,都只是一面鏡子、一個夥伴、一個放大器 — 它放大的,是你的問題、你的判斷、你的關懷。它讓「你想說什麼」,變得比以往任何時候,都更重要。

這就是數位建築的六十年:一部電腦如何一步步走進設計、又如何把「人該扮演什麼角色」這個問題,一次比一次尖銳地,丟回給我們的歷史。它沒有結局 — 因為下一章,正由每一個願意拿起這些工具、卻不忘記問「為什麼」的人,繼續寫下去。從 Sketchpad 的光筆,到你的下一個 prompt,這條線,還長著呢。

來源

1. Machine learning — Wikipedia
https://en.wikipedia.org/wiki/Machine_learning
2. Generative adversarial network (Ian Goodfellow) — Wikipedia
https://en.wikipedia.org/wiki/Generative_adversarial_network
3. Diffusion model — Wikipedia
https://en.wikipedia.org/wiki/Diffusion_model
4. Midjourney — Wikipedia
<https://en.wikipedia.org/wiki/Midjourney>
5. Stable Diffusion — Wikipedia
https://en.wikipedia.org/wiki/Stable_Diffusion
6. ArchiGAN (Stanislas Chaillou) — NVIDIA Developer Blog
<https://developer.nvidia.com/blog/archigan-generative-stack-apartment-building-design/>
7. ControlNet — Wikipedia
<https://en.wikipedia.org/wiki/ControlNet>
8. Latent space — Wikipedia
https://en.wikipedia.org/wiki/Latent_space
9. Refik Anadol — Wikipedia
https://en.wikipedia.org/wiki/Refik_Anadol
10. Neural radiance field (NeRF) — Wikipedia
https://en.wikipedia.org/wiki/Neural_radiance_field
11. AI and authorship / copyright — Wikipedia
https://en.wikipedia.org/wiki/Artificial_intelligence_and_copyright
12. Algorithmic bias — Wikipedia
https://en.wikipedia.org/wiki/Algorithmic_bias
13. The Architecture Machine (Negroponte) — MIT Press
<https://mitpress.mit.edu/9780262640107/the-architecture-machine/>